

Electronic Design *VHDL Programming*

Dr. Carlos Cruz de la Torre

University of Alcalá

Department of Electronic. Área: Tecnología Electrónica

01/07/2025

These slides are based on the book *Circuit Design with VHDL* (3rd edition) by Volnei A. Pedroni, published by MIT Press in 2020."



Creative Commons Attribution 4.0 International

Introduction to VHDL

1. Versions and purposes

VHDL = VHSIC Hardware Description Language

(VHSIC = Very High Speed ICs, USA Dept. of Defense, 1980s)

Versions:

- VHDL-87
- VHDL-93
- (VHDL-2000)
- VHDL-2002
- VHDL-2008

Purposes:

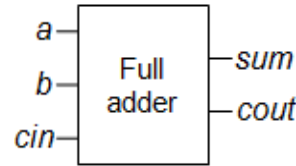
- FPGA programming
- Digital ASIC design/fabrication (masks generation)

Introduction to VHDL

1. Versions and purposes

General
concept

Desired
functionality



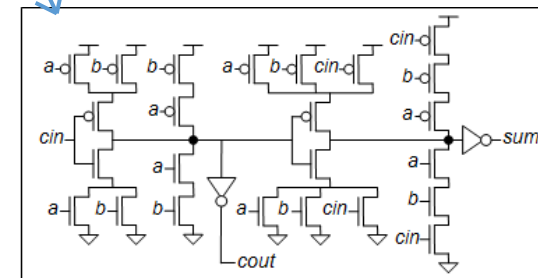
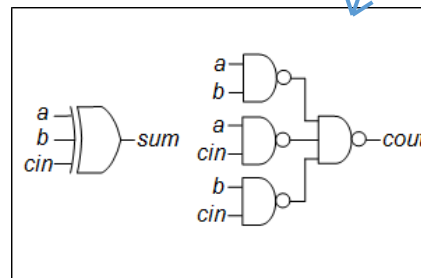
a	b	cin	sum	cout
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

Output	Equation
sum	$a \oplus b \oplus cin$
cout	$a \cdot b + a \cdot cin + b \cdot cin$

VHDL code

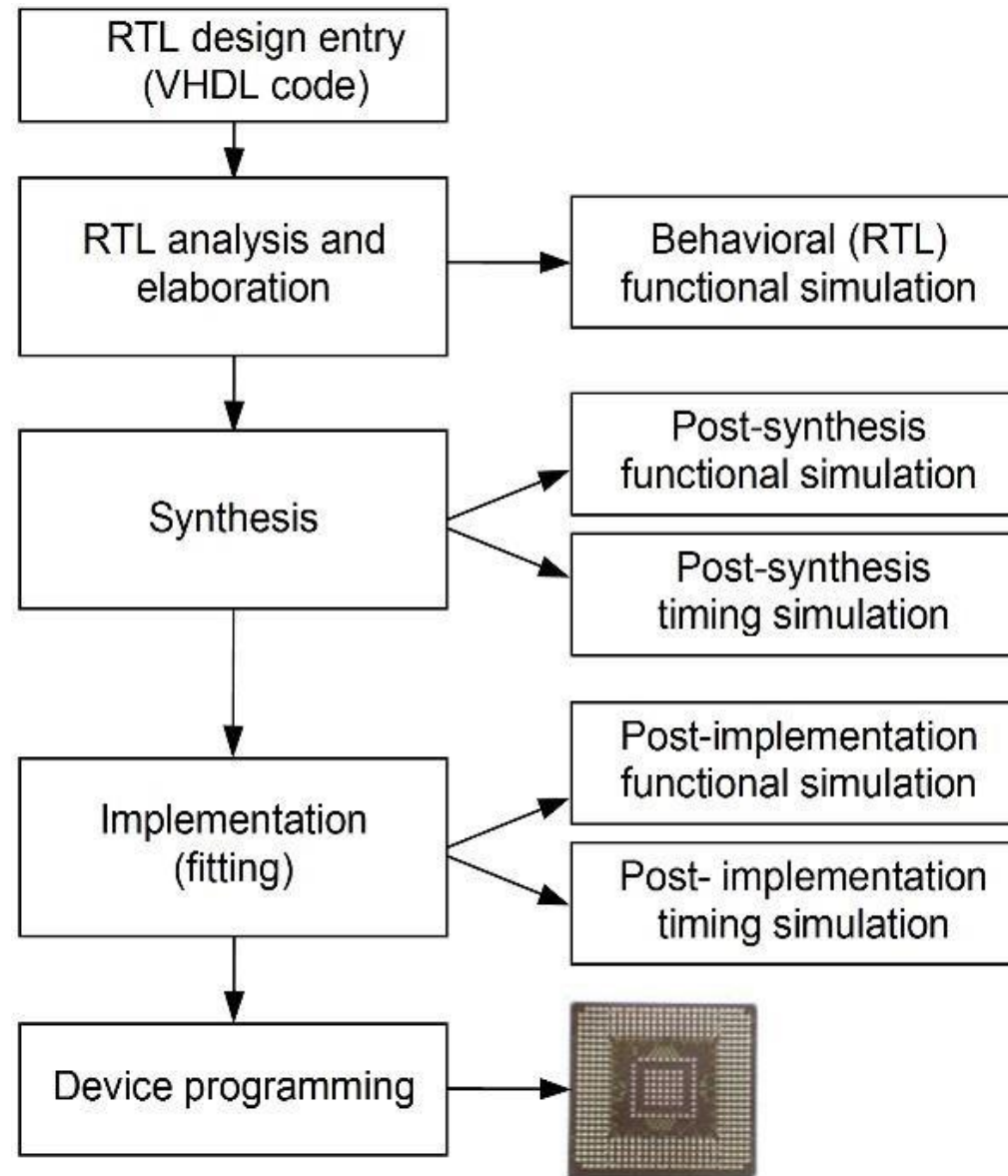
```
entity full_adder_unit is
  port (
    a, b, cin: in bit;
    sum, cout: out bit);
end entity full_adder_unit;
-----
architecture boolean_equations of full_adder_unit is
begin
  sum <= a xor b xor cin;
  cout <= (a and b) or (a and cin) or (b and cin);
end architecture boolean_equations;
```

Compliant
circuit



Introduction to VHDL

2. Simplified design flow

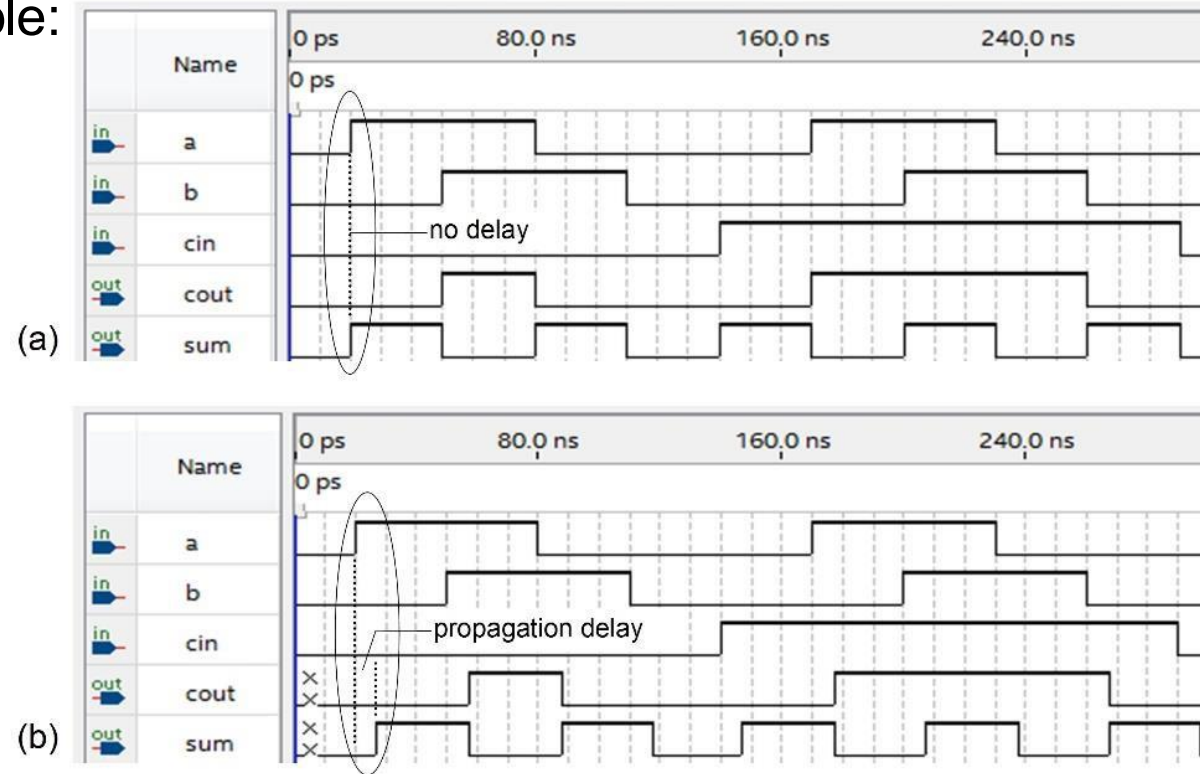


Introduction to VHDL

3. Simulation types

- **Functional simulation:** Logic behavior only
- **Timing simulation:** Propagation delays included

Example:



Introduction to VHDL

4. Concurrent versus sequential statements

- **Concurrency** is a major feature of VHDL (or any other HDL)
- So VHDL is a **code**, not a program
- And it has **statements**, not “instructions”
- Only code placed inside a **process** (or subprogram) is interpreted as in a program
- But a process, as a whole, is still **concurrent** with respect to all other statements

Examples:

These 3 pairs of statements are equivalent:

```
total <= a1 + a2*a3;  
flag <= '1' when total > LIMIT else '0';  
  
flag <= '1' when total > LIMIT else '0';  
total <= a1 + a2*a3;  
  
total <= a1 + a2*a3;  
flag <= '1' when a1 + a2*a3 > LIMIT else '0';
```

Illegal:

```
sum <= 0;  
...  
sum <= a + b;
```

OK (last value prevails):

```
process (all)  
begin  
    sum <= 0;  
    ...  
    sum <= a + b;  
end process;
```

Introduction to VHDL

5. A special data type: *std_ulogic*

Before (1st VHDL version):

Type <i>bit</i>	
'0'	Logic 0
'1'	Logic 1

After (2nd VHDL version on):

Type <i>std_ulogic</i>	
'U'	Uninitialized
'X'	Forcing unknown
'0'	Forcing 0
'1'	Forcing 1
'Z'	High impedance
'W'	Weak unknown
'L'	Weak 0
'H'	Weak 1
'_'	Don't care

Synthesis tools:

'L' = '0'
'H' = '1'
'X' = '_'

Good design practice:

- 1) Use only '0', '1', '-' for inputs
- 2) Use only '0', '1', '-', 'Z' for outputs
- 3) Use only '0', '1' for arithmetic circuits (inputs and outputs)

Introduction to VHDL

6. Lexical elements of VHDL

a) Bit and bit string

- Single quotes for single bit

Examples: '0', '1'

- Double quotes for multiple bits

Examples: "0111", b"0111", B"0111"(prefix optional for **binary**)

x"C2F", X"C2F"(prefix mandatory for **hexadecimal** and other bases)

- Underline character requires prefix, except for **integers**

Examples: "10_0010"(illegal), b"10_0010"(legal), 10_0010(legal; an integer)

- Negative values are represented in **two's complement** format (except floating-point)

Examples:

Unsigned system: "0111" (7), "1000" (8), "00000000" (= 0), "11111111" (= 255)

Signed system: "0111" (7), "1000" (−8), "00000000" (= 0), "11111111" (= −1)

Introduction to VHDL

6. Lexical elements of VHDL

b) Integers

- Default = 32 bits (so don't forget to specify range)
- Max range = -2^{31} to $2^{31}-1$

Examples: 5, -32, 3250, 3_250, 3E2(=300), 52e3(=52_000)

c) Character and character string

- 256-character ISO 8859-1 standard
- Single quotes for single character
- Double quotes for multiple characters

Examples: 'a', 'A', "timed_out"

Introduction to VHDL

6. Lexical elements of VHDL

d) Assignment symbols

- For signals: `<=`
- For variables, constants, and initial/default values: `:=`

Examples: `sig<="00000000";`
`var:=255;`

e) Comments

- Up to the end of the line: `--`
- Enclosed region (in one or multiple lines): `/* */`

Examples: `data_out<=data_in when ena else 'Z'; --tri-state buffer`

```
/*  
... (commented-out region)  
*/
```

Introduction to VHDL

6. Lexical elements of VHDL

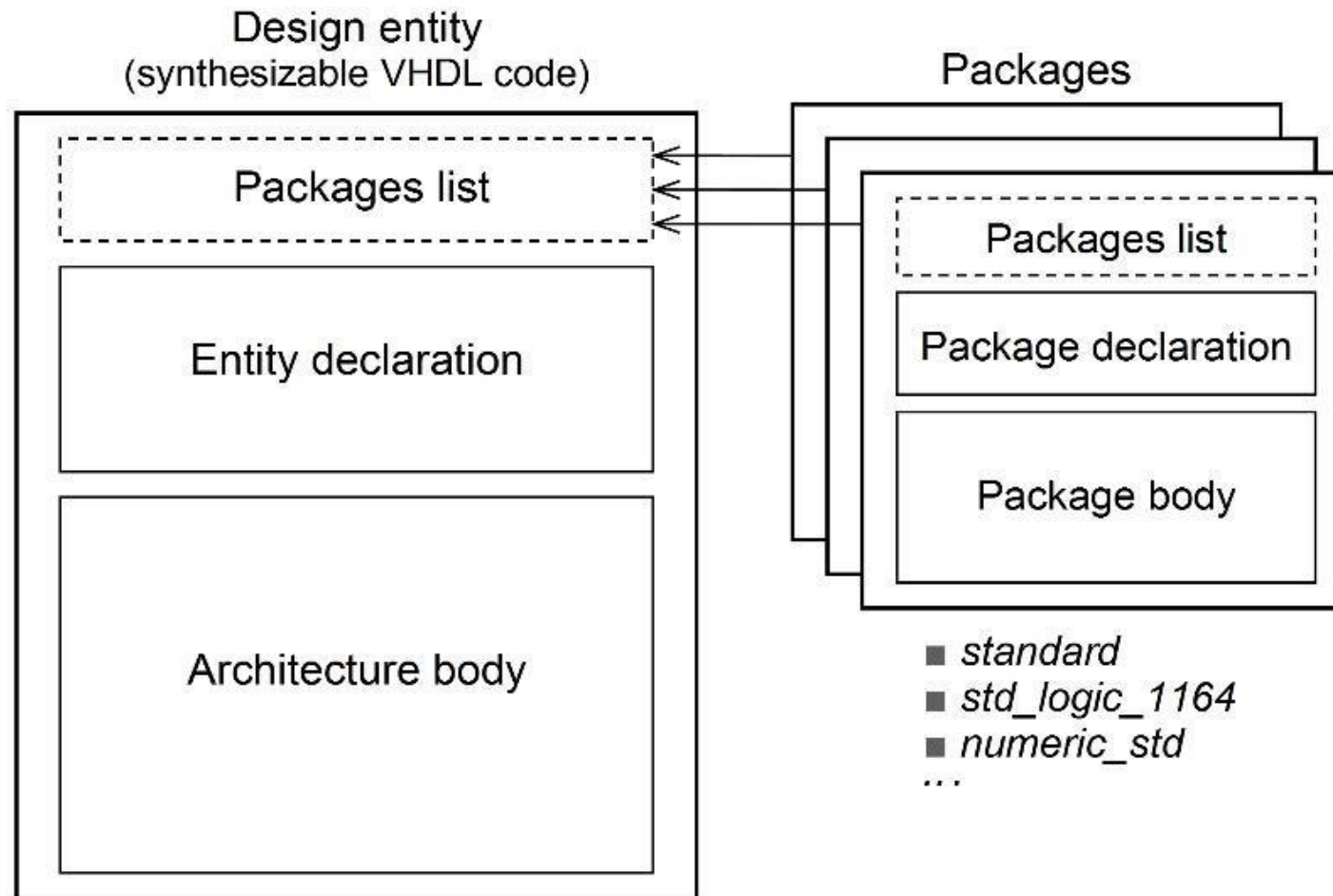
f) Identifiers (names of VHDL entities)

- Can contain only letters (a to z, A to Z), decimal digits (0 to 9), and underline character
- Must start with a letter
- Must not have two adjacent underline characters or end with an underline character
- Must not be a reserved VHDL word
- VHDL is **not** case sensitive
- Examples:
 - Legal names: `clk`, `clk_5MHz`, `data_ready`, `NUMBER_OF_BITS`
 - Illegal names: `!rst`, `ena_`, `4input_nand`, `return`

g) Delimiters and reserved words

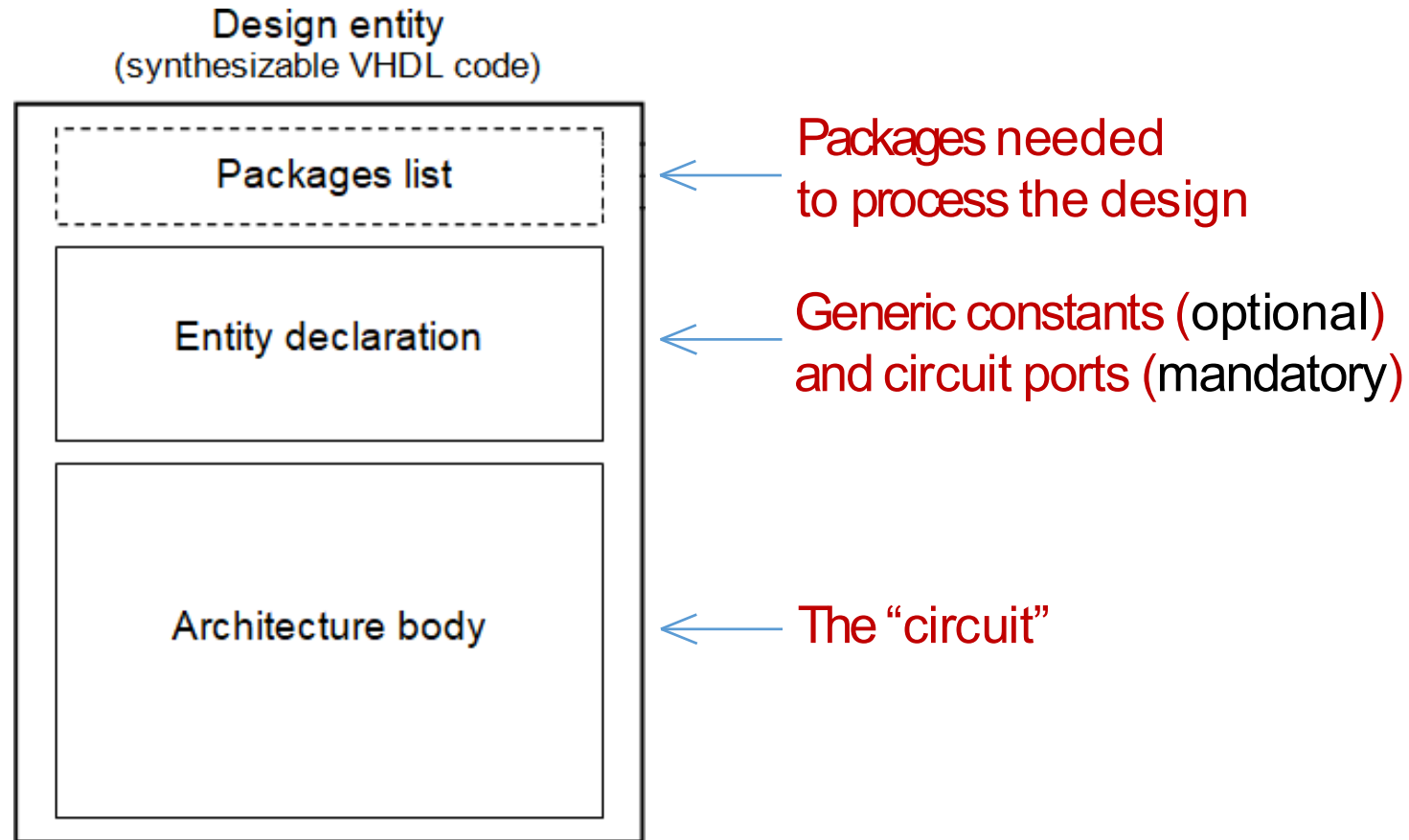
Code Structure and Composition

1. Structure of VHDL code



Code Structure and Composition

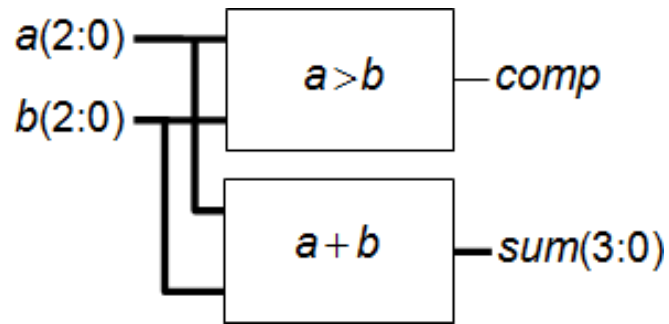
1. Structure of VHDL code



Code Structure and Composition

1. Structure of VHDL code

Example:



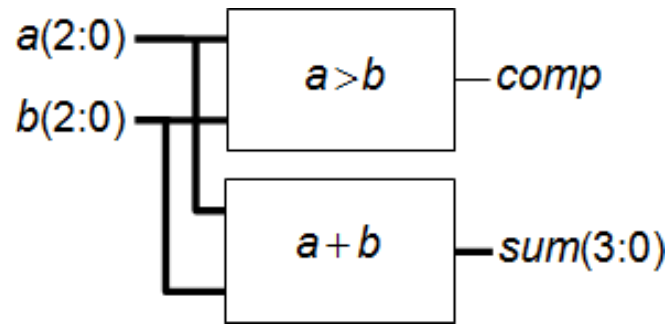
Solution 1:

```
-----  
entity add_compare_cell is  
  port (  
    a, b: in integer range 0 to 7; comp:  
    out bit;  
    sum: out integer range 0 to 15);  
end entity;  
  
architecture dataflow of add_compare_cell is begin  
  comp <= '1' when a > b else '0'; sum <= a +  
  b;  
end architecture;  
-----
```

Code Structure and Composition

1. Structure of VHDL code

Example:



Solution 2:

```
-----  
library ieee;  
use ieee.std_logic_1164.all;  
use ieee.numeric_std.all;  
  
entity add_compare_cell is  
  port (  
    a, b: in std_logic_vector(2 downto 0);  
    comp: out std_logic;  
    sum: out std_logic_vector(3 downto 0));  
end entity;  
  
architecture dataflow of add_compare_cell is signal  
  a_uns, b_uns: unsigned(3 downto 0);  
begin  
  a_uns <= unsigned('0' & a); b_uns <=  
    unsigned('0' & b);  
  comp <= '1' when a_uns > b_uns else '0'; sum <=  
    std_logic_vector(a_uns + b_uns);  
end architecture;  
-----
```

Code Structure and Composition

2. Official packages

Table 6.1. VHDL libraries and packages.

Library	Package		Typical usage
<i>std</i>	1	<i>standard</i>	For implementing all sorts of logic and integer-based arithmetic circuits, but with major limitations; not recommended for arithmetic circuits and circuit ports
	2	<i>textio</i>	For dealing with text and files
	3	<i>env</i>	For communication with the simulation environment
<i>ieee</i>	4	<i>std_logic_1164</i>	For implementing any logic or arithmetic circuit (for the latter, must associate with another package, like #7, #10, or #13)
	5	<i>std_logic_textio</i>	Complement to package #2
	6	<i>numeric_bit</i>	For implementing integer arithmetic circuits with type unsigned or signed, having bit as base type
	7	<i>numeric_std</i>	Same as above, but with std_ulogic as base type
	8	<i>numeric_bit_unsigned</i>	For doing unsigned operations with type bit_vector
	9	<i>numeric_std_unsigned</i>	Same as above, for type std_ulogic_vector
	10	<i>fixed_pkg</i>	For implementing fixed-point arithmetic circuits
	11	<i>fixed_generic_pkg</i>	
	12	<i>fixed_float_types</i>	
	13	<i>float_pkg</i>	For implementing floating-point arithmetic circuits (associated with package #12)
	14	<i>float_generic_pkg</i>	
	15	<i>math_real</i>	For determining generic parameters (support not required for synthesis, but might exist for real values that are static)
	16	<i>math_complex</i>	

Code Structure and Composition

3. Package declaration

```
library library_name;  
use library_name.package_name.all;
```

Included by default:

```
library std, work;  
use std.standard.all;
```

Examples of needed (non-default) declarations:

```
library ieee;  
use ieee.std_logic_1164.all;  
use ieee.numeric_std.all;  
use work.functions_pkg.all;  
use work.procedures_pkg.sort;
```

Code Structure and Composition

4. Entity declaration

```
entity entity_name is
```

```
  [generic (
```

```
    constant_name: constant_type [:= constant_value];
```

```
    constant_name: constant_type [:= constant_value];
```

```
  ...);]
```

```
  port (
```

```
    port_name: port_mode port_type;
```

```
    port_name: port_mode port_type;
```

```
  ...);
```

```
end [entity] [entity_name];
```

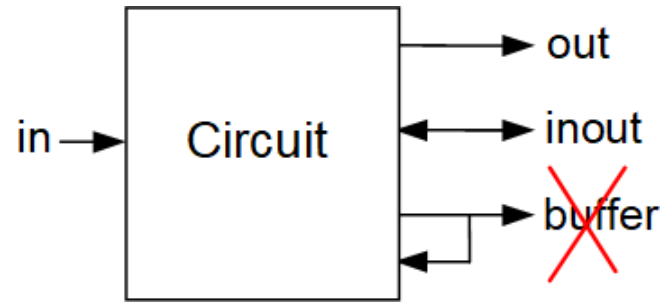
← Generic constants
(optional)

← Circuit ports
(mandatory for
synthesis)

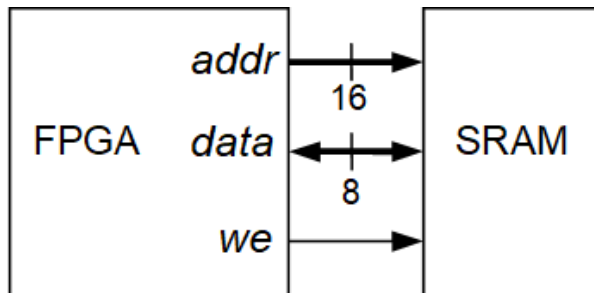
Code Structure and Composition

4. Entity declaration

Port modes: *in*, *out*, *inout*



Example:

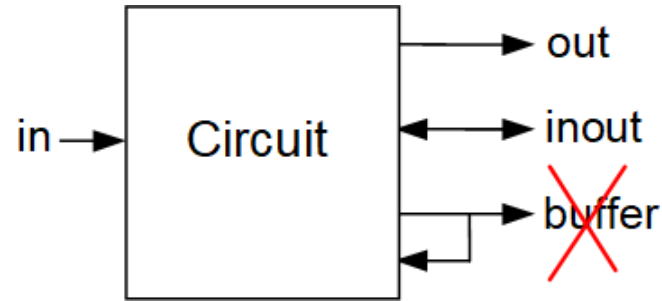


```
entity SRAM_interface is
  port (
    we: out std_logic;
    addr: out std_logic_vector(15 downto 0);
    data: inout std_logic_vector(7 downto 0));
end entity;
```

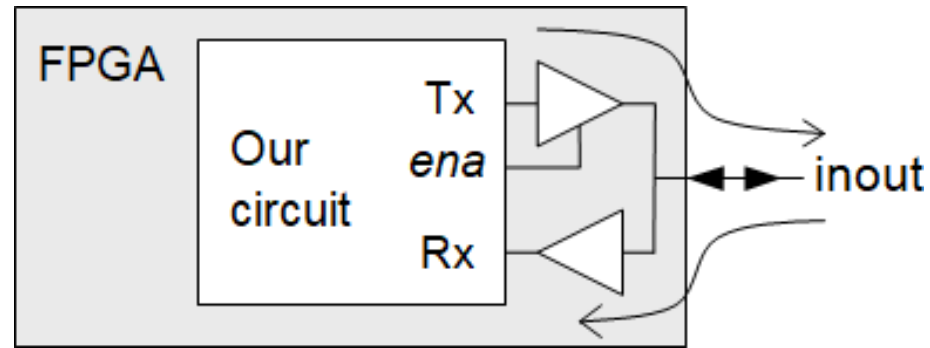
Code Structure and Composition

4. Entity declaration

Port modes: *in*, *out*, *inout*



How is *inout* constructed?

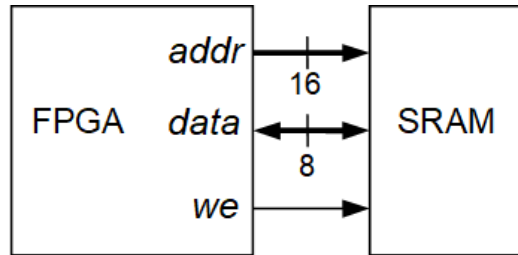


Code Structure and Composition

4. Entity declaration

Generic constants

Example:



Without generics:

```
entity SRAM_interface is
    port (
        we: out std_logic;
        addr: out std_logic_vector(15 downto 0); data: inout
            std_logic_vector(7 downto 0));
end entity;
```

With generics:

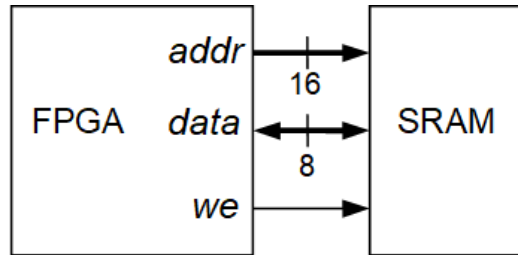
```
entity SRAM_interface is
    generic (
        ADDR_WIDTH: natural := 16;
        DATA_WIDTH: natural := 8);
    port (
        we: out std_logic;
        addr: out std_logic_vector(...???.);
        data: inout std_logic_vector(...???.));
end entity;
```

Code Structure and Composition

4. Entity declaration

Generic constants

Example:



Without generics:

```
entity SRAM_interface is
  port (
    we: out std_logic;
    addr: out std_logic_vector(15 downto 0); data: inout
    std_logic_vector(7 downto 0));
end entity;
```

With generics:

```
entity SRAM_interface is
  generic (
    ADDR_WIDTH: natural := 16;
    DATA_WIDTH: natural := 8);
  port (
    we: out std_logic;
    addr: out std_logic_vector(ADDR_WIDTH-1 downto 0); data: inout
    std_logic_vector(DATA_WIDTH-1 downto 0));
end entity;
```

Code Structure and Composition

4. Entity declaration

Generic constants

VHDL-2008:

- Expressions allowed in generic declarations (helpful)
- Type declarations allowed but require instantiation (not helpful in general)

Example: **generic** (
 NUM_BITS: natural := 16; DEPTH: natural :=
 2**NUM_BITS
);

Code Structure and Composition

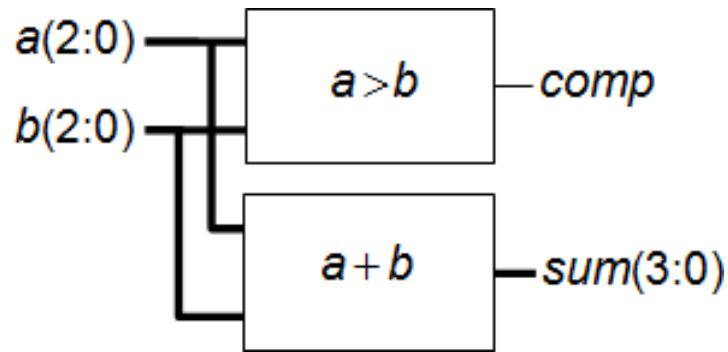
5. Architecture body

```
architecture architecture_name of entity_name is
    [architecture_declarative_part]
begin
    architecture_statement_part
end [architecture] [architecture_name];
```

← Declarative region
(optional)

← Statements region
(mandatory)

Example (seen earlier):



```
-----  
architecture dataflow of add_compare_cell is begin  
    comp <= '1' when a > b else '0'; sum <= a +  
    b;  
end architecture;  
-----
```


Code Structure and Composition

6. Object classes

- **Object:** a named item that has a value of a type
- There are four kinds of objects (or object classes) in VHDL:
 - Constant
 - Signal
 - Variable
 - File (for simulation)

Code Structure and Composition

6. Object classes

Constant:

```
constant constant_name: constant_type := constant_value;
```

Examples:

```
constant CLK_FREQ_MHZ: natural := 150;
```

```
constant MASK: std_logic_vector(7 downto 0) := "00001111";
```

```
constant BIT_MATRIX: bit_matrix_type := (('1','1','0'),  
                                           ('0','1','1'),  
                                           ('1','0','0'));
```

Code Structure and Composition

6. Object classes

Signal:

```
signal signal_name: signal_type [:= default_value];
```

Examples:

```
--Signal declarations:  
signal rst, ena: std_logic;  
signal count: natural range 0 to 255;  
  
--Signal assignments:  
if rst then reg <= (others => '0');  
outp <= count when ena else 0;
```

Code Structure and Composition

6. Object classes

Variable:

```
variable variable_name: variable_type [:= initial_value];
```

Example: **variable** count: natural **range** 0 **to** 2**DEPTH-1;
...
if rising_edge(clk) **then**
 count := count + 1;
end if;

Code Structure and Composition

6. Object classes

Additional notes:

- Shared variables: **Avoid** them for synthesis (and even for simulation)
- Signal x Variable:
- Initial/default values: Why are they **not** synthesizable?

Code Structure and Composition

Example Parity detector

Example. D-type flip-flop (DFF)

Example. Registered add-and-compare



```
1  -----
2  library ieee;
3  use ieee.std_logic_1164.all;
4
5  entity flip_flop is
6      port (
7          d, clk, rst: in std_logic;
8          q: out std_logic);
9  end entity;
10
11 architecture flip_flop of flip_flop is
12 begin
13     process (clk, rst)
14     begin
15         if rst then
16             q <= '0';
17         elsif rising_edge(clk) then
18             q <= d;
19         end if;
20     end process;
21 end architecture;
22 -----
```

Predefined Data Types

1. Main *synthesizable* types

Category	Types	Packages
Standard types	<code>bit</code> , <code>bit_vector</code> <code>boolean</code> , <code>boolean_vector</code> <code>integer</code> , <code>natural</code> , <code>positive</code> , <code>integer_vector</code> <code>character</code> , <code>string</code>	<i>std</i>
Standard-logic types	<code>std_ulogic</code> , <code>std_ulogic_vector</code> <code>std_logic</code> , <code>std_logic_vector</code>	<i>std_logic_1164</i>
Unsigned/signed types	<code>unsigned</code> , <code>signed</code>	<i>numeric_std</i>
Fixed-point types	<code>ufixed</code> , <code>sfixed</code>	<i>fixed_generic_pkg</i> (<i>fixed_pkg</i>)
Floating-point type	<code>float</code>	<i>float_generic_pkg</i> (<i>float_pkg</i>)

Predefined Data Types

2. Type declarations (how types are created)

For scalar types:

```
type type_name is type_definition;
```

Examples (of original VHDL declarations):

```
type bit is ('0','1');
```

```
type integer is range -2147483648 to 2147483647;
```

For array types:

```
type type_name is array (range_spec) of base_type_name [range_spec].
```

Examples (of original VHDL declarations):

```
type bit_vector is array (natural range <>) of bit;
```

```
type integer_vector is array (natural range <>) of integer;
```


Predefined Data Types

3. Subtypes

- Subtype = A type with a constraint
- Advantage (over a new type): Inherits all properties (operators and other functions) of parent type

Subtype declaration:

```
subtype subtype_name is [resolution_function] type_name [range specification];
```

Example (of original VHDL declaration):

```
type integer is range -2147483648 to 2147483647; --range  $-2^{31}$  to  $2^{31}-1$   
subtype natural is integer range 0 to integer'high; --range 0 to  $2^{31}-1$ 
```

Predefined Data Types

4. Record types

- Can have multiple members
- Elements can be of different types
- Useful to manipulate several objects together
- Particularly helpful in [simulation](#) (e.g. example 18.2)

Example:

–Record type declaration:

```
type memory_access is      record word_addr:  
    natural range          0 to 4095; data:  
    std_logic_vector(15 downto 0);  
end record;
```

–Record type usage:

```
signal ram_interface: memory_access;
```

```
...
```

```
if ram_interface.word_addr=4095 then
```

```
...
```

Predefined Data Types

5. Description of predefined types, one-by-one

Category	Types	Abbreviations
Standard types	bit, bit_vector boolean, boolean_vector integer, natural, positive, integer_vector character, string	B, BV BO, BOV INT, NAT, POS, INTV CHAR, STR
Standard-logic types	std_ulogic, std_ulogic_vector std_logic, std_logic_vector	SU, SUV SL, SLV
Unsigned/signed types	unsigned signed	UNS SIG
Fixed-point types	ufixed sfixed	UFIX SFIX
Floating-point type	float	FLO

Predefined Data Types

5. Description of predefined types, one-by-one

a) Standard types

<code>bit</code> (2 values)	<code>x <= '0'; y <= '1';</code>
<code>bit_vector</code>	<code>x <= "1101"; y <= b"101_0000";</code>
<code>boolean</code> (2 values)	<code>x <= false; y <= true;</code>
<code>boolean_vector</code>	<code>x <= (false, true, false);</code>
<code>integer</code> (32 bits)	<code>x <= -255; y <= 1111;</code>
<code>natural</code>	<code>x <= 0;</code>
<code>positive</code>	<code>x <= 255; fclk <= 50_000_000;</code>
<code>integer_vector</code>	<code>x <= (0, 1, 2, -3);</code>
<code>character</code> (8 bits)	<code>x <= 'a';</code>
<code>string</code>	<code>x <= "VHDL";</code>

Predefined Data Types

5. Description of predefined types, one-by-one

b) Standard-logic types

Type <code>std_ulogic</code>	
'U'	Uninitialized
'X'	Forcing unknown
'0'	Forcing 0
'1'	Forcing 1
'Z'	High impedance
'W'	Weak unknown
'L'	Weak 0
'H'	Weak 1
'_'	Don't care

`std_ulogic` (type) `std_logic`
(subtype)

`std_ulogic_vector` (type)
`std_ulogic_vector` (subtype)

- Subtypes above = resolved versions of types above
- Resolution table: see [section 7.6.2](#)
- These are the most common types in the industry

Predefined Data Types

5. Description of predefined types, one-by-one

b) Standard-logic types

Examples with `std_ulogic` or `std_logic`:

```
x <= '0';  
y <= '-';  
z <= 'Z';
```

Examples with `std_ulogic_vector` or `std_logic_vector`:

```
v <= "1101";  
x <= b"1101_—";  
y <= "011-1-0";  
z <= (others => 'Z');
```

Predefined Data Types

5. Description of predefined types, one-by-one

b) Standard-logic types

Synthesis tools:

'L' = '0'

'H' = '1'

'X' = '—'

Good design practice:

- 1) Use only ...???... for inputs
- 2) Use only ...???... for outputs
- 3) Use only ...???... for arithmetic circuits (inputs and outputs)

Predefined Data Types

5. Description of predefined types, one-by-one

b) Standard-logic types

Synthesis tools:

'L' = '0'

'H' = '1'

'X' = '—'

Good design practice:

1) Use only '0', '1', '—' for inputs

2) Use only '0', '1', '—', 'Z' for outputs

3) Use only '0', '1' for arithmetic circuits (inputs and outputs)

Predefined Data Types

5. Description of predefined types, one-by-one

b) Standard-logic types

Synthesis tools:

'L' = '0'

'H' = '1'

'X' = '—'

Good design practice:

- 1) Use only '0', '1', '—' for inputs
- 2) Use only '0', '1', '—', 'Z' for outputs
- 3) Use only '0', '1' for arithmetic circuits (inputs and outputs)
- 4) But for arithmetic, use **SUV/SLV** only for ports; for computations, adopt:
 - For integers: **UNS/SIG**
 - For fixed-point: **UFIX/SFIX**
 - For floating-point: **FLOAT**

Predefined Data Types

5. Description of predefined types, one-by-one

c) Unsigned and signed types

- `unsigned(L downto R)` or `unsigned(L to R)`
- `signed(L downto R)` or `signed(L to R)`
- Represent **integers**
- Based on `std_ulogic`
- `signed` employs two's complement for negative values

Recommendations:

- Use these types for implementing **integer arithmetic circuits**
- Use only values '0' and '1'
- Use only `R=0`

Predefined Data Types

5. Description of predefined types, one-by-one

c) Unsigned and signed types

Examples:

```
signal x: unsigned(5 downto 0);
```

...

```
x <= "100000";
```

--?

```
x <= "10_0000";
```

--?

```
x <= b"10_0000";
```

--?

```
x <= (others => '1');
```

--?

Predefined Data Types

5. Description of predefined types, one-by-one

c) Unsigned and signed types

Examples:

```
signal x: unsigned(5 downto 0);
```

...

```
x <= "100000"; x <= -- 32
```

```
"10_0000"; -- illegal
```

```
x <= b"10_0000"; -- 32
```

```
x <= (others => '1'); -- 63
```

Predefined Data Types

5. Description of predefined types, one-by-one

c) Unsigned and signed types

Examples:

```
signal x: unsigned(5 downto 0);
```

...

```
x <= "100000"; x <= -- 32
```

```
"10_0000"; -- illegal
```

```
x <= b"10_0000"; -- 32
```

```
x <= (others => '1'); -- 63
```

```
signal y: signed(5 downto 0);
```

...

```
y <= "100000"; -- ?
```

```
y <= b"10_0000"; -- ?
```

```
y <= (others => '1'); -- ?
```

Predefined Data Types

5. Description of predefined types, one-by-one

c) Unsigned and signed types

Examples:

```
signal x: unsigned(5 downto 0);
```

...

```
x <= "100000"; x <= -- 32
```

```
"10_0000"; -- illegal
```

```
x <= b"10_0000"; -- 32
```

```
x <= (others => '1'); -- 63
```

```
signal y: signed(5 downto 0);
```

...

```
y <= "100000"; -- -32
```

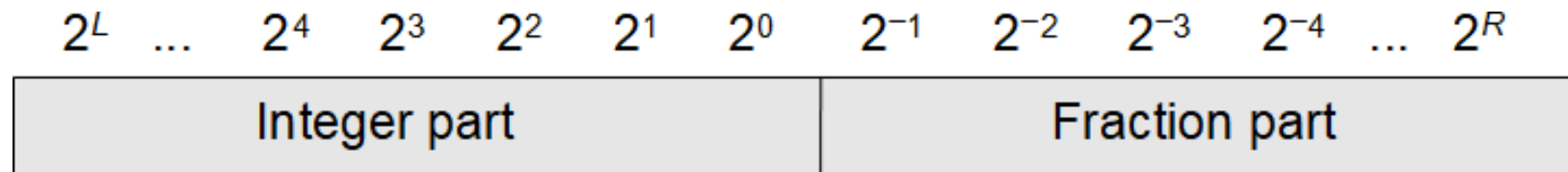
```
y <= b"10_0000"; -- -32
```

```
y <= (others => '1'); -- -1
```

Predefined Data Types

5. Description of predefined types, one-by-one

d) Fixed-point types



- `ufixed(L downto R)`
- `sfixed(L downto R)`
- Based on `std_ulogic`
- Employs two's complement for negative values

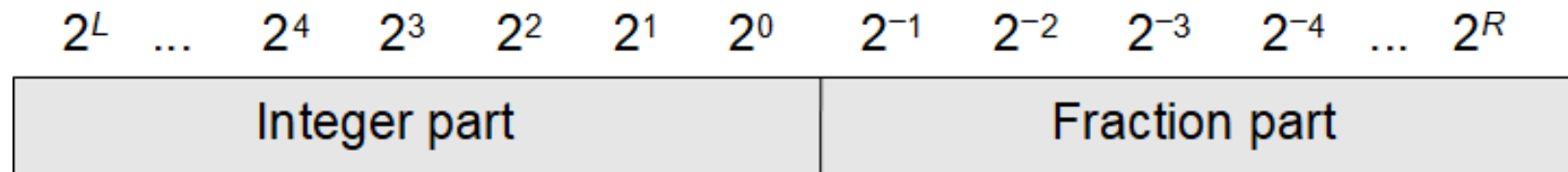
Recommendations:

- Use these types for implementing **fixed-point arithmetic circuits**
- Use only values '0' and '1'
- Remember that here $L > R$ and usually $R < 0$

Predefined Data Types

5. Description of predefined types, one-by-one

d) Fixed-point types



Examples (with $L=3$ and $R=-2$):

signal x: ufixed(3 downto -2);

x <= "100101"; $-2^3 + 2^0 + 2^{-2} = 9.25$ or $\text{uns}/2^{-R} = 37/4 = 9.25$

x <= b"1001_01"; --same

signal y: sfixed(3 downto -2);

y <= "100101"; $-\text{ufix} - 2^{(L+1)} = 9.25 - 16 = -6.75$ or $(\text{uns} - 2^{\text{BITS}})/2^{-R} = -6.75$

y <= b"1001_01"; --same

Predefined Data Types

5. Description of predefined types, one-by-one

e) Floating-point types

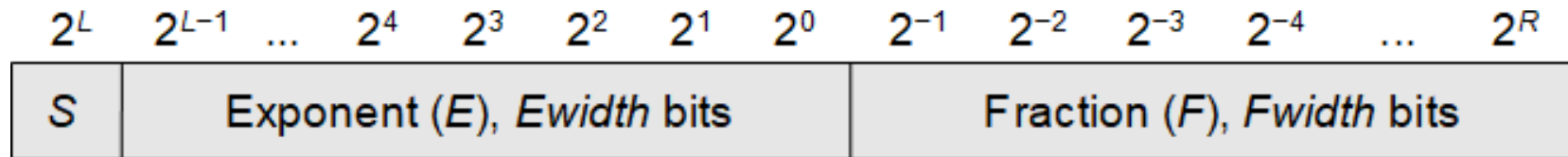
Review section [1.6.5 Floating-Point Arithmetic](#):

- IEEE standard
- Legal values
- Corresponding decimal value
- Rounding options
- Overflow, saturation, ...

Predefined Data Types

5. Description of predefined types, one-by-one

e) Floating-point types



- float32
- float64
- float128
- float(L **downto** R)
- Based on `std_ulogic`
- Employs **sign bit** (no two's complement here)

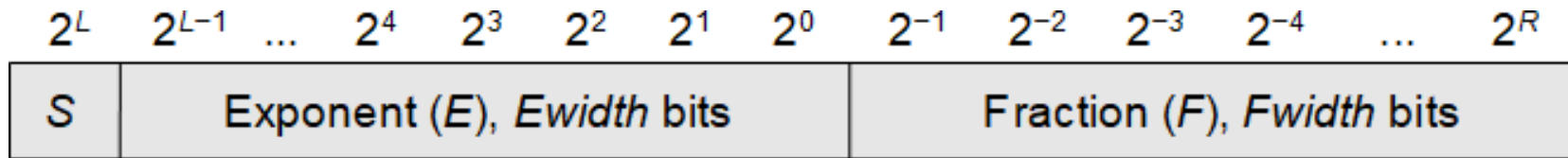
Recommendations:

- Use these types for implementing **floating-point arithmetic circuits**
- Use only values '0' and '1'
- Remember that here **L > R** and **R < 0**

Predefined Data Types

5. Description of predefined types, one-by-one

e) Floating-point types



Decimal
value:

$$dec = (-1)^S (1+F) 2^{E-BIAS}$$

Where:

$$BIAS = 2^{Ewidth-1} - 1$$

Example (with $L=3$ and $R=-4$):

```
signal x: float(3 downto -4);
x <= "10100110";
```

Decimal value of x:

$$x = (S=1)(E=010)(F=0110) = -(1+0.375)2^{2-3} = -0.6875$$

Predefined Data Types

5. Description of predefined types, one-by-one

f) Type *real*

- Not synthesizable in general
- But can be used as part of expressions whose output is synthesizable
- Requires package *math_real*

Example: Number of bits needed to represent an integer

```
library ieee;  
use ieee.math_real.all;  
  
...  
signal count: natural range 0 to MAX;  
constant NUM_BITS: natural := integer(ceil(log2(real(MAX+1))));
```

Predefined Data Types

5. Description of predefined types, one-by-one

Closing example...

First, recall the main synthesizable types:

Category	Types	Abbreviations
Standard types	bit, bit_vector boolean, boolean_vector integer, natural, positive, integer_vector character, string	B, BV BO, BOV INT, NAT, POS, INTV CHAR, STR
Standard-logic types	std_ulogic, std_ulogic_vector std_logic, std_logic_vector	SU, SUV SL, SLV
Unsigned/signed types	unsigned signed	UNS SIG
Fixed-point types	ufixed sfixed	UFIX SFIX
Floating-point type	float	FLO

Predefined Data Types

5. Description of predefined types, one-by-one

Closing example... Which are the possible types of the data below?

```
x1 <= '1';  
x2 <= false;  
x3 <= "false";  
x4 <= "0-00000-";  
x5 <= 011011;  
x6 <= 50_000_000;  
x7 <= (others=>'Z');  
x8 <= ('1','1','0','1','0');  
x9 <= "11" & "010";
```

Predefined Data Types

5. Description of predefined types, one-by-one

Closing example... Which are the possible types of the data below?

x1 <= '1'; --B, SU/SL, CHAR

x2 <= false;

x3 <= "false";

x4 <= "0-00000-";

x5 <= 011011;

x6 <= 50_000_000;

x7 <= (others=>'Z');

x8 <= ('1','1','0','1','0');

x9 <= "11" & "010";

Predefined Data Types

5. Description of predefined types, one-by-one

Closing example... Which are the possible types of the data below?

x1 <= '1'; --B, SU/SL, CHAR

x2 <= false; --BO

x3 <= "false";

x4 <= "0-00000-";

x5 <= 011011;

x6 <= 50_000_000;

x7 <= (others=>'Z');

x8 <= ('1','1','0','1','0');

x9 <= "11" & "010";

Predefined Data Types

5. Description of predefined types, one-by-one

Closing example... Which are the possible types of the data below?

x1 <= '1'; --B, SU/SL, CHAR

x2 <= false; --BO

x3 <= "false"; --STR

x4 <= "0-00000-";

x5 <= 011011;

x6 <= 50_000_000;

x7 <= (others=>'Z');

x8 <= ('1','1','0','1','0');

x9 <= "11" & "010";

Predefined Data Types

5. Description of predefined types, one-by-one

Closing example... Which are the possible types of the data below?

x1 <= '1'; --B, SU/SL, CHAR

x2 <= false; --BO

x3 <= "false"; --STR

x4 <= "0-00000-"; --STR, SUV/SLV, UNS/SIG, UFIX/SFIX, FLOAT

x5 <= 011011;

x6 <= 50_000_000;

x7 <= (others=>'Z');

x8 <= ('1','1','0','1','0');

x9 <= "11" & "010";

Predefined Data Types

5. Description of predefined types, one-by-one

Closing example... Which are the possible types of the data below?

x1 <= '1'; --B, SU/SL, CHAR

x2 <= false; --BO

x3 <= "false"; --STR

x4 <= "0-00000-"; --STR, SUV/SLV, UNS/SIG, UFIX/SFIX, FLOAT

x5 <= 011011; --INT, NAT, POS

x6 <= 50_000_000;

x7 <= (others=>'Z');

x8 <= ('1','1','0','1','0');

x9 <= "11" & "010";

Predefined Data Types

5. Description of predefined types, one-by-one

Closing example... Which are the possible types of the data below?

x1 <= '1'; --B, SU/SL, CHAR

x2 <= false; --BO

x3 <= "false"; --STR

x4 <= "0-00000-"; --STR, SUV/SLV, UNS/SIG, UFIX/SFIX, FLOAT

x5 <= 011011; --INT, NAT, POS

x6 <= 50_000_000; --same as x5

x7 <= (others=>'Z'); --same as x4

x8 <= ('1','1','0','1','0');

x9 <= "11" & "010";

Predefined Data Types

5. Description of predefined types, one-by-one

Closing example... Which are the possible types of the data below?

x1 <= '1'; --B, SU/SL, CHAR

x2 <= false; --BO

x3 <= "false"; --STR

x4 <= "0-00000-"; --STR, SUV/SLV, UNS/SIG, UFIX/SFIX, FLOAT

x5 <= 011011; --INT, NAT, POS

x6 <= 50_000_000; --same as x5

x7 <= (others=>'Z'); --same as x4

x8 <= ('1','1','0','1','0'); --same as x4 plus BV

x9 <= "11" & "010"; --same as x8

Predefined Data Types

5. Description of predefined types, one-by-one

Closing example... Which are the possible types of the data below?

x1 <= '1'; --B, SU/SL, CHAR

x2 <= false; --BO

x3 <= "false"; --STR

x4 <= "0-00000-"; --STR, SUV/SLV, UNS/SIG, UFIX/SFIX, FLOAT

x5 <= 011011; --INT, NAT, POS

x6 <= 50_000_000; --same as x5

x7 <= (others=>'Z'); --same as x4

x8 <= ('1','1','0','1','0'); --same as

x4 plus BV

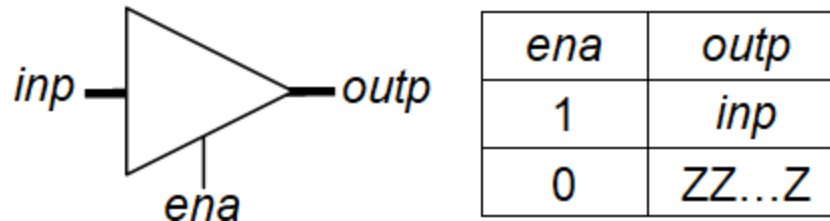
x9 <= "11" & "010"; --same as x8

Which options should be eliminated and why? Use only '0' and '1' for arithmetic!

Predefined Data Types

6. Application examples for *standard-logic* types

1) Why does this circuit need a *standard-logic* type?

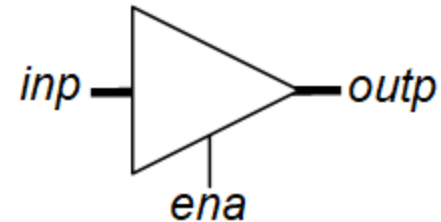


Because of the **high-impedance** state ('Z')

Predefined Data Types

1) Why does this circuit need a [standard-logic](#) type?

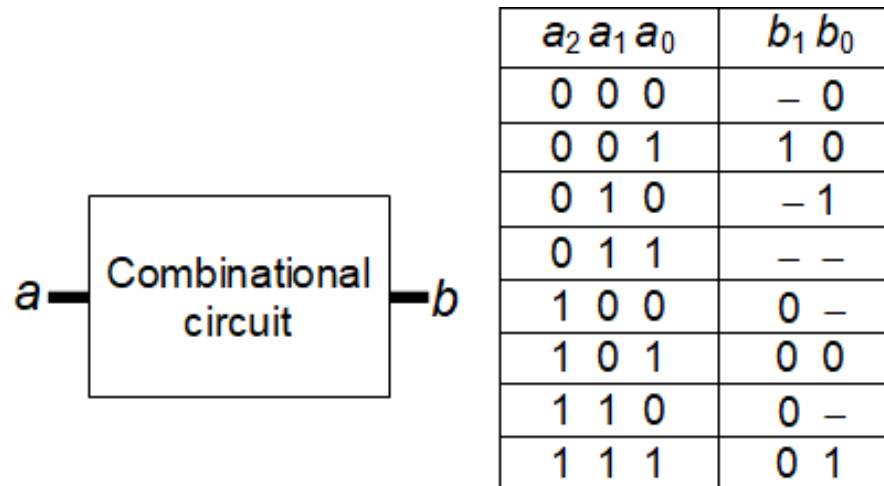
```
1  -----
2  library ieee;
3  use ieee.std_logic_1164.all;
4
5  entity tri_state_buffer is
6      generic (
7          NUM_BITS: natural := 16);
8      port (
9          inp: in std_logic_vector(NUM_BITS-1 downto 0);
10         ena: in std_logic;
11         outp: out std_logic_vector(NUM_BITS-1 downto 0));
12  end entity;
13
14  architecture tri_state of tri_state_buffer is
15  begin
16      outp <= inp when ena else (others => 'Z');
17  end architecture;
18  -----
```



Predefined Data Types

6. Application examples for *standard-logic* types

2) Why does this circuit need a *standard-logic* type?



Because of the **don't care** state ('–')

Predefined Data Types

6. Application examples for *standard-logic* types

2) Why does this circuit need a *standard-logic* type?

```
1  -----
2  library ieee;
3  use ieee.std_logic_1164.all;
4
5  entity circuit_with_dontcare is
6      port (
7          a: in std_logic_vector(2 downto 0);
8          b: out std_logic_vector(1 downto 0));
9  end entity;
10
11  architecture truth_table of circuit_with_dontcare is
12  begin
13      with a select
14          b <= "-0" when "000",
15              "10" when "001",
16              "-1" when "010",
17              "--" when "011",
18              "0-" when "100" | "110",
19              "00" when "101",
20              "01" when others;
21  end architecture;
22  -----
```

Predefined Data Types

7. Type conversion

Three options:

- Automatic Conversion
- Type Cast
- Type-Conversion Functions

a) Automatic Conversion

Occurs when dealing directly with the **base** type

Examples: `bv(0)`

`<= b; sl <= slv(7);`

--single element of type `bit` on both sides

--single element of type `std_logic` on both sides

Predefined Data Types

7. Type conversion

Three options:

- Automatic Conversion
- Type Cast
- Type-Conversion Functions

b) Type cast

- Between integer and real
- Between std_(u)logic_vector, unsigned, and signed

Examples:

```
slv <= std_logic_vector(uns);
```

--type cast from UNS to SLV

```
uns <= unsigned(slv);
```

--type cast from SLV to UNS

```
sig <= signed(uns);
```

--type cast from UNS to SIG

```
int <= integer(re);
```

--type cast from RE to INT

Predefined Data Types

7. Type conversion

Three options:

- Automatic Conversion
- Type Cast
- Type-Conversion Functions

c) With a type-conversion function

- See table next

Examples:

```
uns <= to_unsigned(int, 8); int <=
to_integer(uns);
sfix <= to_sfixed(int, 3, -4);
```

```
--INT to UNS, with 8 bits
--UNS to INT, with same # of bits
--INT to SFIX, with L=3 and R=-4
```

Predefined Data Types

7. Type conversion

Three options:

- Automatic Conversion
- Type Cast
- Type-Conversion Functions

Table 7.10

Predefined type-conversion functions for synthesizable types and subtypes

From type	To type	Type-conversion function		Package of origin
integer, natural, positive	std_ulogic_vector	1	<i>to_std_ulogic_vector(arg, width)</i>	<i>numeric_std_unsigned</i>
	std_logic_vector	2	<i>to_std_logic_vector(arg, width)</i>	<i>numeric_std_unsigned</i>
	unsigned	3	<i>to_unsigned(arg, width)</i>	<i>numeric_std</i>
	signed	4	<i>to_signed(arg, width)</i>	<i>numeric_std</i>
	ufixed	5	<i>to_ufixed(arg, L, R)</i>	<i>fixed_generic_pkg</i>
	sfixed	6	<i>to_sfixed(arg, L, R)</i>	<i>fixed_generic_pkg</i>
	float	7	<i>to_float(arg, Ewidth, Fwidth)</i>	<i>float_generic_pkg</i>
	real	8	Type cast: <i>real(arg)</i>	<i>math_real</i>
bit	std_ulogic, std_logic	9	<i>to_stdulogic(arg)</i>	<i>std_logic_1164</i>
bit_vector	std_ulogic_vector	10	<i>to_stdulogicvector(arg)</i>	<i>std_logic_1164</i>
	std_logic_vector	11	<i>to_stdlogicvector(arg)</i>	<i>std_logic_1164</i>
std_ulogic,	bit	12	<i>to_bit(arg)</i>	<i>std_logic_1164</i>

Predefined Data Types

7. Type conversion

Aspecial case: How to convert **INT** ↔ **SLV/SUV**?

```
use ieee.std_logic_1164.all;
```

```
use ieee.numeric_std.all;
```

```
...
```

```
slv <= std_logic_vector(to_signed(int,8));
```

–from INT to SLV

```
int <= to_integer(signed(slv));
```

–from SLV to INT

(assumed that package *numeric_std_unsigned* is not supported)

Predefined Data Types

8. Aggregation, concatenation, and resizing

a) Aggregation

- Single-values, written between parentheses
- Separated by comma
- Keyword *others* often helpful
- Multi-value pieces allowed in VHDL 2008

Examples (with *positional* and *named* associations):

```
signal s1, s2, s3, s4, s5, s6: std_logic_vector(5 downto 0);
...
s1 <= ('1', '0', '0', '0', '1', '0');           --s1="100010"
s2 <= ('1', '0', '0', others => 'Z');           --s2="100ZZZ"
s3 <= ('0', '-', others => s1(5));              --s3="0-1111"
s4 <= (1 => '1', 5 downto 2 => '0', 0 => '-');   --s4="00001-"
s5 <= (5|1 => '1', others => '0');              --s5="100010"
s6 <= ('1', "000", "11");                      --s6="100011" (VHDL 2008)
```


Predefined Data Types

8. Aggregation, concatenation, and resizing

b) Concatenation

- Uses “&” operator
- Parentheses are optional (might improve readability)
- Keyword *others* not allowed
- Concatenation of aggregates is legal

Examples:

```
signal s1, s2, s3: std_logic_vector(5 to 0);  
...  
s1 <= "11" & "0000";           --s1="110000"  
s2 <= '1' & s1(0) & s1(5 downto 4) & "00";  --s2="101100"  
s3 <= s1(5) & s1(5) & ('1', '0', '1', '0');  --s3="111010"
```

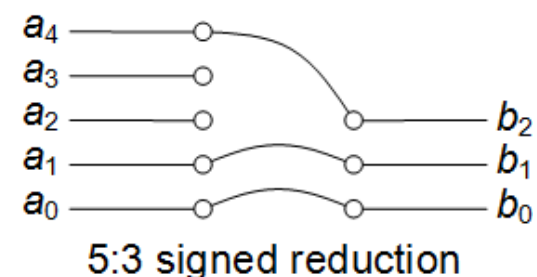
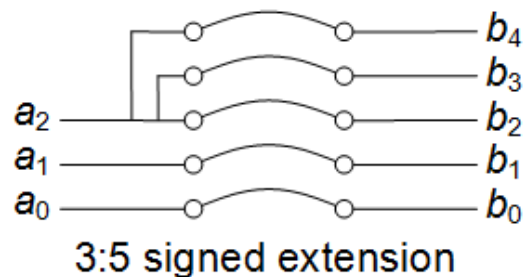
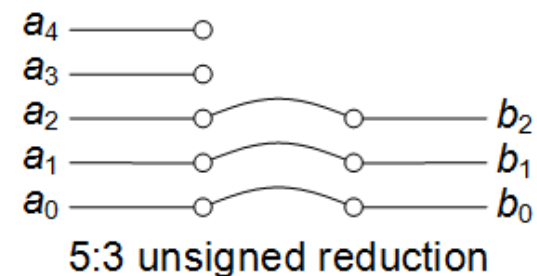
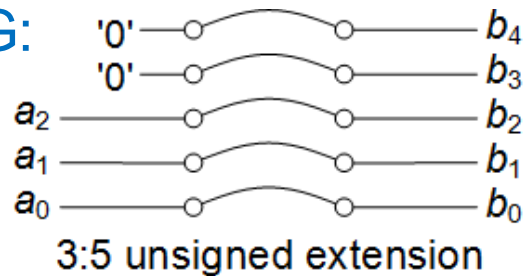
Predefined Data Types

8. Aggregation, concatenation, and resizing

c) The *resize* function

- VHDL-2008: Resize includes BV, SUV/SLV, UNS/SIG, UFIX/SFIX, FLO
- Full details shown in [section 7.9.3](#)
- Except for UNS/SIG, compilation support might still be limited

The case of UNS/SIG:



Predefined Data Types

8. Aggregation, concatenation, and resizing

c) The *resize* function

- VHDL-2008: Resize includes BV, SUV/SLV, UNS/SIG, UFIX/SFIX, FLO
- Full details shown in **section 7.9.3**
- Except for UNS/SIG, compilation support might still be limited

The case of UNS/SIG:

Example (for SIG):

```
signal s1:signed(7downto 0);    –8 bits
```

```
signal s2:signed(5downto 0);    –6 bits
```

...

```
s1<= resize(s2, 8); s2<=
```

–result: s1 = s2(5) & s2(5) & s2

```
resize(s1, 6);
```

–result: s2 = s1(7) & s1(4**downto**

0)

Predefined Data Types

9. Type-qualification expressions

- Help compiler resolve ambiguities about **type** of an object
- Require the ' (tick) symbol, as shown below

```
type_name'(expression);
```

Example:

```
signal x,y: integer range          ...;
```

...

```
y<=x when    x>"1001" else          0;           –illegal (type of "1001" undetermined)
```

```
y<=x when    x>signed("10001") else 0;           –legal
```

How do we know the 1st expression above is **illegal**?

- It involves a **comparison operator** (>)
- So we check **table 9.5**, where we see that **INT** can be compared to **INT**, **SIG**, **SFIX**, **FLO**, ...
- Therefore, the type of "10001" cannot be determined unambiguously

User-Defined Data Types

Type conversion

Table 8.1. Representation of synthesizable predefined types.

Type class	Subclass	Type or subtype	Representation examples
Scalar types	Integer types	<code>integer</code>	<code>-35</code> (≤ 32 bits)
		<code>natural</code>	<code>0</code> (≤ 31 bits)
		<code>positive</code>	<code>255</code> (≤ 31 bits)
	Enumeration types	<code>boolean</code>	<code>true</code> (1 bit)
		<code>bit</code> <code>std_(u)logic</code>	<code>'1'</code> (1 bit)
		<code>character</code>	<code>'A'</code> (8 bits)
Composite types	Array types	<code>integer_vector</code>	<code>63</code> <code>-9</code> <code>0</code> <code>127</code>
		<code>boolean_vector</code>	<code>true</code> <code>false</code> <code>false</code> <code>true</code>
		<code>bit_vector</code> <code>std_(u)logic_vector</code> <code>unsigned/signed</code> <code>ufixed/sfixed</code> <code>float</code>	<code>'1'</code> <code>'0'</code> <code>'0'</code> <code>'0'</code>
		<code>string</code>	<code>'V'</code> <code>'H'</code> <code>'D'</code> <code>'L'</code>

User-Defined Data Types

1. User-defined types

- a) Integer types
- b) Enumeration types
- c) Array types

Notes on **built-in types**:

- Are those from the package *standard*
- For example, a **user-defined** type based on **integer** or **bit** or **boolean** inherits the base-type properties (i.e., operators and other functions)
- But they are still considered to be **different types**

User-Defined Data Types

1. User-defined types

a) Integer types

- Recall that **integer** is a built-in type
- Integer types are declared (created) as shown below:

```
type type_name is range range_specification;
```

Example:

```
type small_integer is range -32 to 31;
signal s1,s2: integer range -32 to 31;    --with predefined integer
signal s3,s4: small_integer;              --with user-defined integer above
...
s1 <= s2**2 + 5;                          --legal?
s3 <= s4**2 + 5;                          --legal?
s1 <= s3;                                  --legal?
```

User-Defined Data Types

1. User-defined types

a) Integer types

- Recall that **integer** is a built-in type
- Integer types are declared (created) as shown below:

```
type type_name is range range_specification;
```

Example:

```
type small_integer is range -32 to 31;
signal s1,s2: integer range -32 to 31;    --with predefined integer
signal s3,s4: small_integer;              --with user-defined integer above
...
s1 <= s2**2 + 5;    --legal? Yes (same type and with support for arith. operators)
s3 <= s4**2 + 5;    --legal? Yes (same as above)
s1 <= s3;           --legal? No (type mismatch)
```


User-Defined Data Types

1. User-defined types

b) Enumeration types

- A list of **named values** (a list of “symbols”)
- Very useful for **finite state machines** (chapters 15-16)
- They are declared (created) as shown below:

```
type type_name is (type_values_list);
```

Examples showing **predefined** types:

```
type boolean is (false, true); --predefined type boolean  
type bit is ('0', '1'); --predefined type bit
```

Example of **user-defined** type:

```
type state_type is (hold, read, add, shift, store);  
signal pr_state, nx_state: state_type;
```

User-Defined Data Types

1. User-defined types

c) Array types

- Recall that **bit**, **boolean**, **integer**, ... are built-in types
- So vectors of these types inherit the base-type properties (operators, ...)
- But they are still different types
- Array types are declared (created) as shown below:

```
type type_name is array (range_spec) of base_type_name [range_spec];
```

Examples showing **predefined** array types:

```
type integer_vector is array (natural range <>) of integer;
```

```
type std_ulogic_vector is array (natural range <>) of std_ulogic;
```

Examples of **user-defined** array types:

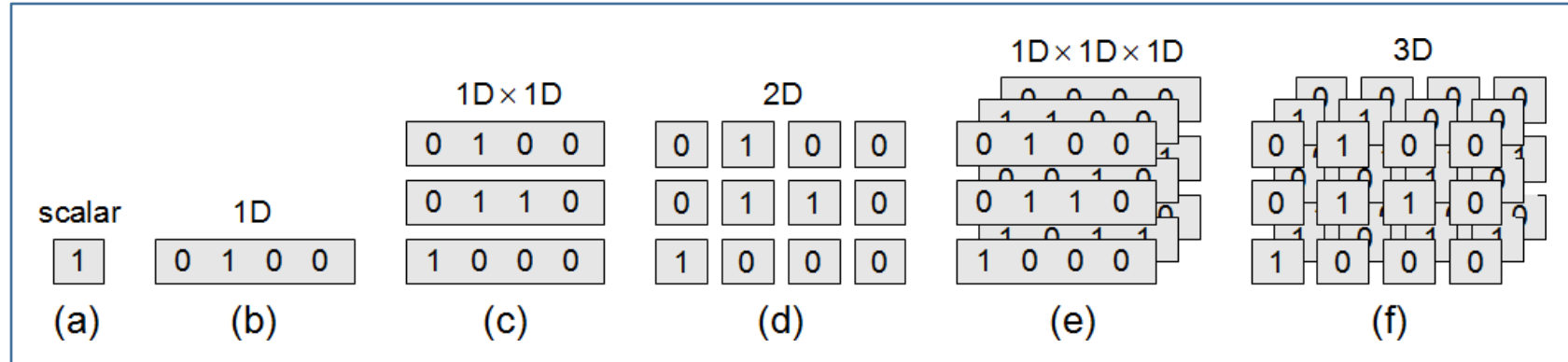
```
type int_vec is array (natural range <>) of integer range -32 to 31; – 6-bit integers
```

```
type bv_array is array (0 to 255) of bit_vector; – 256 values of unspecified size
```

User-Defined Data Types

2. Array dimensionality

Dimension when “bit” is the measurement unit:



- Very important because 1 bit = 1 wire (routing density)
- An addressing scheme, not “physical” dimension
- Values above 3D or 1Dx1Dx1D generally not expected in hardware

Example: What is the dimension of these types?

std_logic → ?

integer → ?

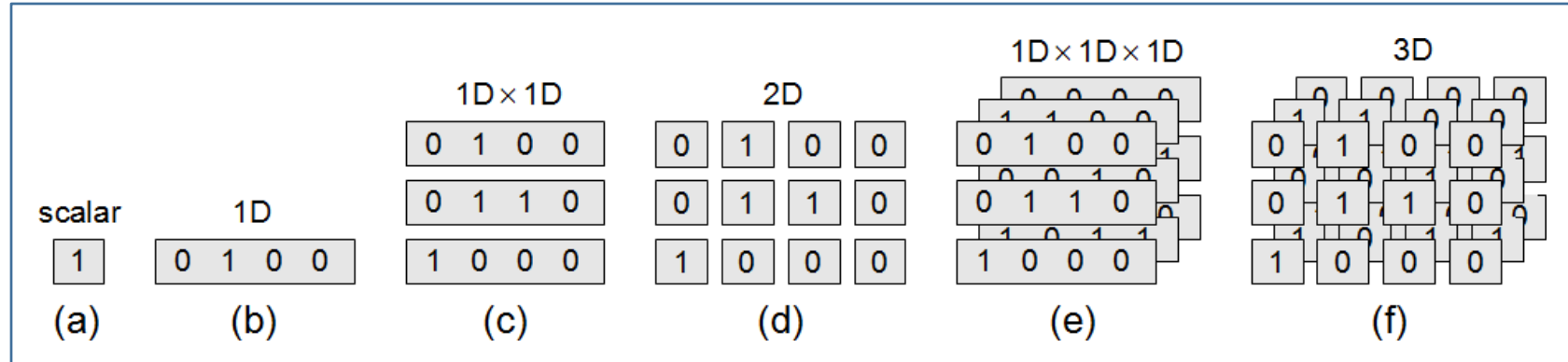
signed → ?

string → ?

User-Defined Data Types

2. Array dimensionality

Dimension when “bit” is the measurement unit:



- Very important because 1 bit = 1 wire (routing density)
- An addressing scheme, not “physical” dimension
- Values above 3D or 1Dx1Dx1D generally not expected in hardware

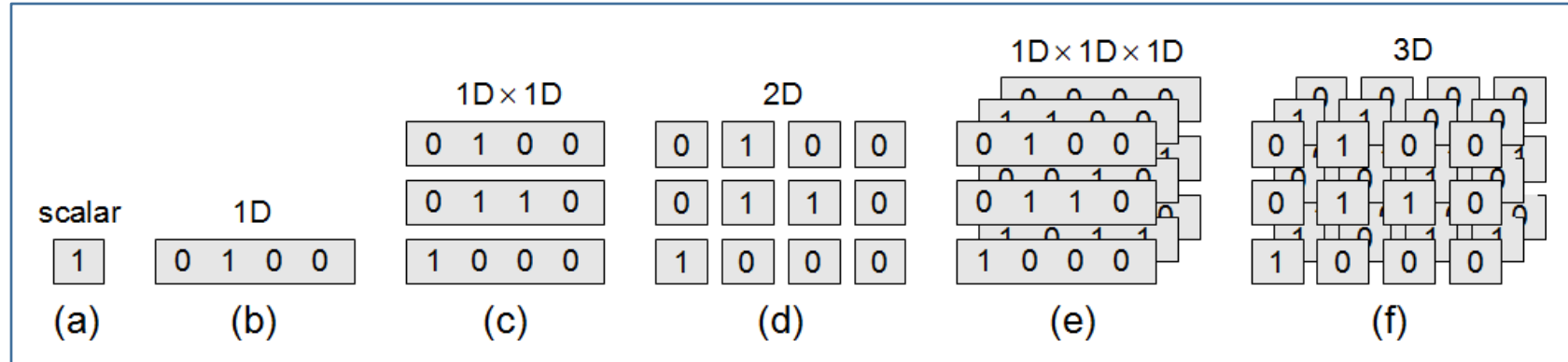
Example: What is the dimension of these types?

`std_logic` → scalar
`integer` → 1D
`signed` → 1D
`string` → ?

User-Defined Data Types

2. Array dimensionality

Dimension when “bit” is the measurement unit:



- Very important because 1 bit = 1 wire (routing density)
- An addressing scheme, not “physical” dimension
- Values above 3D or 1Dx1Dx1D generally not expected in hardware

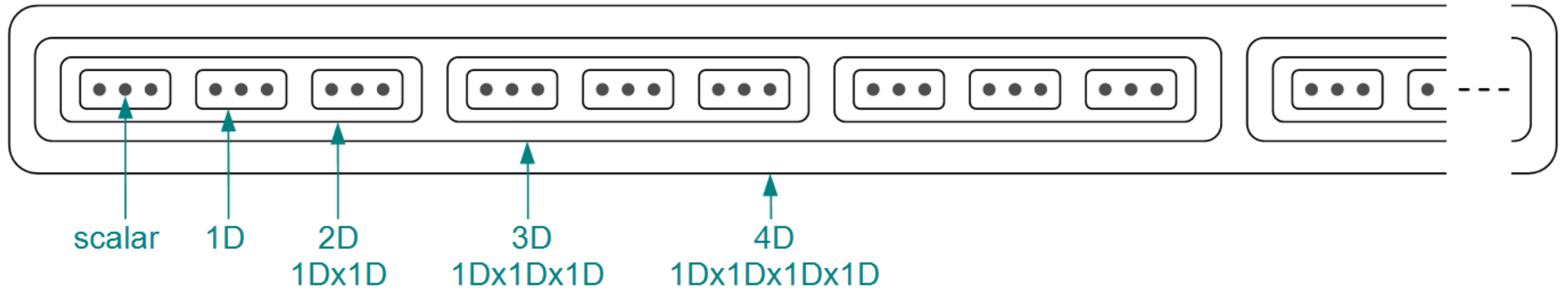
Question: What are the only (two) predefined types above 1D?

- string (1Dx1D)
- integer_vector (1Dx1D)

User-Defined Data Types

2. Array dimensionality

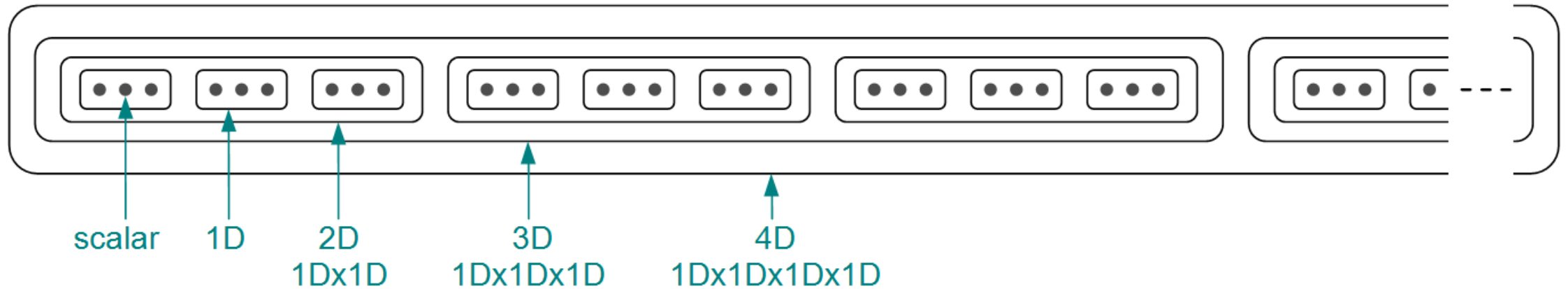
Another way of representing it:



User-Defined Data Types

2. Array dimensionality

Another way of representing it:



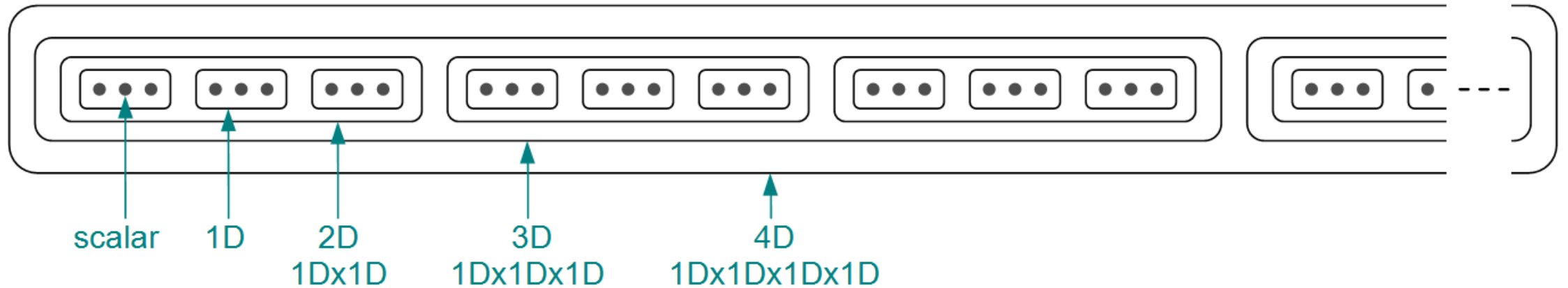
The type above could be declared as follows:

```
type type1 is array (? downto ?) of std_logic;  
type type2 is array (? downto ?) of ?;  
type type3 is array (? downto ?) of ?;  
type type4 is array (? downto ?) of ?;
```

User-Defined Data Types

2. Array dimensionality

Another way of representing it:



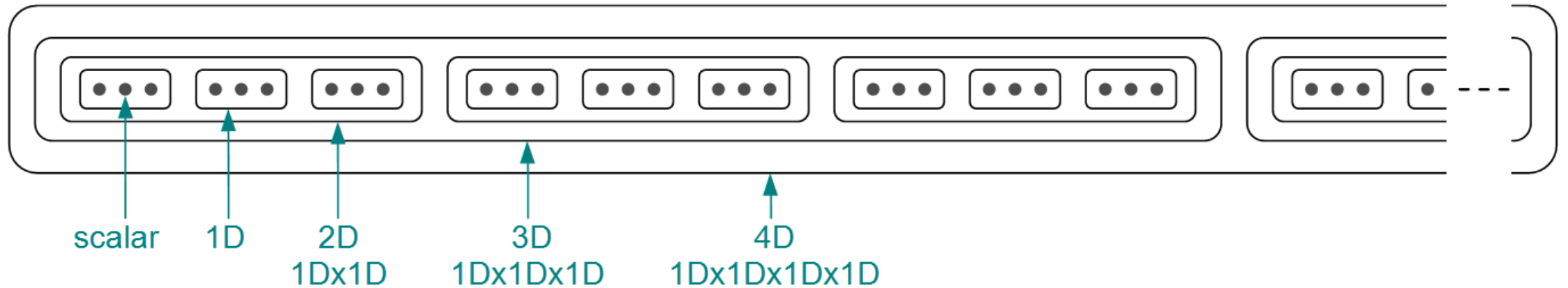
The type above could be declared as follows:

```
type type1 is array (2 downto 0) of std_logic;  
type type2 is array (? downto ?) of ?;  
type type3 is array (? downto ?) of ?;  
type type4 is array (? downto ?) of ?;
```


User-Defined Data Types

2. Array dimensionality

Another way of representing it:



The type above could be declared as follows:

```
type type1 is array (2 downto 0) of std_logic;  
type type2 is array (2 downto 0) of type1;  
type type3 is array (2 downto 0) of type2;  
type type4 is array (1 downto 0) of type3;
```

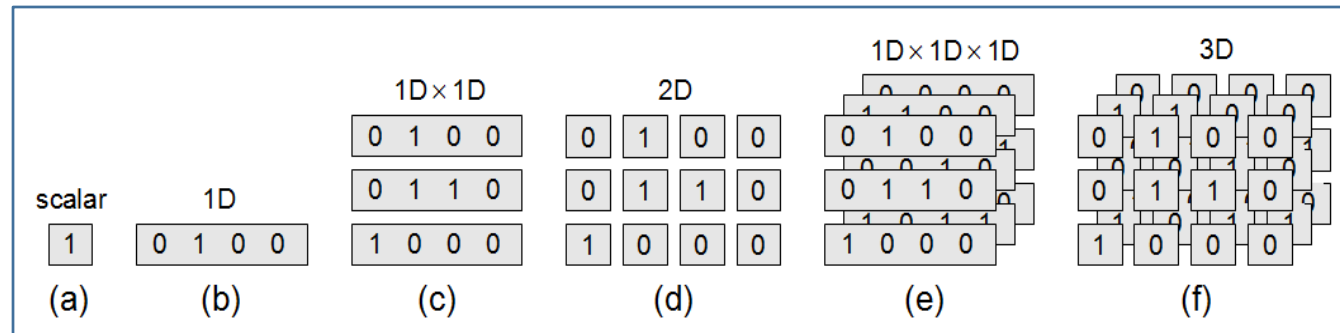
User-Defined Data Types

2. Array dimensionality

Another example:

What is the dimension of each signal below? Leave `char` and `string` out.

```
x1 <= '1';  
x2 <= 'Z';  
x3 <= "0000";  
x4 <= 50_000_000;  
x5 <= ("0001", "0000", "1111 ");  
x6 <= (others=>'Z');  
x7 <= ('1','1','0','1','0');  
x8 <= false;  
x9 <= (("000","000"),("000","000"));
```



User-Defined Data Types

2. Array dimensionality

Another example:

What is the dimension of each signal below? Leave `char` and `string` out.

x1 <= '1'; —scalar

x2 <= 'Z';

x3 <= "0000";

x4 <= 50_000_000;

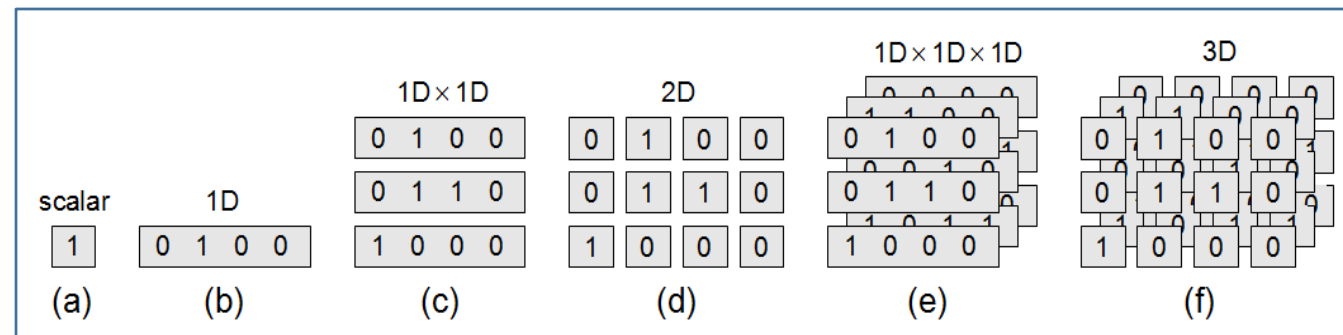
x5 <= ("0001", "0000", "1111 ");

x6 <= (others=>'Z');

x7 <= ('1','1','0','1','0');

x8 <= false;

x9 <= (("000","000"),("000","000"));



User-Defined Data Types

2. Array dimensionality

Another example:

What is the dimension of each signal below? Leave `char` and `string` out.

x1 <= '1'; —scalar

x2 <= 'Z'; —scalar

x3 <= "0000";

x4 <= 50_000_000;

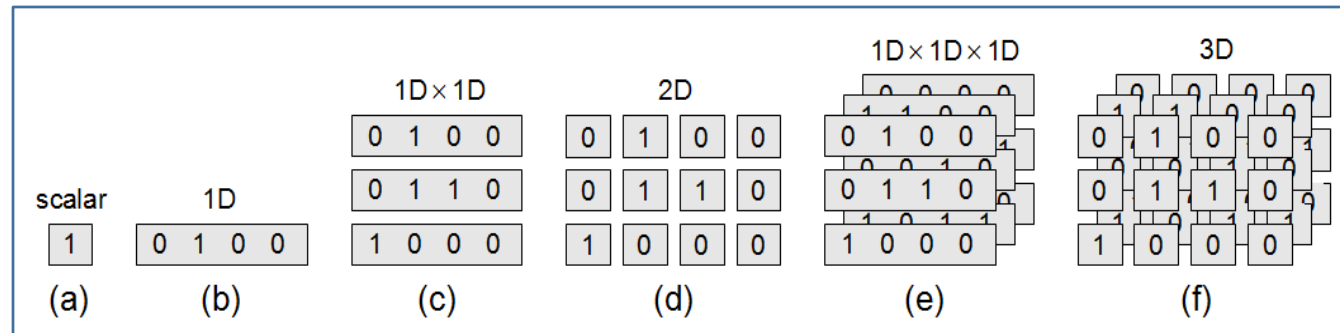
x5 <= ("0001", "0000", "1111 ");

x6 <= (others=>'Z');

x7 <= ('1','1','0','1','0');

x8 <= false;

x9 <= (("000","000"),("000","000"));



User-Defined Data Types

2. Array dimensionality

Another example:

What is the dimension of each signal below? Leave `char` and `string` out.

x1 <= '1'; *—scalar*

x2 <= 'Z'; *—scalar*

x3 <= "0000"; *—1D*

x4 <= 50_000_000;

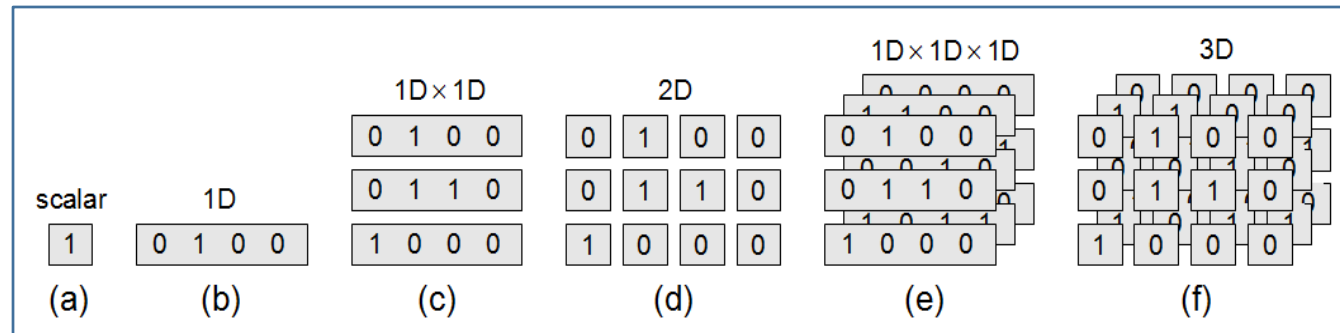
x5 <= ("0001", "0000", "1111 ");

x6 <= (others=>'Z');

x7 <= ('1','1','0','1','0');

x8 <= false;

x9 <= (("000","000"),("000","000"));



User-Defined Data Types

2. Array dimensionality

Another example:

What is the dimension of each signal below? Leave `char` and `string` out.

x1 <= '1'; *--scalar*

x2 <= 'Z'; *--scalar*

x3 <= "0000"; *--1D*

x4 <= 50_000_000; *--1D (INT, NAT, or POS)*

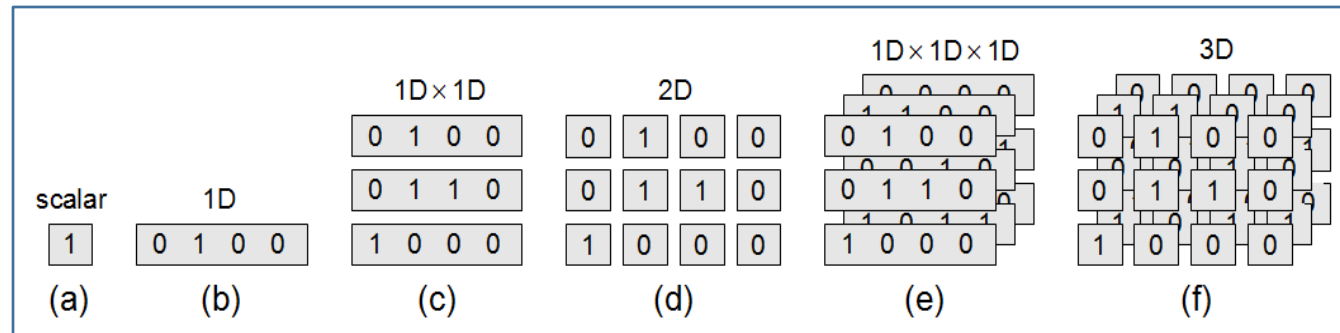
x5 <= ("0001", "0000", "1111 ");

x6 <= (others=>'Z');

x7 <= ('1','1','0','1','0');

x8 <= false;

x9 <= (("000","000"),("000","000"));



User-Defined Data Types

2. Array dimensionality

Another example:

What is the dimension of each signal below? Leave `char` and `string` out.

x1 <= '1'; *—scalar*

x2 <= 'Z'; *—scalar*

x3 <= "0000"; *—1D*

x4 <= 50_000_000; *—1D (INT, NAT, or POS)*

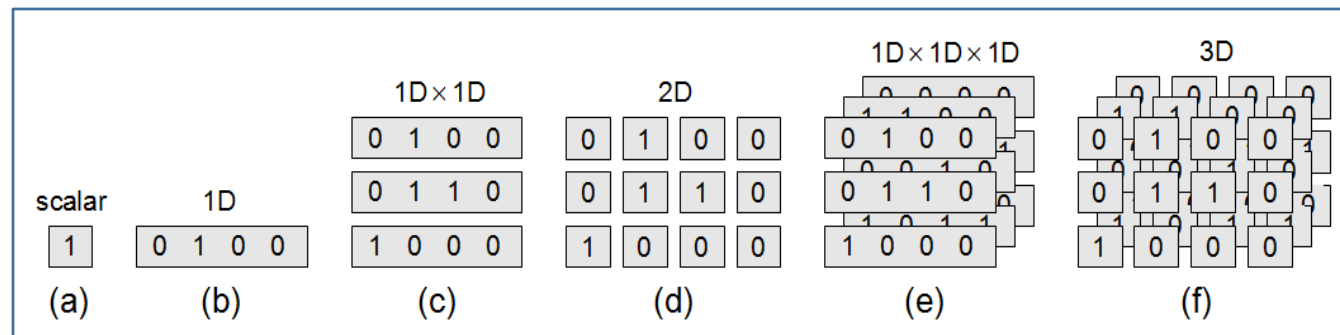
x5 <= ("0001", "0000", "1111 "); *—1Dx1D or 2D*

x6 <= (others=>'Z');

x7 <= ('1','1','0','1','0');

x8 <= false;

x9 <= (("000","000"),("000","000"));



User-Defined Data Types

2. Array dimensionality

Another example:

What is the dimension of each signal below? Leave `char` and `string` out.

x1 <= '1'; *—scalar*

x2 <= 'Z'; *—scalar*

x3 <= "0000"; *—1D*

x4 <= 50_000_000; *—1D (INT, NAT, or POS)*

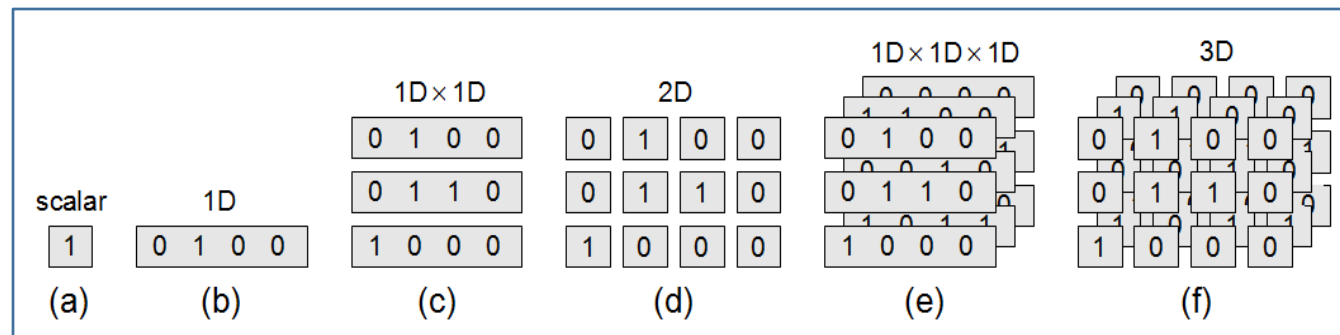
x5 <= ("0001", "0000", "1111 "); *—1Dx1D or 2D*

x6 <= (others=>'Z'); *—1D*

x7 <= ('1','1','0','1','0');

x8 <= false;

x9 <= (("000","000"),("000","000"));



User-Defined Data Types

2. Array dimensionality

Another example:

What is the dimension of each signal below? Leave `char` and `string` out.

x1 <= '1'; *—scalar*

x2 <= 'Z'; *—scalar*

x3 <= "0000"; *—1D*

x4 <= 50_000_000; *—1D* (INT, NAT, or POS)

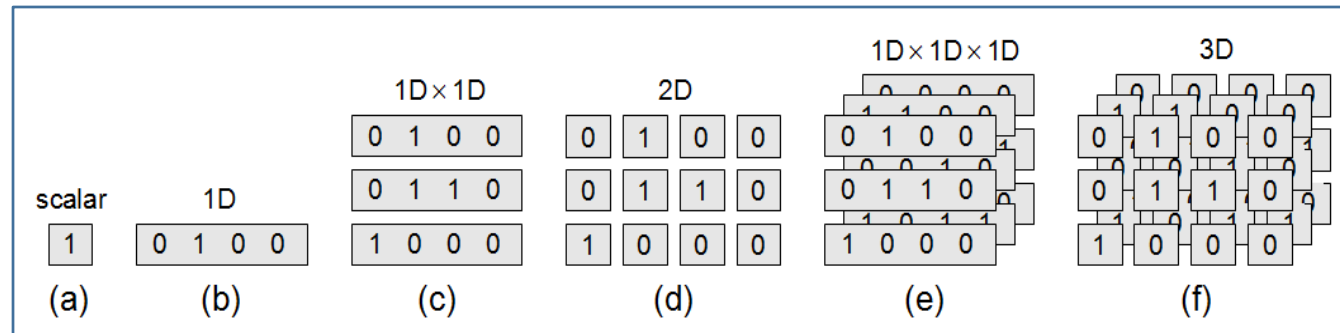
x5 <= ("0001", "0000", "1111 "); *—1Dx1D or 2D*

x6 <= (others=>'Z'); *—1D*

x7 <= ('1','1','0','1','0'); *—1D*

x8 <= false; *—scalar*

x9 <= (("000","000"),("000","000")); *—1Dx1Dx1D or 3D*



User-Defined Data Types

3. Building and addressing complex arrays

Example 1: **1Dx1D array**

1D × 1D

0	1	0	0
0	1	1	0
1	0	0	0

Example 2: **2D array**

2D

0	1	0	0
0	1	1	0
1	0	0	0

User-Defined Data Types

3. Building and addressing complex arrays

1D × 1D

0	1	0	0
0	1	1	0
1	0	0	0

Example 1: **1Dx1D array**

```
type type_1Dis array (1to 3)of std_logic_vector(3downto 0);  
signal s:type_1D;
```

...

```
s <= ("0100", "0110", "1000");
```

```
s(3) <= "1000";
```

```
s(3)(3) <= '1'; --notice two pairs of parentheses
```

2D

0	1	0	0
0	1	1	0
1	0	0	0

Example 2: **2D array**

```
type type_2Dis array (natural range <>, natural range <>)of std_logic;  
signal s:type_2D(1to 3, 3downto 0);
```

...

```
s <= ("0100", "0110", "1000");
```

```
s(3) <= "1000";
```

```
s(3, 3) <= '1'; --notice a single pair of parentheses
```

User-Defined Data Types

4. Checking and resetting data arrays

a) Zeroing entire array

Example: 1D×1D or 2D array

`s <= _____ ??? _____ ;` –all zero

User-Defined Data Types

4. Checking and resetting data arrays

a) Zeroing entire array

Example: 1D×1D or 2D array

```
s <= (others => (others => '0'));
```

—all zero

b) Checking whether entire array is zero

Example: For integer-related types (INT, UNS, SIG, ...)

```
if s=0 then ...
```

—direct comparison to zero is fine

User-Defined Data Types

4. Checking and resetting data arrays

a) Zeroing entire array

Example: 1D×1D or 2D array

```
s <= (others => (others => '0'));    --all zero
```

b) Checking whether entire array is zero

Example: For integer-related types (INT, UNS, SIG, ...)

```
if s=0 then ...    --direct comparison to zero is fine
```

Example: For any vector type:

```
if (others => '0') then ...    ???  
if (s'range => '0') then ...    ???  
if (s'range => (others => '0')) then    ???
```

User-Defined Data Types

4. Checking and resetting data arrays

a) Zeroing entire array

Example: 1D×1D or 2D array

```
s <= (others => (others => '0'));    --all zero
```

b) Checking whether entire array is zero

Example: For integer-related types (INT, UNS, SIG, ...)

```
if s=0 then ...    --direct comparison to zero is fine
```

Example: For any vector type:

```
if (others => '0') then ...    --1D, illegal
```

```
if (s'range => '0') then ...    --1D, legal
```

```
if (s'range => (others => '0')) then ... --1D×1D, legal
```

User-Defined Data Types

5. Classical mistakes in assignments

Table 8.2

Common errors in assignments

Error		Comments/examples
1	Incorrect assignment symbol	Use <= for signals Use := for variables, constants, and initial/default values
2	Type mismatch	Type must be the same on both sides of an expression
3	Invalid value or invalid representation	Incorrect use of quotation marks Type <code>bit</code> cannot receive value 'Z'
4	Incorrect indexing	Incorrect use of parentheses for range specifications, incorrect range direction, index out of range
5	Size mismatch	Both sides of an expression must have same dimensionality and same number of bits in each dimension
6	Illegal aggregation or illegal concatenation	Can be tricky; see examples in sections 7.9.1 and 7.9.2

User-Defined Data Types

5. Classical mistakes in assignments

Error 1: Incorrect assignment symbol (s=signal, v=variable)

s	:=	25;	–legal?
s	:=	v;	–legal?
s	<=	v;	–legal?
v	<=	25;	–legal?
v	:=	s;	–legal?
v	:=	25;	–legal?

User-Defined Data Types

5. Classical mistakes in assignments

Error 1: Incorrect assignment symbol (s=signal, v=variable)

s := 25;	–legal? No
s := v;	–legal? No
s <= v;	–legal? Yes
v <= 25;	–legal? No
v := s;	–legal? Yes
v := 25;	–legal? Yes

User-Defined Data Types

5. Classical mistakes in assignments

Error 1: Incorrect assignment symbol (s=signal, v=variable)

s := 25;	--legal? No
s := v;	--legal? No
s <= v;	--legal? Yes
v <= 25;	--legal? No
v := s;	--legal? Yes
v := 25;	--legal? Yes

Error 2: Type mismatch

std_logic ←	bit	--legal?
bit_vector ←	std_logic_vector signed(0)	--legal?
←	unsigned(1) signed(0) ←	--legal?
	std_logic_vector(1)	--legal?

User-Defined Data Types

5. Classical mistakes in assignments

Error 1: Incorrect assignment symbol (s=signal, v=variable)

s := 25;	--legal? No
s := v;	--legal? No
s <= v;	--legal? Yes
v <= 25;	--legal? No
v := s;	--legal? Yes
v := 25;	--legal? Yes

Error 2: Type mismatch

std_logic ← bit	--legal? No (different types)
bit_vector ← std_logic_vector	--legal? No (different types)
signed(0) ← unsigned(1)	--legal? Yes (SU element on both sides)
signed(0) ← std_logic_vector(1)	--legal? Yes (SU element on both sides)

User-Defined Data Types

5. Classical mistakes in assignments

Error 3: Invalid value or invalid representation

bit ←	'Z' integer ←	—legal?
	'1111' std_logic	—legal?
←	'Z' integer ←	—legal?
	1111	—legal?
bit_vector ←	0000	—legal?

User-Defined Data Types

5. Classical mistakes in assignments

Error 3: Invalid value or invalid representation

bit ← 'Z'	--legal?	No
integer ← "1111"	--legal?	No
std_logic ← 'Z'	--legal?	Yes
integer ← 1111	--legal?	Yes
bit_vector ← 0000	--legal?	No

Error 4: Incorrect indexing

signal s1: std_logic_vector(7 downto	0);	
signal s2: std_logic_vector(1 to	4);	
...		
s1(7 downto 5) <= s2(4 downto 2);	--legal?	
s1(7 downto 5) <= s2(0 to 2);	--legal?	
s1(7 downto 5) <= s2(2 to 4);	--legal?	
s1(7,5) <= "110";	--legal?	

User-Defined Data Types

5. Classical mistakes in assignments

Error 3: Invalid value or invalid representation

bit ← 'Z'	—legal?	No
integer ← "1111"	—legal?	No
std_logic ← 'Z'	—legal?	Yes
integer ← 1111	—legal?	Yes
bit_vector ← 0000	—legal?	No

Error 4: Incorrect indexing

signal s1: std_logic_vector(7 downto 0);	
signal s2: std_logic_vector(1 to 4);	
...	
s1(7 downto 5) <= s2(4 downto 2);	—legal? No (wrong direction for s2)
s1(7 downto 5) <= s2(0 to 2);	—legal? No (s2 is out of range)
s1(7 downto 5) <= s2(2 to 4);	—legal? Yes
s1(7,5) <= "110";	—legal? No (should be (7 downto 5))

User-Defined Data Types

5. Classical mistakes in assignments

Error 5: Size mismatch

bit_vector(7 downto	0) ←	bit_vector(0	to 3)	–legal?
std_logic_vector(0) ←		std_logic		–legal?
bit_vector(7 downto	4) ←	bit_vector(0	to 3)	–legal?
std_logic_vector(1 to	2) ←	std_logic		–legal?

User-Defined Data Types

5. Classical mistakes in assignments

Error 5: Size mismatch

bit_vector(7 downto 0) ←	bit_vector(0 to 3)	—legal?	No (8b x 4b)
std_logic_vector(0) ←	std_logic	—legal?	Yes (1b x 1b, same type)
bit_vector(7 downto 4) ←	bit_vector(0 to 3)	—legal?	Yes (4b x 4b, same type)
std_logic_vector(1 to 2) ←	std_logic	—legal?	No (2b x 1b, type mismatch)

User-Defined Data Types

Final example:

```
type row is array (7 downto 0) of std_logic;
type matrix1 is array (0 to 3) of row;
type matrix2 is array (0 to 3, 7 downto 0) of std_logic;
...
signal a: bit;
signal b: std_logic;
signal c: std_logic_vector(3 downto 0);
signal d: row;
signal e: matrix1;
signal f: matrix2;
```

Why are the assignments below illegal?

```
a <= 'Z';           --
a(0) <= '0';        --
a <= b;             --
c <= d;             --
e(0) <= "1111";     --
e(0,7) <= 0; f(0)(8) := '1'; --
--
```

Error	
1	Incorrect assignment symbol
2	Type mismatch
3	Invalid value or invalid representation
4	Incorrect indexing
5	Size mismatch
6	Illegal aggregation or illegal concatenation

User-Defined Data Types

Final example:

```
type row is array (7 downto 0) of std_logic;
type matrix1 is array (0 to 3) of row;
type matrix2 is array (0 to 3, 7 downto 0) of std_logic;
...
signal a: bit;
signal b: std_logic;
signal c: std_logic_vector(3 downto 0);
signal d: row;
signal e: matrix1;
signal f: matrix2;
```

Why are the assignments below illegal?

```
a <= 'Z';           -error 3
a(0) <= '0';        -error 4
a <= b;             -error 2
c <= d;             -error 2 + error 5
e(0) <= "1111";     -error 5
e(0,7) <= 0;        -error 3 + error 4
f(0)(8) := '-';     -error 1 + error 4 (wrong parent. and out of range)
```

Error	
1	Incorrect assignment symbol
2	Type mismatch
3	Invalid value or invalid representation
4	Incorrect indexing
5	Size mismatch
6	Illegal aggregation or illegal concatenation

Operators and Attributes

1. Operators

The predefined operators are divided into six categories:

- a) Logical operators
- b) Arithmetic operators
- c) Comparison (relational) operators
- d) Shift operators
- e) Concatenation operator
- f) Condition operator

Operators and Attributes

1. Operators

Table 9.1

Predefined VHDL operators

VHDL-2008

Category	Operators	Comments
Logical	not	The <i>not</i> operator
	and, nand, or, nor, xor, xnor	Regular and reduction logical operators
Arithmetic	-	Negation operator
	+, -, *, /, **, mod, rem, abs	Regular arithmetic operators
Comparison	=, /=, >, <, >=, <=	Regular comparison operators
	?=, ?/=:, ?>, ?<, ?>=, ?<=	Matching comparison operators
	minimum, maximum	Additional comparison functions
Shift	sll, srl, rol, ror	Logical shift operators
	sla, sra	Arithmetic shift operators
Concatenation	&	Array-forming operator
Condition	??	Converts bit or std_ulogic to boolean

Operators and Attributes

1. Operators

a) Logical operators

Binary (two-input) operators	
and	Regular <i>and</i> operator
nand	Regular <i>nand</i> operator
or	Regular <i>or</i> operator
nor	Regular <i>nor</i> operator
xor	Regular <i>xor</i> operator
xnor	Regular <i>xnor</i> operator

Unary (single-input) operators	
not	The <i>not</i> operator
and	Reduction <i>and</i> operator
nand	Reduction <i>nand</i> operator
or	Reduction <i>or</i> operator
nor	Reduction <i>nor</i> operator
xor	Reduction <i>xor</i> operator
xnor	Reduction <i>xnor</i> operator

Note: Unary operators have precedence over binary operators

Operators and Attributes

1. Operators

a) Logical operators

Table 9.2

Logical operators (including output sizes)

VHDL-2008

Package	Binary operations			Unary operations					
	Regular and , nand , or , nor , xor , xnor			Reduction and , nand , or , nor , xor , xnor			not operator		
	Left, Right	Outp	Size	Inp	Outp	Size	Inp	Outp	Size
<i>standard</i>	B, B	B	(1)	BV	B	(3)	B	B	(4)
	BV, BV	BV		—	—		BV	BV	
	BV, B	BV	(2)	—	—		—	—	
	BO, BO	BO	(1)	BOV	BO		BO	BO	
	BOV, BOV	BOV		—	—		BOV	BOV	
	BOV, BO	BOV	(2)	—	—		—	—	
<i>std_logic_1164</i>	SU, SU	SU	(1)	SUV	SU	(3)	SU	SU	(4)
	SUV, SUV	SUV		—	—		SUV	SUV	

See complete table in section 9.1.1

(1) Outp'length = L'length = R'length

(3) Outp'length = 1

(2) Outp'length = Inp_vector'length

(4) Outp'length = Inp'length

Operators and Attributes

1. Operators

a) Logical operators

Examples of *unary* logical operations:

and "1110" = '1' **and** '1' **and** '1' **and** '0' = '0'

nand "1110" = **not** (**and** "1110") = **not** ('0') = '1'

Examples of *binary* logical operations:

'1' **and** '1' = '1'

"0011" **xor** "0101" = "0110"

"0011" **or** **not** "0101" = "0011" **or** (**not** "0101") = "1011"

'1' **and** "0011" = ('1' **and** '0') ('1' **and** '0') ('1' **and** '1') ('1' **and** '1') = "0011"

Operators and Attributes

1. Operators

b) Arithmetic operators

Binary (two-input) operators	
+	Addition operator
-	Subtraction operator
*	Multiplication operator
/	Division operator
**	Exponentiation operator
rem	Remainder operator
mod	Modulo operator

Unary (single-input) operators	
-	Negation operator
abs	Absolute-value operator

Operators and Attributes

1. Operators

b) Arithmetic operators

Table 9.4

Arithmetic operators (including output sizes or ranges)

VHDL-2008

Binary operations							Unary operations		
Left, Right	Outp	+, -	*	/	**	rem, mod	Inp	Outp	-, abs
INT, INT	INT	(1)	(2)	(3)	(4) (5)	(6)	INT	INT	(7)
UNS, UNS	UNS	(8)	(11)	(14)	—	(15)	—	—	—
UNS, NAT		(9)	(12)	(9)	—	(9)	—	—	—
UNS, SU		(9)	—	—	—	—	—	—	—
SIG, SIG	SIG	(8)	(11)	(14)	—	(15)	SIG	SIG	(10)
SIG, INT		(10)	(13)	(10)	—	(10)	—	—	—
SIG, SU		(10)	—	—	—	—	—	—	—
UFX, UFX	UFX	(16)	(18)	(20)	—	(24)	—	—	—

Operators and Attributes

1. Operators

b) Arithmetic operators

Rules can be complex (see complete set in table 9.4):

For integer (virtually any size is legal; below are the minimum overflow-free sizes):

(1) Outp'length = maximum(L'length, R'length) + 1)

(2) Outp'length = L'length + R'length

...

For unsigned and signed (ranges normalized to "... downto 0"):

(8) maximum(L'length, R'length)-1 downto 0 (output length = largest input length)

(9) UNS'length-1 downto 0 (output length = length of unsigned input)

...

For ufixed and sfixed:

(16) maximum(L'left, R'left)+1 downto minimum(L'right, R'right)

(17) FIX'left+1 downto minimum(0, FIX'right) (size based on UFIX or SFIX element)

...

For float:

(28) maximum(L'left, R'left) downto minimum(L'right, R'right)

(29) FLO'range (range of float input)

Operators and Attributes

1. Operators

b) Arithmetic operators

Examples involving **types** (all rules are in table 9.4):

int	<=	int	+	int;	–legal?
sig	<=	sig	-	int;	–legal?
uns	<=	uns	-	sig;	–legal?
flo	<=	flo	-	sfix;	–legal?

Operators and Attributes

1. Operators

b) Arithmetic operators

Examples involving **types** (all rules are in table 9.4):

int	<=	int	+	int;	–legal?	Yes	(INT+INT=INT	available	in table 9.4)
sig	<=	sig	-	int;	–legal?	Yes	(SIG-INT=SIG	available	in table 9.4)
uns	<=	uns	-	sig;	–legal?	No	(UNS-SIG=UNS	not available	in table 9.4)
flo	<=	flo	-	sfix;	–legal?	No	(FLO-SFIX=FLO	not available	in table 9.4)

Table 9.4

Arithmetic operators (including output sizes or ranges)

Binary operations							Unary operations		
Left, Right	Outp	+, -	*	/	**	rem, mod	Inp	Outp	-, abs
INT, INT	INT	(1)	(2)	(3)	(4) (5)	(6)	INT	INT	(7)
UNS, UNS	UNS	(8)	(11)	(14)	–	(15)	—	—	—
UNS, NAT		(9)	(12)	(9)	–	(9)	—	—	—
UNS, SU		(9)	–	–	–	–	—	—	—
SIG, SIG	SIG	(8)	(11)	(14)	–	(15)	SIG	SIG	(10)
SIG, INT		(10)	(13)	(10)	–	(10)	—	—	—
SIG, SU		(10)	–	–	–	–	—	—	—

Operators and Attributes

1. Operators

b) Arithmetic operators

Examples involving **ranges** (all rules are in table 9.4):

`sig(6:0) <= sig(5:0) + su + su + su; ufix(5:-4) <= ufix(5:-1) -` –legal?

`ufix(3:-3);` –legal?

`flo(9:-7) <= flo(5:-5) + flo(4:-2);` –legal?

Operators and Attributes

1. Operators

b) Arithmetic operators

Examples involving **ranges** (all rules are in table 9.4):

`sig(6:0) <= sig(5:0) + su + su + su; ufix(5:-4) <= ufix(5:-1) -` –legal?

`ufix(3:-3);` –legal?

`flo(9:-7) <= flo(5:-5) + flo(4:-2);` –legal?

Binary operations				
Left, Right	Outp	+, -	*	
INT, INT	INT	(1)	(2)	
UNS, UNS	UNS	(8)	(11)	
UNS, NAT		(9)	(12)	
UNS, SU		(9)	—	
SIG, SIG	SIG	(8)	(11)	
SIG, INT		(10)	(13)	
SIG, SU		(10)	—	
UFX, UFX	UFX	(16)	(18)	
UFX, NAT		(17)	(19)	
SFIX, SFIX	SFIX	(16)	(18)	
SFIX, INT		(17)	(19)	
FLO, FLO	FLO	(28)	(28)	
FLO, INT		(29)	(29)	

Operators and Attributes

1. Operators

b) Arithmetic operators

Examples involving **ranges** (all rules are in table 9.4):

`sig(6:0) <= sig(5:0) + su + su + su;`

--legal? **No** (output should be sig(5:0))

`ufix(5:-4) <= ufix(5:-1) - ufix(3:-3);`

--legal? **No** (output should be ufix(6:-3))

`flo(9:-7) <= flo(5:-5) + flo(4:-2);`

--legal? **No** (output should be flo(5:-5))

Binary operations				
Left, Right	Outp	+, -	*	
INT, INT	INT	(1)	(2)	
UNS, UNS	UNS	(8)	(11)	
UNS, NAT		(9)	(12)	
UNS, SU		(9)	—	
SIG, SIG	SIG	(8)	(11)	
SIG, INT		(10)	(13)	
SIG, SU		(10)	—	
UFX, UFX	UFX	(16)	(18)	
UFX, NAT		(17)	(19)	
SFIX, SFIX	SFIX	(16)	(18)	
SFIX, INT		(17)	(19)	
FLO, FLO	FLO	(28)	(28)	
FLO, INT		(29)	(29)	

(10) SIG'length-1 downto 0 (output length = length of signed input)

(16) maximum(L'left, R'left)+1 downto minimum(L'right, R'right)

(28) maximum(L'left, R'left) downto minimum(L'right, R'right)

Operators and Attributes

1. Operators

c) Comparison operators

Regular comparison operators	
=	Equality
/=	Inequality
<	Smaller than
<=	Smaller than or equal to
>	Greater than
>=	Greater than or equal to
minimum(), maximum()	

Matching comparison operators	
?=	Equality
?/=	Inequality
?<	Smaller than
?<=	Smaller than or equal to
?>	Greater than
?>=	Greater than or equal to

- Regular comparison: All nine SU values are considered to be different
- Matching comparison: Considers '0'='L', '1'='H', '-'=any other value

Operators and Attributes

1. Operators

c) Comparison operators

Regular comparison operators	
=	Equality
/=	Inequality
<	Smaller than
<=	Smaller than or equal to
>	Greater than
>=	Greater than or equal to
minimum(), maximum()	

Matching comparison operators	
?=	Equality
?/=	Inequality
?<	Smaller than
?<=	Smaller than or equal to
?>	Greater than
?>=	Greater than or equal to

Allowed compositions:

`x<='1' when      a=b else '0';`

`x<='1' when      a?=b else '0';`

`x<=a?=b;`

–with regular comparison

–with matching comparison

–with matching comparison

Operators and Attributes

1. Operators

c) Comparison operators

Table 9.5

Comparison operators (main options)

VHDL-2008

Package	Regular comparison		Matching comparison		min/max functions	
	Left, Right	Outp	Left, Right	Outp	Left, Right	Outp
<i>standard</i>	B, B	BO	B, B	B	B, B	B
	BV, BV	BO	BV, BV ¹	B	BV, BV	BV
	BO, BO	BO	—	—	BO, BO	BO
	BOV, BOV	BO	—	—	BOV, BOV	BOV
	INT, INT	BO	—	—	INT, INT	INT
	INTV, INTV	BO	—	—	INTV	INT
	CHAR, CHAR	BO	—	—	CHAR, CHAR	CHAR
	STR, STR	BO	—	—	STR	CHAR
<i>std_logic_1164</i> ²	SU, SU	BO	SU, SU	SU	—	—
	SUV, SUV	BO	SUV, SUV ¹	SU	—	—
	UNS, UNS	BO	UNS, UNS	SU	UNS, UNS	UNS

See complete table in section 9.1.2

Operators and Attributes

1. Operators

c) Comparison operators

How comparisons are made:

- For INT, UNS/SIG, UFIX/SFIX, FLO, ...: The “usual” way
- For BV and SUV/SLV: “Unusual” way

Equality test: Vectors must have same length

Other tests: Rightmost bits of longer vector are discarded; if then the vectors become equal, the longer vector is considered larger

Operators and Attributes

1. Operators

c) Comparison operators

Examples with UNS:

"1111" > "011110" **false** (15 < 30)

"1100" = "001100" **true** (12 = 12)

Examples with SIG:

"1000" < "000011" **true** (−8 < 3)

"1100" = "001100" **false** (−4 < 12)

Examples with BV or SUV/SLV:

"1000" > "100000" **false** (truncation: "1000" < "1000"; so longer vector wins)

"1111" > "011111" **true** (truncation: "1111" > "0111"; so left vector wins) "01000" = "001000" **false**

(never equal when sizes are different)

Operators and Attributes

1. Operators

d) Shift operators

sll	Shift left logical (positions on the right are filled with zeros)
srl	Shift right logical (positions on the left are filled with zeros)
sla	Shift left arithmetic (rightmost bit is replicated on the right)
sra	Shift right arithmetic (leftmost bit is replicated on the left)
rol	Rotate left (circular shift to the left, reentering on the right)
ror	Rotate right (circular shift to the right, reentering on the left)

Operators and Attributes

1. Operators

d) Shift operators

Table 9.6

Shift operators

VHDL-2008

Package	Type	sll, srl, rol, ror		sla, sra	
		Left, Right	Outp	Left, Right	Outp
<i>standard</i>	BV	BV, INT	BV	BV, INT	BV
	BOV	BOV, INT	BOV	BOV, INT	BOV
<i>std_logic_1164</i>	SUV	SUV, INT	SUV	—	—
<i>numeric_std</i>	UNS	UNS, INT	UNS	UNS, INT	UNS
	SIG	SIG, INT	SIG	SIG, INT	SIG
<i>fixed_generic_pkg</i>	UFIX	UFIX, INT	UFIX	UFIX, INT	UFIX
	SFIX	SFIX, INT	SFIX	SFIX, INT	SFIX

Operators and Attributes

1. Operators

d) Shift operators

Examples:

"100011" **sll** 2 = "001100"

"100100" **sra** 3 = "111100"

"100110" **ror** -2 = "011010"

IMPORTANT: Shift can be done with the concatenation operator (&):

```
signal x,y,z: std_logic_vector(5downto 0);
```

...

```
y<=xsll 2; → y<=x(3downto 0)&"00";
```

```
z<=xsra 2; → z<=x(5)&x(5)&x(5downto 2);
```


Operators and Attributes

1. Operators

e) Concatenation operator (&)

- Review section 7.9.2 and its many examples

f) Condition operator (??)

- Converts a `bit` or `std_ulogic/std_logic` value to `boolean`
- Can be used implicitly (recommended)
- Example:

<code>if x='1' then</code>	<code>...</code>	–before VHDL-2008	
<code>if ??x then</code>	<code>...x</code>	–after VHDL-2008, explicit	
<code>if then</code>	<code>...</code>	–after VHDL-2008, implicit	(recommended)

Another example:

<code>y <= x when</code>	<code>a='1' and b='1' and c='0' else</code>	<code>'0';</code>	–before VHDL-2008
<code>y <= x when</code>	<code>a and b and not c else</code>	<code>'0';</code>	–after VHDL-2008, implicit

Operators and Attributes

1. Overloaded Operators

- An operator for which more than one construction exists
- Nearly all predefined operators are overloaded, like the following:

```
function "+"(L, R: integer)return           integer;  
function "+"(L, R: unsigned)return         unsigned;  
...
```

Example: There is no "+" operator for **BIT+INT** and **INT+BIT**. This operator is further overloaded by the functions below.

```
function "+"(L: integer; R: bit)return       integer is  
begin  
    if R then return L+1;  
    else return L;  
    end if;  
end function "+";
```

```
function "+"(L: bit; R: integer)return       integer is  
...  
end function "+";
```

Operators and Attributes

3. Attributes

- The predefined attributes are divided into four categories:
 - a) Attributes of scalar types
 - b) Attributes of array types and objects
 - c) Attributes of signals
 - d) Attributes of named entities
- Some are rarely used or not even implemented in synthesis tools
- Examples for only the truly needed will be shown next (full details are in sec. 9.3)
- These attributes are always preceded by the “tick” symbol (')

Operators and Attributes

3. Attributes

Example of very common attributes for array (or scalar) types

```
signal x:std_logic_vector(M downto N);
```

x'high → M

x'low → N x'left →

M x'right

→ N x'length

→ M-N+1

x'range → M downto N

x'reverse_range → N to M

x'ascending → false

Related usage examples:

```
signal x:std_logic_vector(7 downto 0);
```

```
for i in x'range loop ... —same as “for i in 7 downto 0 loop ...”
```

```
for i in x'low to x'high loop ... —same as “for i in 0 to 7 loop ...”
```

```
if x'length /= y'length then ... —comparison of two vector lengths
```

```
if x=(x'range => '0') then ... —checks whether x="000...0" (sec. 8.4.2)
```

Operators and Attributes

3. Attributes

Most common attribute for signals:

'event → informs that an event (change of level) has occurred

Also synthesizable, but not really needed:

'stable → informs that no event has occurred

For simulation (chapter 18):

'quiet → useful for detecting glitches

Related usage examples:

```
signal clk:std_logic;
```

```
...
```

```
if clk'event and clk='1' then ...    –a rising-edge of clock
```

```
if clk'event and clk='0' then ...    –a falling-edge of clock
```

Note: Check functions *rising_edge* and *falling_edge* in section 12.2

Operators and Attributes

4. Synthesis attributes

- To communicate with the **compiler** (overrides its setup)
- Tick not used
- Only some of synthesis attributes are standardized
- Two examples will be shown here (see others in section 9.5):
 - a) The “**state machine encoding**” attribute
 - b) The “**keep logic**” (do not simplify it) attribute

Operators and Attributes

4. Synthesis attributes

a) The “state machine encoding” attribute

- In Vivado

Name: *fsm_encoding*

Options: "sequential", "one_hot", "gray", "johnson", "auto"(default)

Example:

```
type state_type is (A, B, C, D, E);  
signal pr_state, nx_state: state_type;  
attribute fsm_encoding: string;  
attribute fsm_encoding of pr_state, nx_state: signal is
```

"one_hot";

- In Quartus Prime

Name: *syn_encoding*

Options: same as above plus "compact"(minimal bits) and user-encoded

Example:

```
type state_type is (A, B, C, D, E);  
signal pr_state, nx_state: state_type;  
attribute syn_encoding: string;  
attribute syn_encoding of state_type: type is
```

"0000 1111 0011 1100 0110";

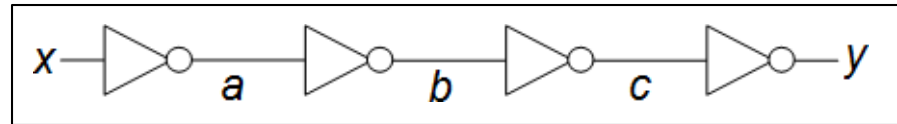
Operators and Attributes

4. Synthesis attributes

b) The *keep* attribute

- Prevents the simplification of combinational logic
- Syntax is the same in Vivado and Quartus Prime

Example: Delay line



Desired circuit

```
architecture with_keep of delay_line is
    signal a,b,c:bit;
    attribute keep:boolean;
    attribute keep of a,b,c:signal is true;
```

```
begin
```

```
    a <= not x;
```

```
    b <= not a;
```

```
    c <= not b;
```

```
    y <= not c;
```

```
end architecture;
```

Synthesis results:
without *keep*
with *keep*



Operators and Attributes

5. Alias

- Provides an alternative name to any named entity (signal, variable, ...)
- Must obey the VHDL naming criteria (chapter 5)
- Usually done in the declarative part of a subprogram or of an architecture
- There are three forms of aliasing:
 - a) Simple object aliasing
 - b) Object aliasing with type modification
 - c) Non-object aliasing
- Cases (a) and (c) are described next; see case (b) is in section 9.7

Operators and Attributes

5. Alias

Simple object aliasing:

```
alias new_name is original_name;
```

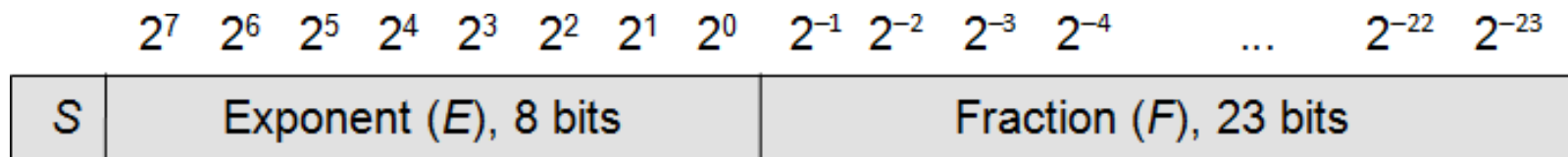
Example:

```
signal floating_point_input:float32;
```

```
alias sign_bit is floating_point_input(8);
```

```
alias exponent is floating_point_input(7 downto 0);
```

```
alias fraction is floating_point_input(-1 downto -23);
```



Operators and Attributes

5. Alias

Simple object aliasing:

```
alias new_name is original_name;
```

Example:

```
signal floating_point_input: float32;  
alias sign_bit is floating_point_input(8);  
alias exponent is floating_point_input(7 downto 0);  
alias fraction is floating_point_input(-1 downto -23);
```

Non-object aliasing:

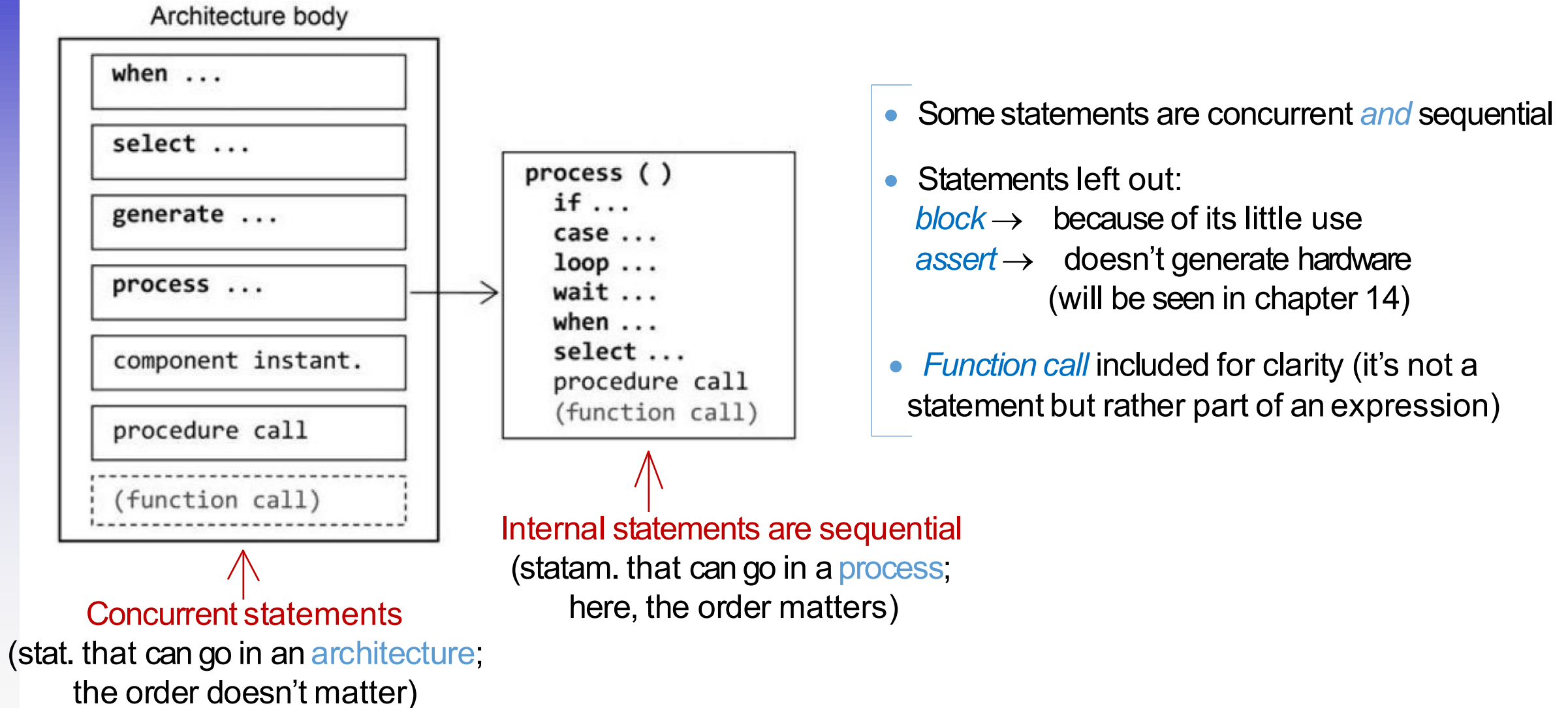
```
alias new_name is original_name [signature];
```

Example:

```
procedure sort(a,b: in integer; x,y: out integer) is ...  
procedure sort(a,b: in unsigned; x,y: out unsigned) is ... alias  
    sort_int is sort [integer, integer, integer, integer]; alias sort_uns is  
    sort [unsigned, unsigned, unsigned, unsigned];
```

Concurrent Code

1. Concurrent statements

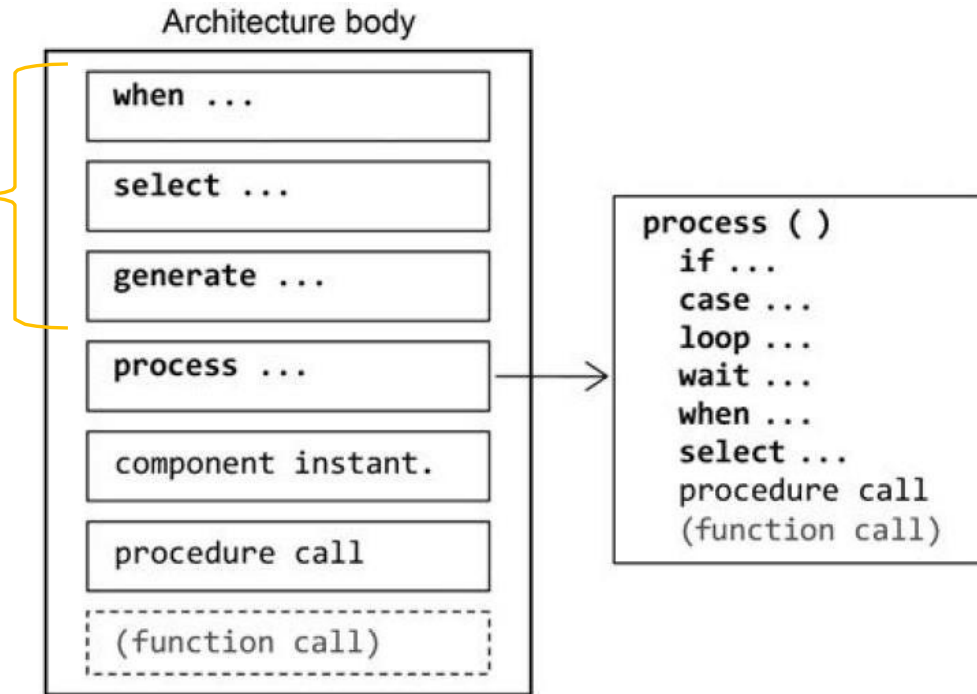


Concurrent Code

2. Concurrent code

- Appropriate for implementing only **combinational** circuits
- **Fundamental concurrent** statements (used **outside** processes and subprograms):

- *when*
- *select*
- *generate*



Concurrent Code

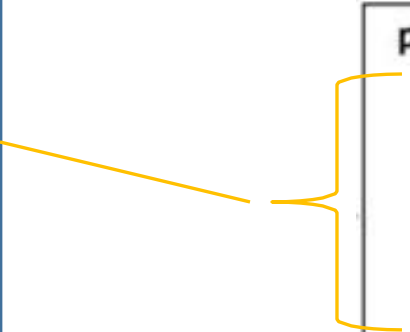
2. Concurrent code

- Appropriate for implementing only **combinational** circuits
- **Fundamental concurrent** statements (used **outside** processes and subprograms):

- *when*
- *select*
- *generate*

- **Fundamental sequential** statements (used **inside** processes and subprograms):

- *if*
- *case*
- *loop*
- *wait*
- *when*
- *select*



```
process ( )  
  if ...  
  case ...  
  loop ...  
  wait ...  
  when ...  
  select ...  
  procedure call  
  (function call)
```

Sequential versions of *when* and *select* were introduced in VHDL-2008

Concurrent Code

2. Concurrent code

- Appropriate for implementing only **combinational** circuits
- **Fundamental concurrent** statements (used **outside** processes and subprograms):

- *when*
- *select*
- *generate*

- **Fundamental sequential** statements (used **inside** processes and subprograms):

- *if*
- *case*
- *loop*
- *wait*
- *when*
- *select*

Sequential versions of *when* and *select* were introduced in VHDL-2008

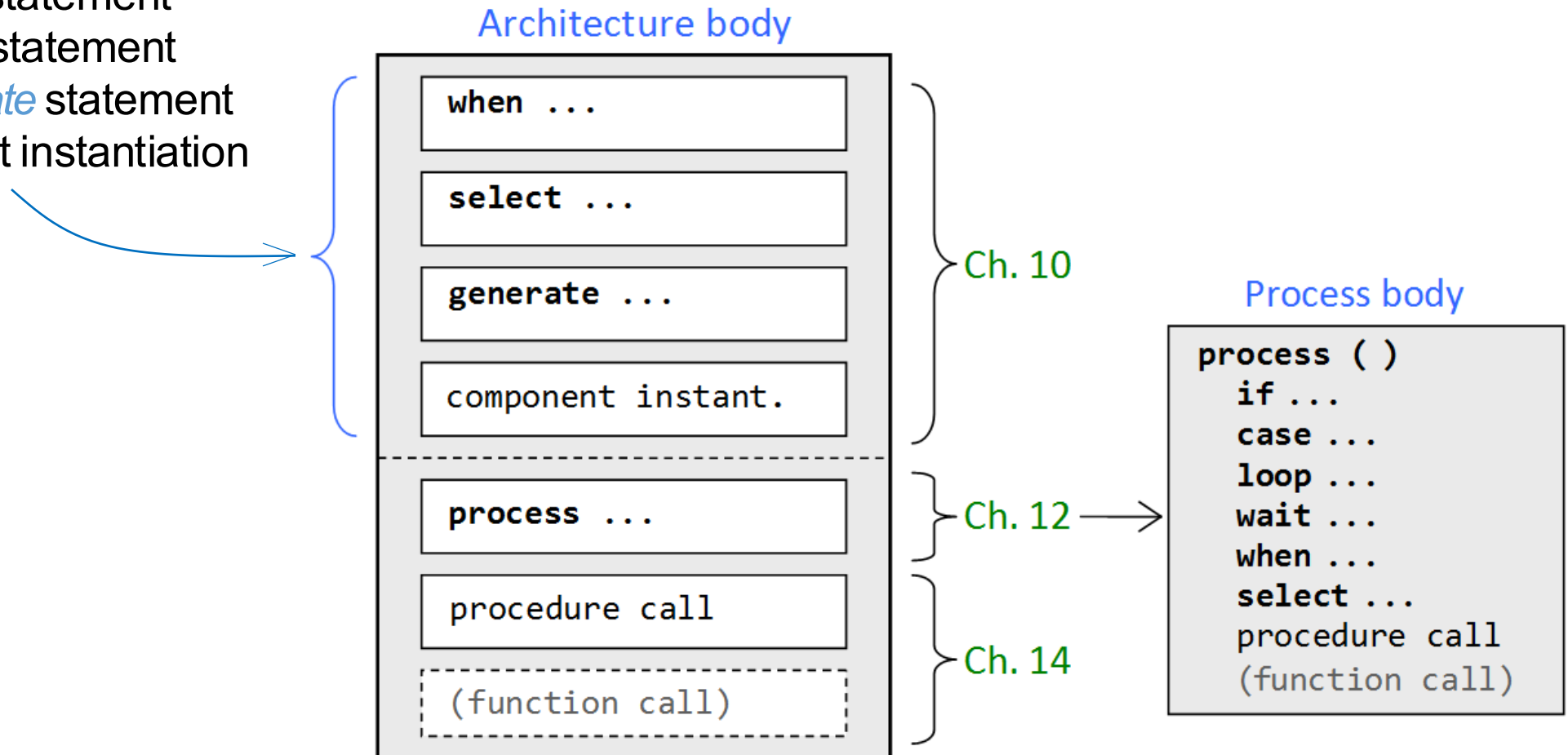
- **Operators** can be used anywhere (**inside** and **outside** processes and subprograms)

Concurrent Code

2. Concurrent code

Next:

- The *when* statement
- The *select* statement
- The *generate* statement
- Component instantiation



Concurrent Code

3. The *when* statement

- Also referred to as *conditional* statement
- Due to its priority-encoding nature, use it only when there are *few conditions* to test
- To enter truth tables in general, use *select* instead
- **IMP.:** Incomplete in-out coverage is *accepted* by the compiler, but *it will infer latches*

Syntax:

```
target <= value when condition else  
          value when condition else  
          value;
```

```
target <= value when condition else  
          value when condition else  
          value when condition;
```

- Ending with *else value* → Easy to cover all remaining values
- Ending with *when condition* → Clearer (shows all options explicitly, but *rarely viable*)

Concurrent Code

3. The *when* statement

Example:

```
outp<=inpwhen      enaelse  (others =>'Z');
```

What circuit is this? Tri-state buffer

What is the number of bits in *outp*? N (generic)

Concurrent Code

3. The *when* statement

Example:

```
outp<=inpwhen      enaelse  (others =>'Z');      --tri-state buffer
```

- There is just one condition (signal *ena*) to test, so *when* is fine
- It ends in “else value”, fine too because if *ena* is of type *SU* or *SL* it is difficult to cover all possibilities

Another example:

--Ending in *else value*:

```
y<=a    when sel=0 else  
      b when sel=1 else  
      c when sel=2 else  
      d;
```

--Ending in *when condition*:

```
y <= a when sel=0 else  
      b when sel=1 else  
      c when sel=2 else  
      d when sel=3;
```

- A (rare) example where all conditions can be shown explicitly
- But even this small multiplexer is better tailored for *select* than for *when*

Concurrent Code

4. The *select* statement

- Also referred to as *selected* statement
- Proper option for entering *truth tables* in general (in concurrent code)
- Incomplete in-out coverage *not* accepted by compiler (*latch-free* circuit)

Syntax:

```
with expression select
  target <= value when choice,
           value when choice,
           value when others;
```

```
with expression select
  target <= value when choice,
           value when choice,
           value when choice;
```

- Ending with *value when others* → Easy to cover all remaining values
- Ending with *value when choice* → Clearer (shows all options explicitly, but *rarely viable*)

Concurrent Code

4. The *select* statement

Example:

–Ending in *value when others*:

with select

 y <= a **when** 0,
 b **when** 1,
 c **when** 2,
 d **when others**;

–Ending in *value when choice*:

with select

 y <= a **when** 0,
 b **when** 1,
 c **when** 2,
 d **when** 3;

- This is the multiplexer of the previous example
- Implemented with *select*, which is the right option for straightforward truth tables
- If *sel* were SUV or SLV, the approach on the right would **not** be viable

Concurrent Code

4. The *select* statement

- Choice expressions can use *to*, *downto*, *|* (means *or*), and *others*

Example:

```
with select  
  x <= a when    0 | 7,  
             b. when 2 to 5,  
             c. when others;
```

Concurrent Code

4. The *select* statement

- Equations are allowed in the selection argument:

Example:

```
with (a>b)select           –parentheses are optional
  y<=awhen    true, b
      when    false;
```

Another example:

```
signal  a,  b:    std_logic_vector(3    downto  0);
signal  x,  y:    std_logic_vector(7    downto  0);
```

...

```
with (axor  b)select
  y<=  xwhen (3downto  0=>'1'),
      not xwhen  others;
```

Equivalent:

```
← xwhen (a'range =>'1') xwhen
← (b'range =>'1')
```

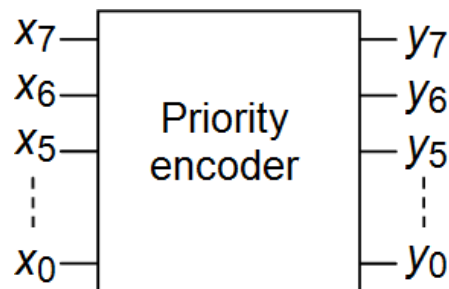
Concurrent Code

4. The *select* statement

The matching *select?* version

- Uses the matching comparison operator ('0'='L', '1'='H', and '-' = any value)
- It can be used for “don't care” values at the input

Example:



$x_7 \dots x_0$	$y_7 \dots y_0$
1xxxxxxx	10000000
01xxxxxx	01000000
001xxxxx	00100000
0001xxxx	00010000
00001xxx	00001000
000001xx	00000100
0000001x	00000010
00000001	00000001
00000000	00000000

```
architecture lutof priority_encoder is begin
  with xselect?
    y <= "1000" when "1—",
        "0100" when "01—",
        "0010" when "001-",
        "0001" when "0001",
        "0000" when others; end
architecture;
```


Concurrent Code

5. The *generate* statement

- *generate* is used to build loops in concurrent code
- It has three forms: *for-generate*, *if-generate*, *case-generate*
- *for-generate* is by far the most common
- So only that will be seen here

for-generate:

```
label: for identifier in generate_range generate  
    [generate_declarative_part  
    begin]  
        concurrent_statements  
end generate [label];
```

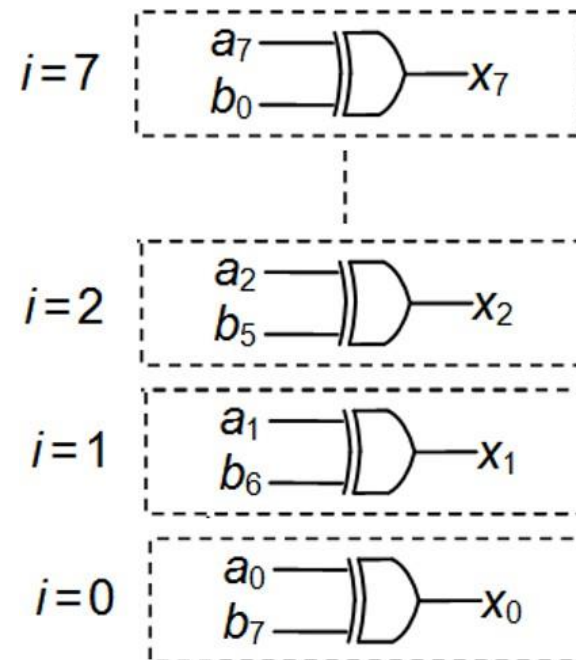
Concurrent Code

5. The *generate* statement

Example:

```
signal a,b,x:std_ulogic_vector(7 downto 0);  
...  
gen:for i in x'range generate  
    x(i)<=a(i)xor b(b'left-i);  
end generate;
```

What's the resulting circuit?



Concurrent Code

6. Component instantiation statements

- The circuit to be instantiated is a **regular** (previous) VHDL design
- It can be combinational or sequential
- But the **instantiation** can only be done in **concurrent** code
- The original design is associated to the new design using:
port map for the circuit ports
generic map for generic constants (if there are any)
- The instantiation can be done in two ways:
 - a) **Component instantiation** (requires **declaration** + **instantiation**)
 - b) **Design entity instantiation** (direct **instantiation**)

Concurrent Code

6. Component instantiation statements

a) Component instantiation

Component declaration:

```
component component_name [is]
  [generic (...);]
  port (...);
end component [component_name];
```

Component instantiation:

```
label: [component] component_name
  [generic map (generic_association_list)]
  port map (port_association_list);
```

Declaration:

- It is a copy of the entity declaration, just with the word *entity* replaced with *component*
- A common place for this declaration is the architecture's declarative region

Instantiation:

- Employs *port map* plus, if necessary, *generic map*
- The associations can be *named* or *positional*
- The keyword *open* is used to leave a port unconnected

Concurrent Code

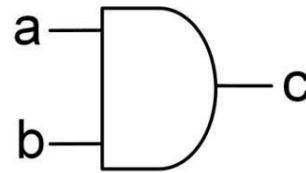
6. Component instantiation statements

a) Component instantiation

Example:

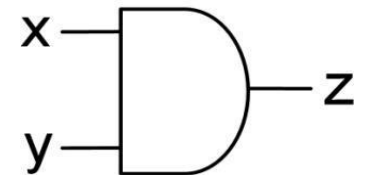
–Component declaration:
component and_gate **is**
 port (
 a,b:**in** bit; c:
 out bit);
end component;

Original design:



- Component instantiation with **named** association: and2x1: and_gate **port**
 map (a=>x, b=>y, c=>z);
- Component instantiation with **positional** association: and2x1: and_gate **port**
 map (x,y,z);

As part of the
new design:



Concurrent Code

6. Component instantiation statements

a) Component instantiation

More detailed example:

A previous project:

```
entity brick is
  generic (
    NUM_BITS: positive);
  port (
    a,b:in ...;
    c:out ...);
end entity;

architecture ...
  ...
end architecture;
```

declaration

instantiation
(named)

Current project:

```
entity wall is
  generic (
    WIDTH: positive := 32);
  port (
    x,y:in ...;
    z:out ...);
end entity wall;

architecture ...
  component brick is
    ...
  end component;
begin
  comp: brick generic map (NUM_BITS=> WIDTH)
    port map (a=>x, b=>y, c=>z);
  ...
end architecture;
```

Concurrent Code

6. Component instantiation statements

a) Component instantiation

A more detailed example:

A previous project:

```
entity brick is
  generic (
    NUM_BITS: positive);
  port (
    a,b:in ...;
    c:out ...);
end entity;

architecture ...
  ...
end architecture;
```

declaration

instantiation
(positional)

Current project:

```
entity wall is
  generic (
    WIDTH: positive := 32);
  port (
    x,y:in ...;
    z:out ...);
end entity wall;
```

```
architecture ...
  component brick is
    ...
  end component;
begin
  comp: brick generic map (WIDTH)
    port map (x,y,z);
  ...
end architecture;
```

Concurrent Code

6. Component instantiation statements

b) Design entity instantiation

Instantiation:

```
label: entity work.entity_name [(architecture_name)]  
  [generic map (generic_association_list)]  
  port map (port_association_list);
```

Advantage: Declaration not needed

Concurrent Code

6. Component instantiation statements

b) Design entity instantiation

Example:

```
entity wall is  
  generic (  
    WIDTH: positive := 32);  
  port (  
    x, y: in ...;  
    z: out ...);  
end entity wall;  
  
architecture ...  
begin  
  comp: entity work.brick generic map (NUM_BITS => WIDTH)  
    port map (a => x, b => y, c => z);  
  ...  
end architecture;
```

instantiation
(named)

Concurrent Code

6. Component instantiation statements

b) Design entity instantiation

Example:

```
entity wall is  
  generic (  
    WIDTH: positive := 32);  
  port (  
    x, y: in ...;  
    z: out ...);  
end entity wall;  
  
architecture ...  
begin  
  comp: entity work.brick generic map (NUM_BITS)  
    port map (x, y, z);  
  ...  
end architecture;
```

instantiation
(positional)

Concurrent Code

We close this chapter with two **very special** cases:

- 1) How to avoid assigning a value to a signal more than once
- 2) How to do math properly with VHDL

Concurrent Code

7. Avoiding multiple assignments to the same signal

- Why can't we do this (outside a process)?

`y <= 0;`

`...`

`y <= x + 1;`

- Because it is a **concurrent code**, so any order of the statements should produce the same result
- **Proposed solution:**

1) Create an internal signal with **extra** dimension:

`Scalar` → `1D`

`1D` → `1D×1D` or `2D`

`1D×1D` → `1D×1D×1D` or `3D`, etc.

2) Compute the values for this signal

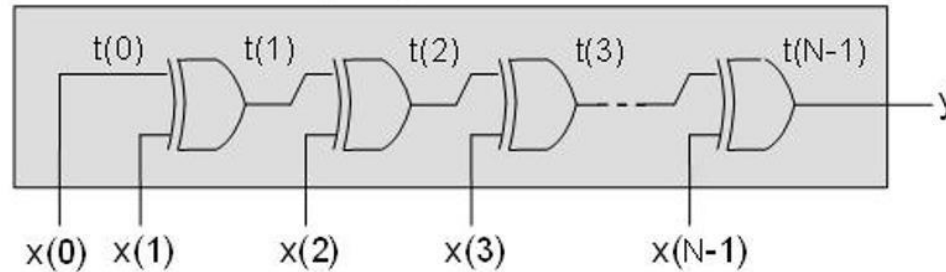
3) Pass its last value to the output signal

Concurrent Code

7. Avoiding multiple assignments to the same signal

Example:

Chain-type parity detector (sec. 1.4.3)



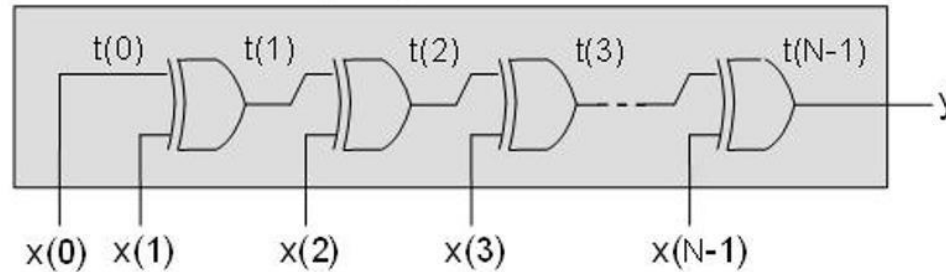
Note: Assume that there is no unary xor operator (`y<=xor x;`)

Concurrent Code

7. Avoiding multiple assignments to the same signal

Example:

Chain-type parity detector (sec. 1.4.3)



Note: Assume that there is no unary xor operator (`y<=xor x;`)

Solution 1:

```
architecture a1 of parity_detector is
begin
  y<=x(0);
  gen: for i in 1 to N-1 generate
    y<=y xor x(i);
  end generate;
end architecture;
```

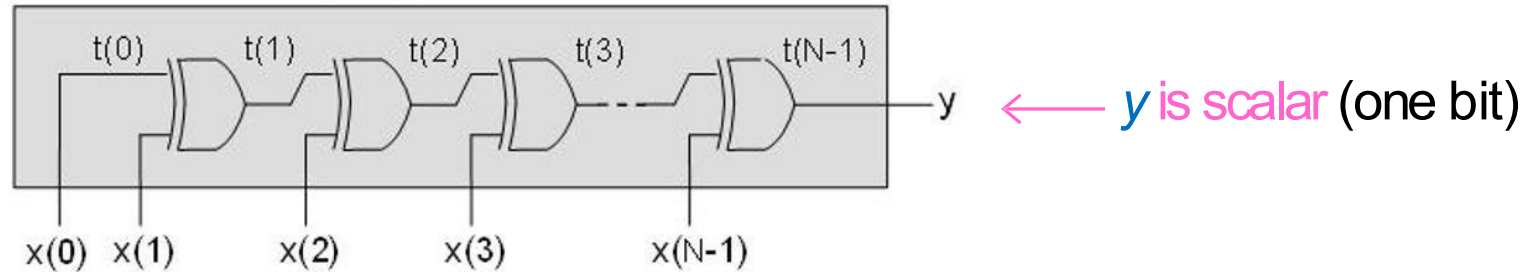
How many assignments
to signal `y`? N

Concurrent Code

7. Avoiding multiple assignments to the same signal

Example:

Chain-type parity detector (sec. 1.4.3)



Solution 2:

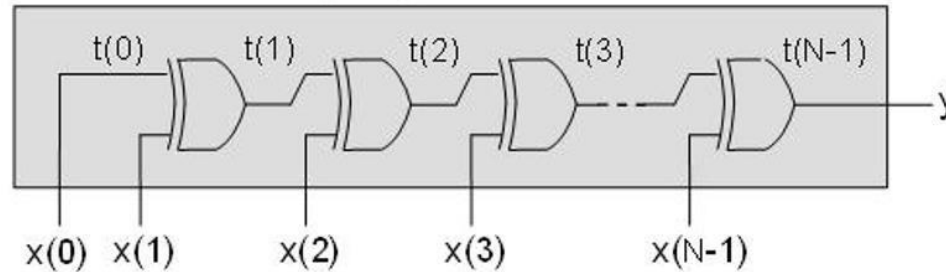
```
architecture a2 of parity_detector is
    signal t: bit_vector(N-1 downto 0); ←  $t$  is 1D (vector of bits)
begin
    t(0) <= x(0);
    gen: for i in 1 to N-1 generate
        t(i) <= t(i-1) xor x(i);
    end generate;
    y <= t(n-1); ← last value of  $t$  passed to  $y$  (single bit)
end architecture;
```

Concurrent Code

7. Avoiding multiple assignments to the same signal

Example:

Chain-type parity detector (sec. 1.4.3)



Solution 2:

```
architecture a2 of parity_detector is
    signal t: bit_vector(N-1 downto 0);
begin
    t(0) <= x(0);
    gen: for i in 1 to N-1 generate
        t(i) <= t(i-1) xor x(i);
    end generate;
    y <= t(n-1);
end architecture;
```

How many assignments
to signal y ? 1

Concurrent Code

8. Doing math right with VHDL

Note: Review the concepts below in section 1.6:

- Integer arithmetic
- Floating-point arithmetic
- Carry bit versus overflow flag
- Extension, truncation, rounding, and saturation

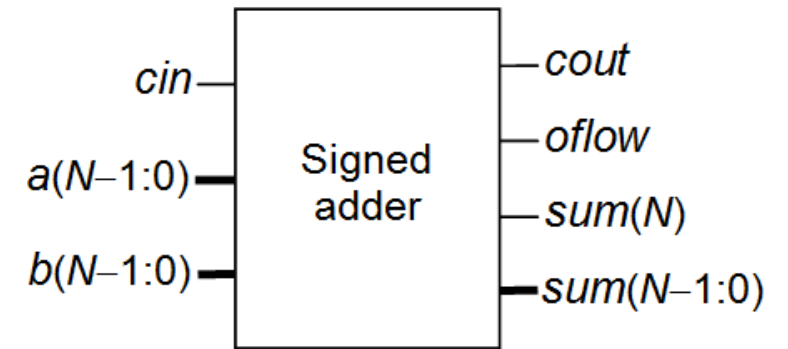
- Consider the following as the “**arithmetic**” types:
 - For integers: `unsigned`, `signed`
 - For fixed-point: `ufixed`, `sfixed`
 - For floating-point: `float`, `float32`, `float64`, `float128`
- Do **not** use type `integer` to do math
- Avoid floating-point whenever possible (a lot of hardware, lower speed, extra power)
- As default, use `standard-logic types` for the circuit ports

Concurrent Code

8. Doing math right with VHDL

Example:

```
1  -----
2  library ieee;
3  use ieee.std_logic_1164.all;
4  use ieee.numeric_std.all;
5
6  entity adder_signed is
7      generic (
8          NUM_BITS: integer := 4);
9      port (
10         a, b: in std_logic_vector(NUM_BITS-1 downto 0);
11         cin: in std_logic;
12         sum: out std_logic_vector(NUM_BITS-1 downto 0);
13         sumMSB, cout, oflow: out std_logic);
14  end entity;
15
```

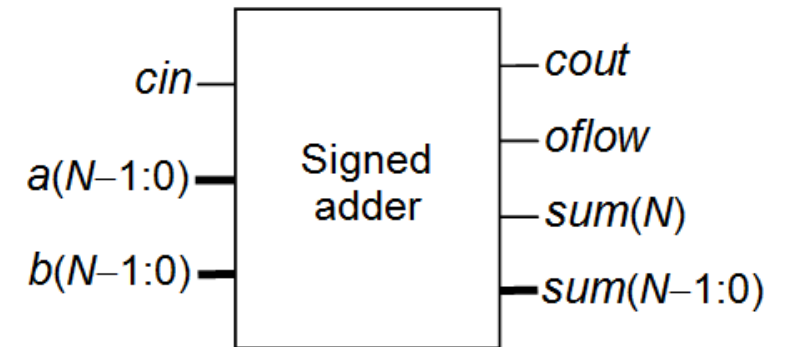


Concurrent Code

8. Doing math right with VHDL

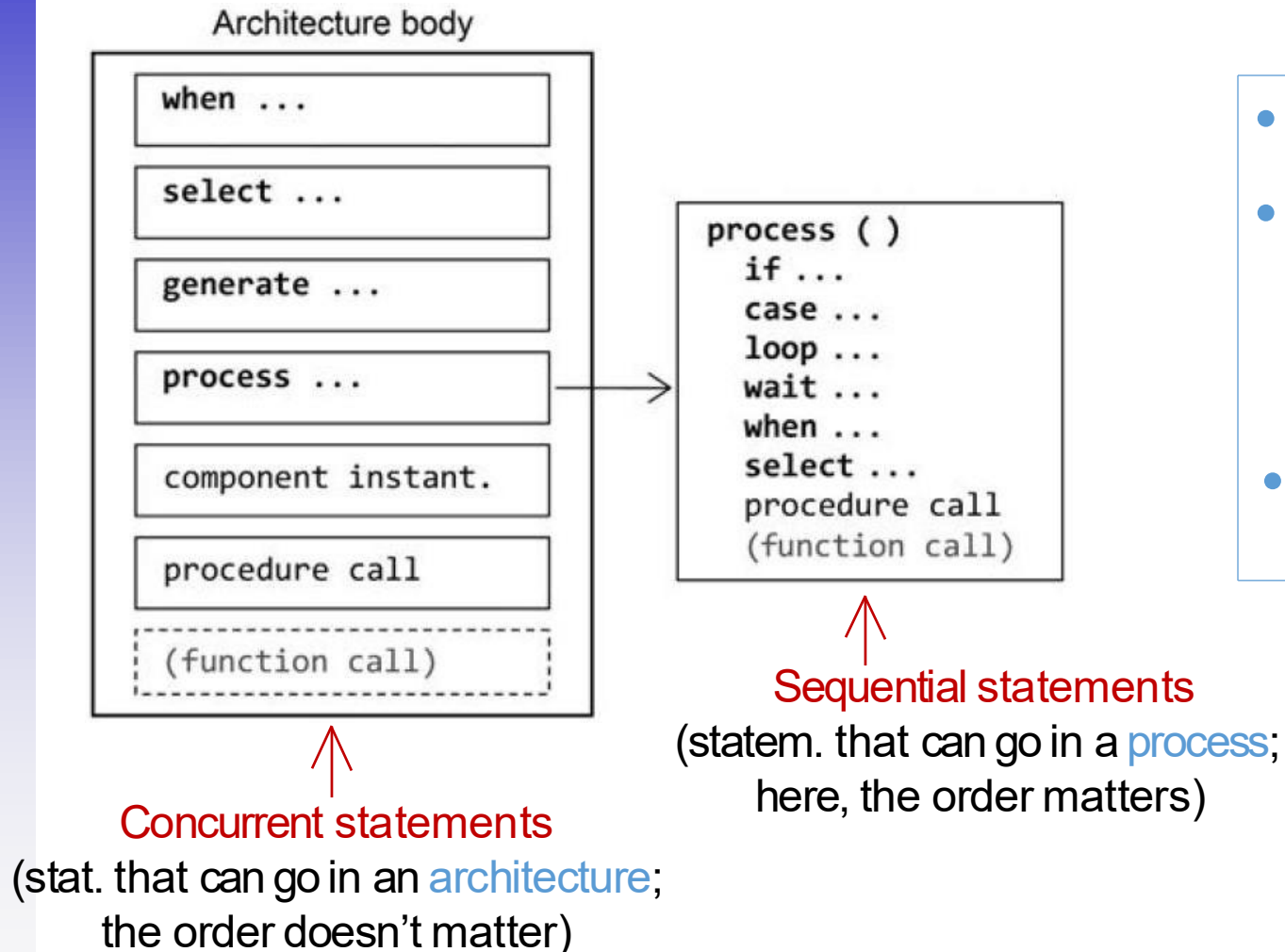
Example:

```
16  architecture suggested of adder_signed is
17      signal sum_sig: signed(NUM_BITS downto 0);
18  begin
19
20      --Sign-extension, conversion to signed, and addition:
21      sum_sig <= signed(a(NUM_BITS-1) & a) + signed(b) + cin;
22      --sum_sig <= resize(signed(a), NUM_BITS+1) + signed(b) + cin;
23
24      --Conversion to std_logic_vector and final operations:
25      sum <= std_logic_vector(sum_sig(NUM_BITS-1 downto 0));
26      sumMSB <= sum_sig(NUM_BITS);
27      cout <= a(NUM_BITS-1) xor b(NUM_BITS-1) xor sumMSB;
28      oflow <= sum_sig(NUM_BITS) xor sum_sig(NUM_BITS-1);
29
30  end architecture;
31  -----
```



Sequential Code

1. Sequential statements



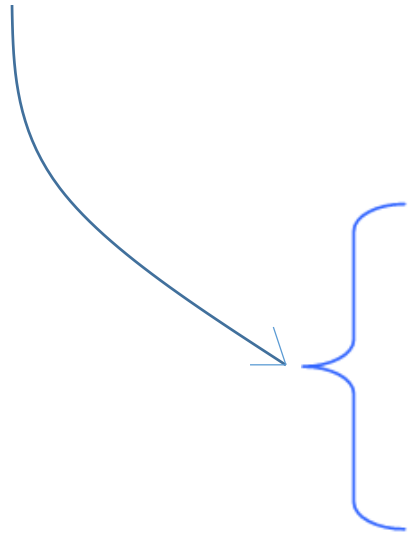
- Some statements are concurrent *and* sequential
- Statements left out:
 - block* → because of its little use
 - assert* → doesn't generate hardware (will be seen in ch. 14)
- *Function call* included for clarity (it's not a statement but rather part of an expression)

Sequential Code

1. Sequential statements

- Used inside `process` and subprograms (i.e., `function`(*) and `procedure`)
- Can implement both `sequential` and `combinational` circuits
- Sequential code allows the use of `variables` (a major feature of VHDL)
- **Fundamental sequential statements**

Process body



```
process ( )  
  if ...  
  case ...  
  loop ...  
  wait ...  
  when ...  
  select ...  
  procedure call  
  (function call)
```

(*) Exception: `wait` cannot be used in `function`

Sequential Code

2. Detecting clock transitions (before we start ...)

- Clock is a **major** component of sequential circuits
(By the way, are all sequential circuits clocked?)
- Clock transitions can be detected as follows:

Option 1:

if	clk'event	and	clk='1'	then	...	–positive	clock	transition
if	clk'event	and	clk='0'	then	...	–negative	clock	transition

Option 2 (recommended):

if	rising_edge	(clk)	then	...	–positive	clock	transition
if	falling_edge	(clk)	then	...	–negative	clock	transition

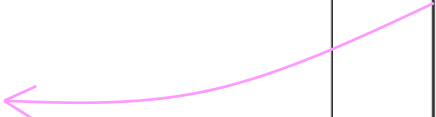
- Option 2 uses the **'event** attribute plus the **To_X01** function (sec. 7.10.4), exploring all **std_ulogic** values
- A good design uses only values **'0'** and **'1'** for clock, but option 2 makes up for bad practices (like the use of **'L'** and **'H'**)
- Option 2 is predefined for **std_ulogic/std_logic**, **bit**, and **boolean**

Sequential Code

3. The *process* statement

```
[label:] process [sensitivity_list] [is]  
    [declarative_part]  
begin  
    sequential_statement_part  
end process [label];
```

```
process ( )  
    if ...  
    case ...  
    loop ...  
    wait ...  
    when ...  
    select ...  
    procedure call  
    (function call)
```



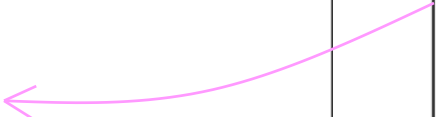
- The **declarative part** can contain declarations of *type*, *constant*, *variable*, ... (declaration of *signal* is **forbidden**)
- The **sequential statements part** can contain only **sequential statements** (listed above), plus **operators** (because these can go in any code)
- The process **awakes** (executes) when a signal in the sensitivity list changes
- The process suspends automatically when it reaches the **end process** keywords, **even if the awaking condition is still true**
- If the process contains a (explicit) **wait** statement, the sensitivity list must be omitted (that **wait** statement is then responsible for awaking the process)

Sequential Code

3. The *process* statement

```
[label:] process [sensitivity_list] [is]  
    [declarative_part]  
begin  
    sequential_statement_part  
end process [label];
```

```
process ( )  
    if ...  
    case ...  
    loop ...  
    wait ...  
    when ...  
    select ...  
    procedure call  
    (function call)
```



Major features of a **process**:

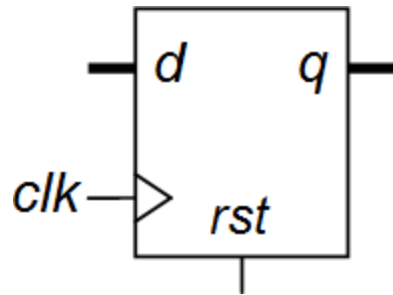
- It gives the code **modularity**
- It is treated (internally) as a **regular program**
- It allows the use of **variables**
- A signal can receive multiple assignments (the last one prevails), so **default values** can be specified

Sequential Code

3. The *process* statement

Example: DFF with asynchronous reset

(without *wait*)



```
process (clk,rst)
begin
  if rst then
    q <= (others => '0');
  elsif rising_edge(clk) then
    q <= d;
  end if;
end process;
```

Example: A simpler DFF

(with *wait*)

```
process
begin
  wait until falling_edge(clk); --or wait until not clk
  q <= d;
end process;
```

Sequential Code

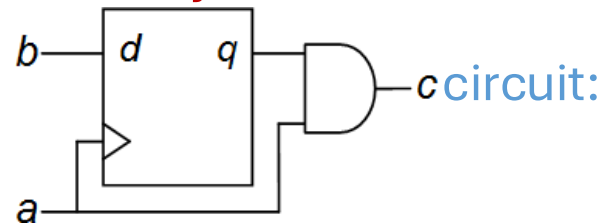
3. The *process* statement

- The *all* keyword:
 - Recommended for the sensitivity list of *combinational circuits*

- Example: Consequences of *incomplete sensitivity list*



–Right code (if
–sequential):
process (all)
begin
 c <= a and b;
end process;



–Written code:**BAD!**
process (a)
begin
 c <= a and b;
end process;

(The compiler might try
to help and include *b* in
the sensitivity list, but
that's not a good design)

Sequential Code

4. The *if* statement

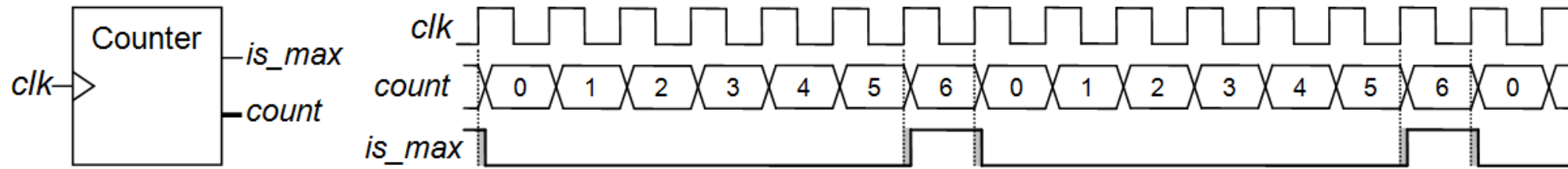
```
[label:] if condition then
    sequential_statements;
[elsif
    sequential_statements;]
[else
    sequential_statements;]
end if [label];
```

- *if* is the sequential counterpart of the concurrent *when* statement
- Like *when*, it has a priority-encoding nature, so use it only when there are few conditions to test
- To enter truth tables in general (in sequential code), use *case* instead
- **IMP.:** Like *when*, incomplete in-out coverage is accepted by the compiler, but it will infer latches (when the output is unregistered)

Sequential Code

4. The *if* statement

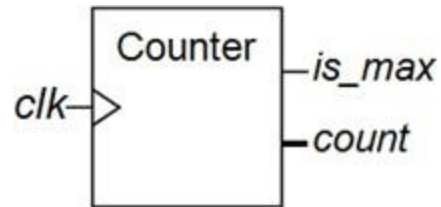
Example:



Sequential Code

4. The *if* statement

Example:



```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

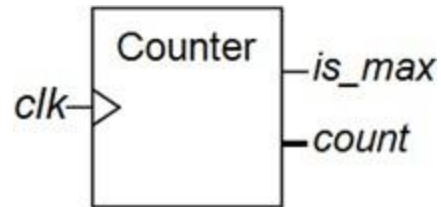
entity counter_with_is_max_flag is
  generic (
    MAX: natural := 6;
    BITS: natural := 3); --not independent (sec. 6.7)
  port (
    clk: in std_logic;
    count: out std_logic_vector(BITS-1 downto 0); is_max: out
    std_logic);
end entity;

architecture rtl of counter_with_is_max_flag is signal i:
  natural range 0 to MAX;
begin
```

Sequential Code

4. The *if* statement

Example:



```
P1: process (clk)
begin
  if rising_edge(clk) then if
    i=MAX-1 then
      is_max<='1'; i<=i+
      1;
    elsif i=MAX then
      is_max<='0';
      i<=0;
    else
      i<=i+1;
    end if;
  end if;
end process;
count<= std_logic_vector(to_
```

```
end architecture;
```

```
P2: process (clk)
begin
  if rising_edge(clk) then

    --Counter:
    if i/=MAX then
      i<= i+1;
    else
      i<=0;
    end if;
    count<=...(same)

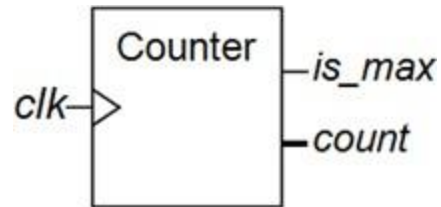
    --Flag generator:
    if i=MAX-1 then
      is_max<='1';
    else
      is_max<='0';
    end if;

  end if;
end process;
```

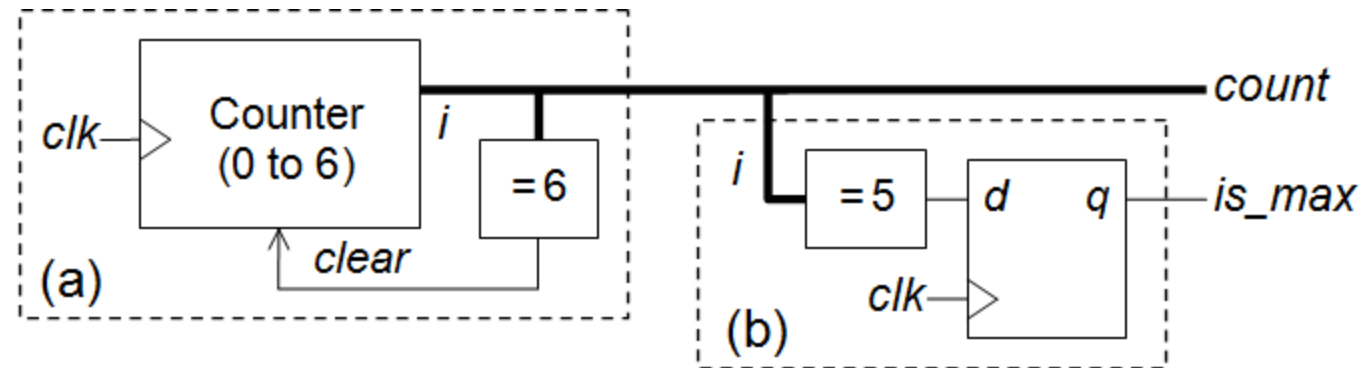
Sequential Code

4. The *if* statement

Example:



Result:



Could we have done better?

- In the example above, some optimization is possible (exercise 13.8)
- But in general, left optimizations (if any) are of little impact

Sequential Code

5. The *case* statement

```
[label] case expression is  
  when choice => assignments;  
  when choice => assignments;  
  when others => assignments;  
end case;
```

```
[label] case expression is  
  when choice => assignments;  
  when choice => assignments;  
  when choice => assignments;  
end case;
```

- *case* is the sequential counterpart of the concurrent *select* statement
- The pros and cons of the two constructions above are similar to those seen for *select*
- Like *select*, it is the proper option for entering *truth tables* in general (in sequential code)
- Like *select*, incomplete in-out coverage is **not** accepted by the compiler, guaranteeing a **latch-free** result
- Like *select*, the choice expressions can employ *to*, *downto*, *|* (means *or*), and *others*; and *equations* are allowed in the selection argument

Sequential Code

5. The *case* statement

Example:

```
signal control: std_logic_vector(3 downto 0);  
...  
case control is  
  when "0000" => action <= A;  
  when "0001" | "1000" | "1111" => action <= B;  
  when others => action <= C;  
end case;
```

- SUV/SLV are the most common vector types in the industry
- Therefore, the syntax ending in “*when others*” is usually the only viable option

Sequential Code

5. The *case* statement

The *unaffected* keyword

- Useful to complete the in-out coverage of *case*
- Use only in *registered* code (see why in *sec. 12.5*)

Example:

```
process (clk)
begin
    if rising_edge(clk) then
        case sel is
            when 0 => y <= a;
            when 1 => y <= b;
            when others => y <= unaffected;
        end case;
    end if;
end process;
```

Sequential Code

5. The *case* statement

The matching *case?* version

- It is the sequential counterpart of *select?*
- It can be used for “don’t care” values at the input

Example:

```
process (all)
begin
  case? x is
    when "1--"    => y <= "1000";
    when "01--"   => y <= "0100";
    when "001-"   => y <= "0010";
    when "0001"   => y <= "0001";
    when others   => y <= "0000";
  end case?;
end process;
```

Sequential Code

6. The *wait* statement

```
[label:] wait;
```

(for simulation)

```
[label:] wait until condition;
```

```
[label:] wait on sensitivity_list;
```

```
[label:] wait for time_expression;
```

(for simulation)

- Four versions, but only 2nd and 3rd are OK for synthesis
- Anyway, not of big help, so *wait* can be ignored for synthesis
- But important for simulation (chapter 18)

Sequential Code

6. The *wait* statement

Example:

–DFF built with “wait until”:

```
process
begin
    wait until not clk;
    if clr then
        q <= '0';
    else
        q <= d;
    end if;
end process;
```

(Notice the absence of sensitivity list when *wait* is used)

Sequential Code

7. The *loop* statement

unconditional loop:

```
[label:] loop  
    sequential_statements;  
end loop [label];
```

while-loop:

```
[label:] while condition loop  
    sequential_statements;  
end loop [label];
```

for-loop:

```
[label:] for identifier in discrete_range loop  
    sequential_statements;  
end loop [label];
```

loop with exit:

```
[loop_label:] [for identifier in discrete_range] loop  
    ...  
    [exit_label:] exit [loop_label] [when condition];  
    ...  
end loop [loop_label];
```

loop with next:

```
[loop_label:] [for identifier in discrete_range] loop  
    ...  
    [next_label:] next [loop_label] [when condition];  
    ...  
end loop [loop_label];
```

Sequential Code

7. The *loop* statement

- *loop* is the sequential counterpart of the concurrent *generate* statement
- There are five options, *for-loop* being by far the most common
- Note that the versions with *exit* and *next* are indeed special cases of *for-loop*

Example with *for-loop*:

```
for i in 0 to 31 loop
    x(i) <= a(i) nand      b(31-i);
end loop;
```

Example with *for-loop* + *exit*:

```
count := 0;
for i in data'range loop
    case data(i) is
        when '1' => count := count + 1;
        when others => exit;
    end case;
end loop;
leading_ones <= count;
```

Sequential Code

8. The sequential *when* and *select* statements

- Introduced in VHDL-2008
- Same syntax seen for the concurrent versions (chapter 10)

Example:

–DFF with synchronous clear:

```
process (clk)
begin
    if rising_edge(clk) then
        q <= '0' when clear else d;
    end if;
end process;
```


Sequential Code

9. *Signal* versus *Variable*

- Variables are a major feature of VHDL
- However, to write code efficiently, it's indispensable to know well their *properties* and their *differences* with respect to *signals*
- To help in that task, we propose the *6 rules* of table 12.1

Sequential Code

9. *Signal* versus *Variable*

Table 12.1
Signal versus variable

Rule/Feature		Signal	Variable (*)
1	Places of declaration	In entity, architecture, package, block, and generate (forbidden in sequential units)	Only in sequential units (i.e., process and subprograms)
2	Scope	Global (can be read and modified anywhere in the architecture statements region)	Local (can be read and modified only inside its own sequential unit)
3	Assignment symbol	<code><=</code> (example: <code>sig <= 255;</code>)	<code>:=</code> (example: <code>var := 255;</code>)
4	Multiple assignments	Allowed only inside sequential units, but be aware of combinational loops	Always fine (because they can only be used inside sequential units anyway)
5	Update	New value available only at the end of the process cycle (after suspension)	Updated immediately (new value ready to be used in the next line of code)
6	Inference of registers	Flip-flops are inferred when an assignment to a signal occurs at the transition of another signal	Flip-flops are inferred when an assignment to a variable occurs at the transition of a signal and this variable affects some other signal

(*) Shared variables not considered (should not be used for synthesis).

Sequential Code

9. *Signal* versus *Variable*

Rule 1: Places of declaration

Signal:

- Declared (created) in the declarative part of entity, architecture, package, block, or generate
- Remember also that all circuit ports are signals

Variable:

- Declared (created) only in sequential units (i.e., process and subprograms)
- The only exception is for *shared variables*, but these should be avoided for synthesis

Rule 2: Scope (places of use)

Signal:

- Global (can be read and modified anywhere, in concurrent and sequential regions)

Variable:

- Always local (can be read and modified only inside the sequential unit where it was created)
- To leave that unit, its value must be passed to a signal or must affect a signal
- The only exception is for shared variables (global), but these should be avoided for synthesis

Sequential Code

9. *Signal* versus *Variable*

Rule 3: Assignment symbol

Signal: $\leq$ Example: `sig  $\leq$  a + b;`

Variable: $:=$ Example: `var := a + b;`

Rule 4: Multiple assignments

Signal:

- In concurrent code, just one assignment is allowed
- In sequential code, multiple assignments are fine (last value prevails), but be aware of combinational loops (sec. 12.12)

Variable:

- Multiple assignments are always fine (because they can only occur in sequential code)
- Last value prevails

Sequential Code

9. *Signal* versus *Variable*

Rule 5: Update

Signal:

- Updated only at the end of the process (when it suspends)

Variable:

- Updated immediately (new value ready to be used in the next line of code)

Rule 6: Inference of registers

Signal:

Flip-flops are inferred when an assignment to a signal occurs at the transition of another signal

Variable:

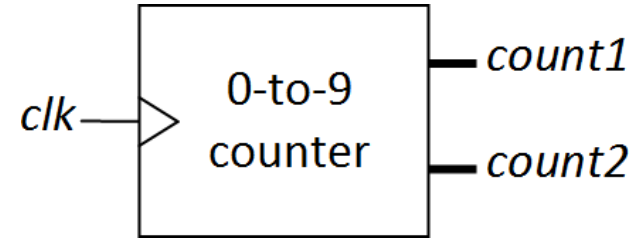
- Flip-flops are inferred when an assignment to a variable occurs at the transition of a signal and this variable affects (directly or indirectly) some other signal

Sequential Code

9. *Signal* versus *Variable*

Example:

We want to build a 0-to-9 free-running counter

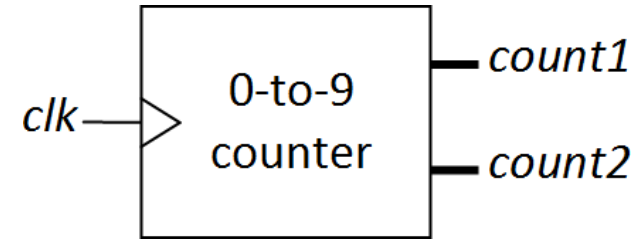


Sequential Code

9. *Signal* versus *Variable*

Example:

We want to build a 0-to-9 free-running counter



--count1 is a port (signal): counter1:

```
process                (clk) begin
  if rising_edge(clk) then count1 <=
    count1 + 1; if count1 = 10 then
      count1 <= 0;
    end if;
  end if;
end process counter1;
```

Counts from 0 to 10

--count2 is another port:

```
counter2: process      (clk)
  variable var: natural range 0 to 10;
begin
  if rising_edge(clk) then
    var := var + 1;
    if var = 10 then
      var := 0;
    end if;
  end if;
  count2 <= var;
end process counter2;
```

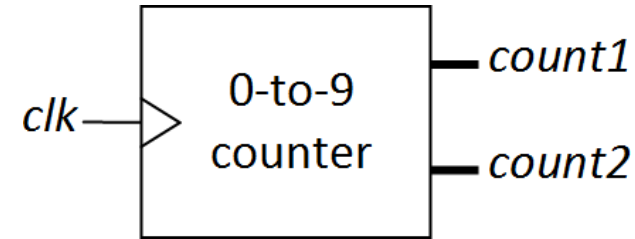
Counts from 0 to 9

Sequential Code

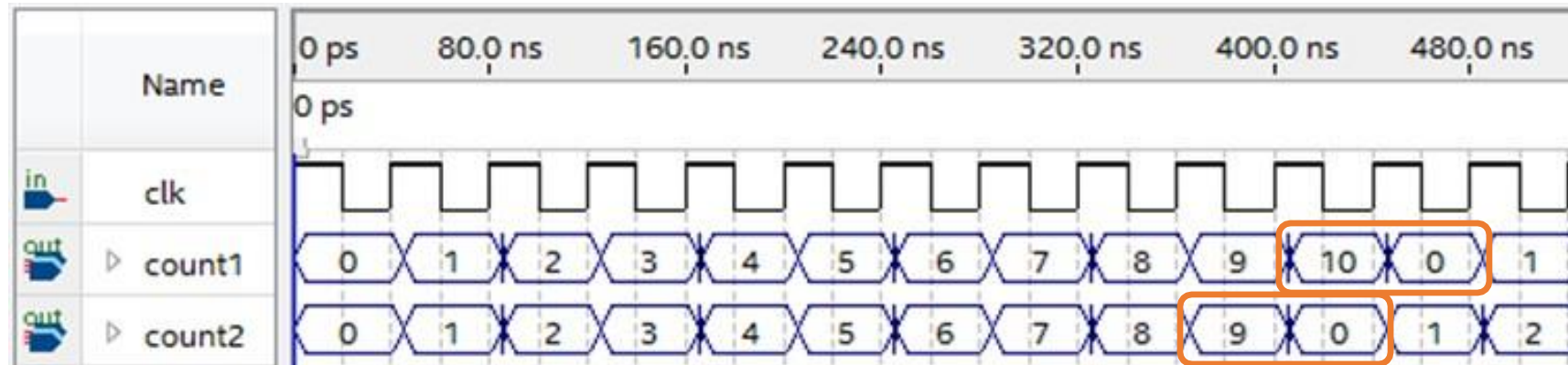
9. *Signal* versus *Variable*

Example:

We want to build a 0-to-9 free-running counter



Simulation results:



Sequential Code

10. More on the update rule of signals and variables

Rule 5: Update

- *Signals* are updated at the end of the process
- *Variables* are updated immediately

Two examples follow:

- 1) Signal x variable in counters
- 2) Odd shift register implementations

Sequential Code

10. More on the update rule of signals and variables

Example 1: Signal x variable in counters

a) *count* is a signal

	<i>Modify-then-read</i> approach:		<i>Read-then-modify</i> approach:
1	P1: process (clk)	1	P2: process (clk)
2	begin	2	begin
3	if rising_edge(clk) then	3	if rising_edge(clk) then
4	count <= count + 1;	4	if count /= 9 then
5	if count=9 then	5	count <= count + 1;
6	count <= 0;	6	else
7	end if;	7	count <= 0;
8	end if;	8	end if;
9	end process;	9	end if;
		10	end process;

Counts up to ?

Counts up to ?

Sequential Code

10. More on the update rule of signals and variables

Example 1: Signal x variable in counters

a) *count* is a signal

	<i>Modify-then-read</i> approach:		<i>Read-then-modify</i> approach:
1	P1: process (clk)	1	P2: process (clk)
2	begin	2	begin
3	if rising_edge(clk) then	3	if rising_edge(clk) then
4	count <= count + 1;	4	if count /= 9 then
5	if count=9 then	5	count <= count + 1;
6	count <= 0;	6	else
7	end if;	7	count <= 0;
8	end if;	8	end if;
9	end process;	9	end if;
		10	end process;

Counts up to 9

Counts up to 9

Sequential Code

10. More on the update rule of signals and variables

Example 1: Signal x variable in counters

b) *i* is a variable

	<i>Modify-then-read</i> approach:		<i>Read-then-modify</i> approach:
1	P3: process (clk)	1	P4: process (clk)
2	variable i: natural range 0 to 10;	2	variable i: natural range 0 to 9;
3	begin	3	begin
4	if rising_edge(clk) then	4	if rising_edge(clk) then
5	i := i + 1;	5	if count /= 9 then
6	if i=10 then	6	i := i + 1;
7	i := 0;	7	else
8	end if;	8	i := 0;
9	end if;	9	end if;
10	count <= i;	10	end if;
11	end process;	11	count <= i;
		12	end process;

Counts up to ?

Counts up to ?

Sequential Code

10. More on the update rule of signals and variables

Example 1: Signal x variable in counters

b) *i* is a variable

	<i>Modify-then-read</i> approach:		<i>Read-then-modify</i> approach:
1	P3: process (clk)	1	P4: process (clk)
2	variable i: natural range 0 to 10;	2	variable i: natural range 0 to 9;
3	begin	3	begin
4	if rising_edge(clk) then	4	if rising_edge(clk) then
5	i := i + 1;	5	if count /= 9 then
6	if i=10 then	6	i := i + 1;
7	i := 0;	7	else
8	end if;	8	i := 0;
9	end if;	9	end if;
10	count <= i;	10	end if;
11	end process;	11	count <= i;
		12	end process;

Counts up to 9

Counts up to 9

Sequential Code

10. More on the update rule of signals and variables

Example 1: Signal x variable in counters

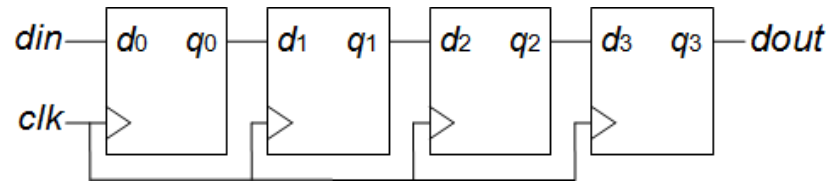
Conclusions and recommendation:

- When using a **signal**: Prefer the *read-then-modify* approach (avoids comparison to the signal's *previous* value)
- When using a **variable**: Either approach is fine
- Recommendation: Use **variables** to implement counters in general

Sequential Code

10. More on the update rule of signals and variables

Example 2: Shift register with odd implementations



```
P1: process (clk)
begin
  if rising_edge(clk) then
    q0 <= din;
    q1 <= q0;
    q2 <= q1;
    dout <= q2;
  end if;
end process P1;
```

4 DFFs (OK)

```
P2: process (clk)
begin
  if rising_edge(clk) then
    dout <= q2;
    q2 <= q1;
    q1 <= q0;
    q0 <= din;
  end if;
end process P2;
```

4 DFFs (OK)

```
P3: process (clk)
  variable q0, q1, q2: std_logic;
begin
  if rising_edge(clk) then
    q0 := din;
    q1 := q0;
    q2 := q1;
    dout <= q2;
  end if;
end process P3;
```

1 DFF (BAD)

```
P4: process (clk)
  variable q0, q1, q2: std_logic;
begin
  if rising_edge(clk) then
    dout <= q2;
    q2 := q1;
    q1 := q0;
    q0 := din;
  end if;
end process P4;
```

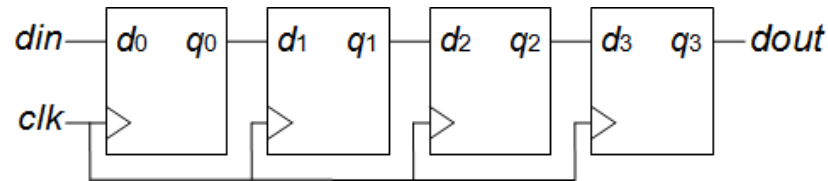
4 DFFs (OK)

Note: These implementations are not recommended (see next slide)

Sequential Code

10. More on the update rule of signals and variables

Example 2: Shift register with odd implementations



Recommended approach (example 12.10):

```
if rising_edge(clk) then
    q <= din & q(0 to NUM_STAGES-2);
end if;
```


Sequential Code

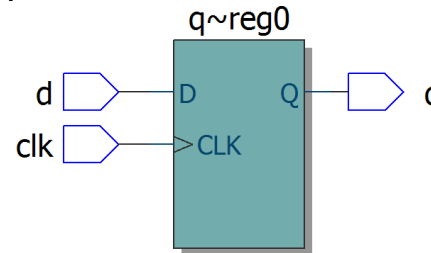
11. More on the inference of registers rule of signals and variables

Rule 6: Inference of registers

A **signal** (or **variable**, if it affects a signal) gets registered if an assignment is made to it at the transition of another signal

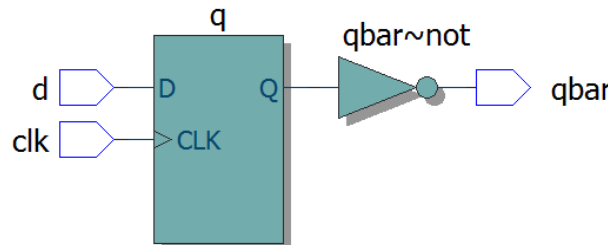
Example: A value is assigned to a **signal** (q) at the transition of another **signal** (clk)

```
if rising_edge(clk) then  
     $q \leq d$ ;  
end if;
```



Example: A value is assigned to a **variable** (q) at the transition of a **signal** (clk), and this variable does affect a **signal** ($qbar$).

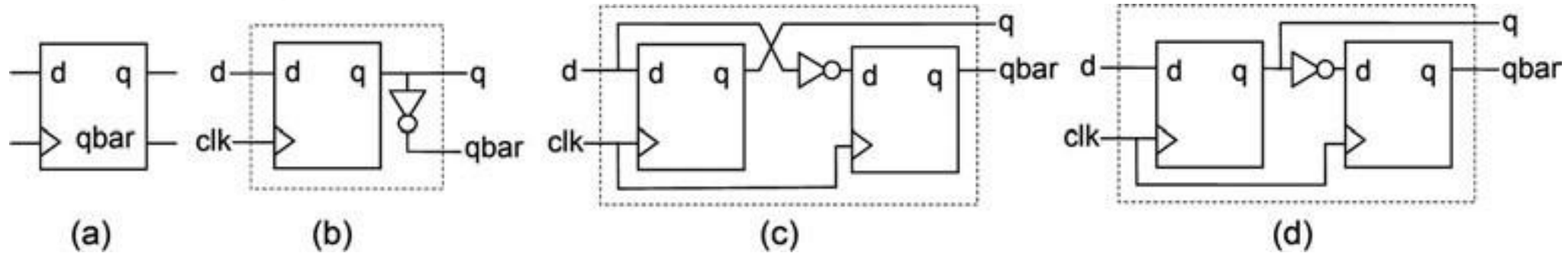
```
if rising_edge(clk) then  
     $q := d$ ;  
end if;  
 $qbar \leq \text{not } q$ ;
```



Sequential Code

11. More on the inference of registers rule of signals and variables

Another example:

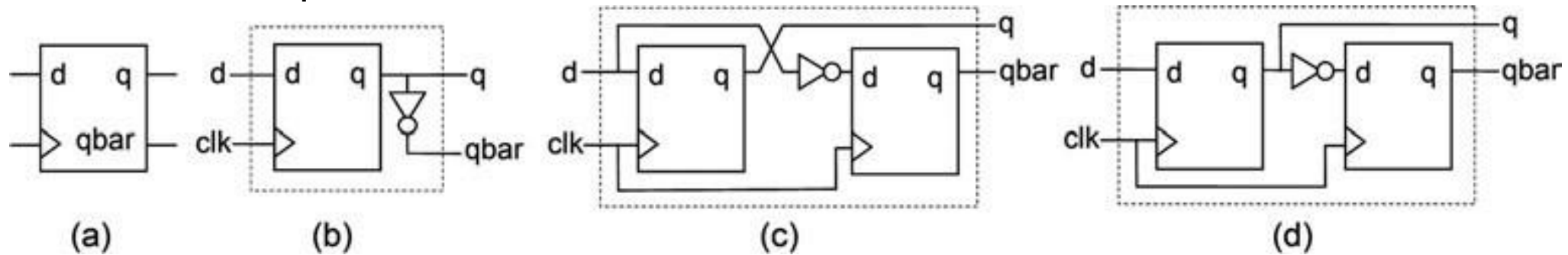


- We want the DFF in (a), but in FPGAs we have to implement it as in (b)
- Which circuit does each code shown next produce?

Sequential Code

11. More on the inference of registers rule of signals and variables

Another example:



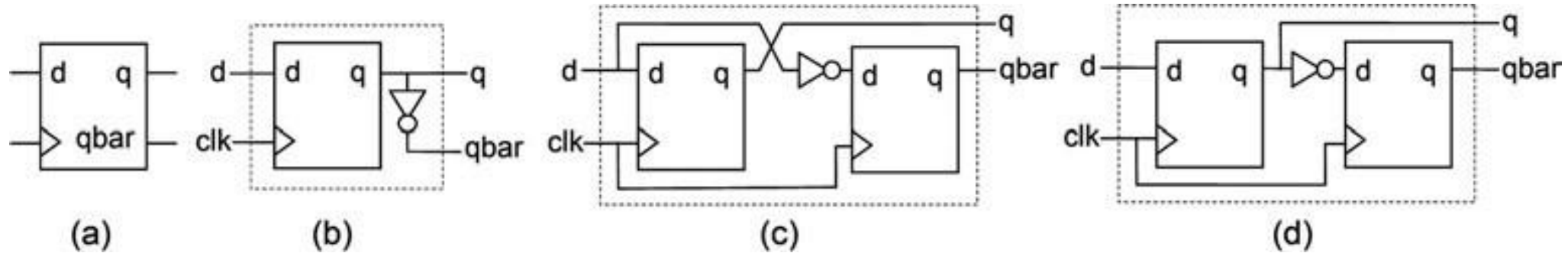
```
entity dff_with_qbar is
  port (
    d,clk:in         std_logic;
    q,qbar:out       std_logic);
end entity;
```

Notice that *q* and *qbar* are **ports** (therefore, **signals**)

Sequential Code

11. More on the inference of registers rule of signals and variables

Another example:



```
P1: process (clk)
begin
  if rising_edge(clk) then
    q <= d;
    qbar <= not d;
  end if;
end process;
```

(c)

```
P2: process (clk)
begin
  if rising_edge(clk) then
    q <= d;
    qbar <= not q;
  end if;
end process;
```

(d)

```
P3: process (clk)
begin
  if rising_edge(clk) then
    q <= d;
  end if;
end process;
qbar <= not q;
```

(b)

- Is (c) functionally equivalent to (b)?
- What about (d)?

Sequential Code

12. Combinational loops

- Can occur when a *signal* is used in a *loop* (sequential code) to try to build a combinational circuit
- Can occur occasionally also with variables (when not explicitly initialized, for example)

Sequential Code

12. Combinational loops

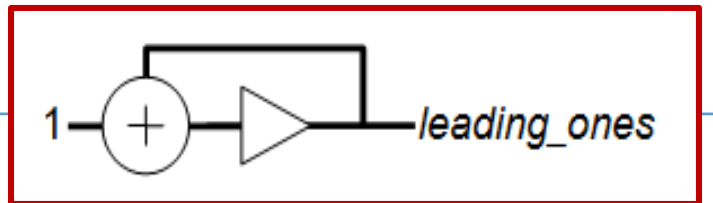
Example: Leading-ones counter

--With a variable (OK):

```
process (all)
  variable count: natural range 0 to BITS;
begin
  count:=0;
  for i in data'range loop
    case data(i) is
      when '1' => count:=count+1;
      when others => exit;
    end case;
  end loop;
  leading_ones<= count;
end process;
```

--With a signal (not OK):

```
process (all)
begin
  leading_ones<= 0;
  for i in data'range loop case
    data(i) is
      when '1' => leading_ones<=leading_ones+1;
      when others => exit;
    end case;
  end loop;
end process;
```



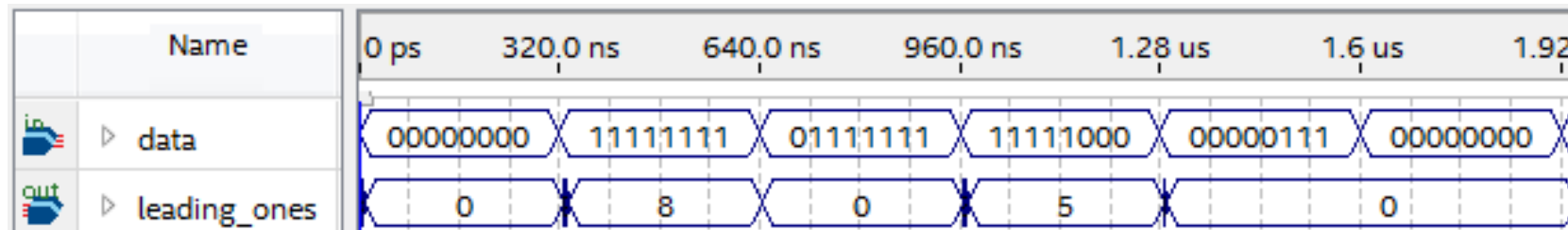
Sequential Code

12. Combinational loops

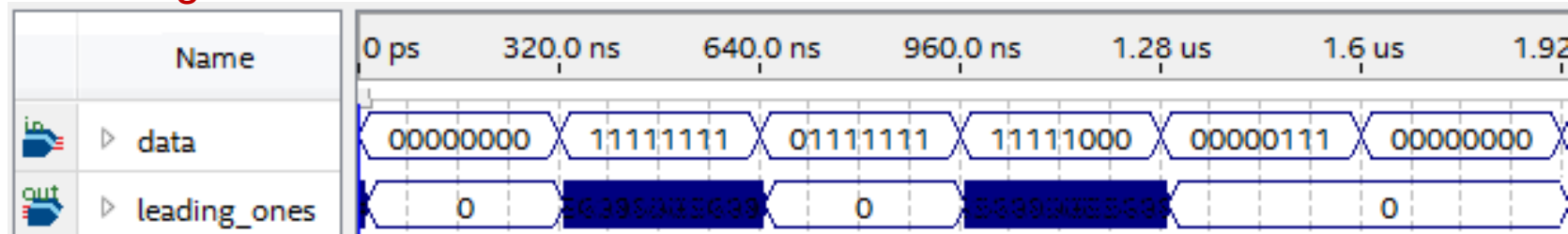
Example: Leading-ones counter

Simulation results (see details in section 12.12):

With variable:



With signal:



Sequential Code

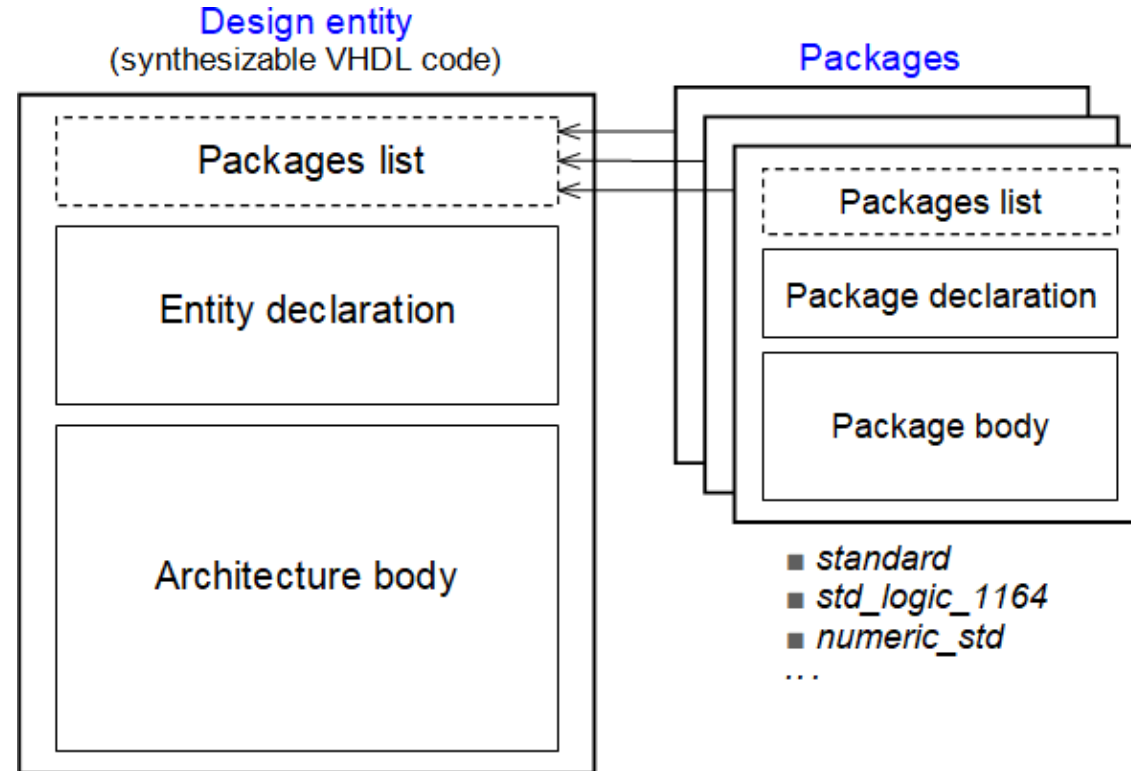
12. Combinational loops

Conclusion and recommendation:

Use always *variables* in recursive equations inside *loops*

Packages and Subprograms

1. Package



- Package = Collection of declarations that are common to a model
- Prevents common solutions from being rewritten
- Helps code organization, simplification, and reusability

Packages and Subprograms

1. Package

Syntax:

```
[libraries/packages list]
package package_name is
    declarative_part
end [package] [package_name];

-----

[package body package_name is
    [subprogram_body]
    [deferred_constant_specification]
end [package body] [package_name]];
```

- **Package declaration:**
Can contain declarations of subprogram, type, constant, signal, variable, package, etc.
- **Package body:**
 - Needed only when the package contains a declaration of **subprogram** or **deferred constant**
 - Must show the subprogram body and/or the constant's value-defining mechanism

Packages and Subprograms

1. Package

Example: A package with **type** and **subprogram** declarations

```
-----  
package example_pkg is  
    type signed_array is      array (1 to 8) of signed(7 downto 0);  
    function negative_edge(signal s:bit) return boolean;  
end package;
```

```
package body example_pkg is
```

???

```
end package body;
```

```
-----
```

Packages and Subprograms

1. Package

Example: A package with **type** and **subprogram** declarations

```
-----  
package example_pkg is  
    type signed_array is array (1 to 8) of signed(7 downto 0);  
    function negative_edge(signal s:bit) return boolean;  
end package;  
  
package body example_pkg is  
    function negative_edge(signal s:bit) return boolean is  
        begin  
            return (s'event and s='0');  
        end function negative_edge;  
end package body;  
-----
```

Packages and Subprograms

2. Package with generics

- VHDL-2008 allows the inclusion of a **generic list** in a package:

```
[libraries/packages list]

package package_name is
    generic (generic_list);
    package_declarative_part
end [package] [package_name];

-----

[package body package_name is
    [subprogram_body]
    [deferred_constant_specification]
end [package body] [package_name];
```

Packages and Subprograms

2. Package with generics

- Such a package is said to be:
 - a) A *generic-mapped package* if a generic map association is included
 - b) An *uninstantiated package* otherwise, i.e.
 - without a *generic map* list or
 - with default values for all generics
- To use the latter, it must first be *instantiated*, using the *new* keyword:

When without a generic list:

```
package package_name is new work.uninstantiated_pkg_name
    generic map (association_list);
```

When with default values:

```
package package_name is new work.uninstantiated_pkg_name
    generic map (default);
```

Packages and Subprograms

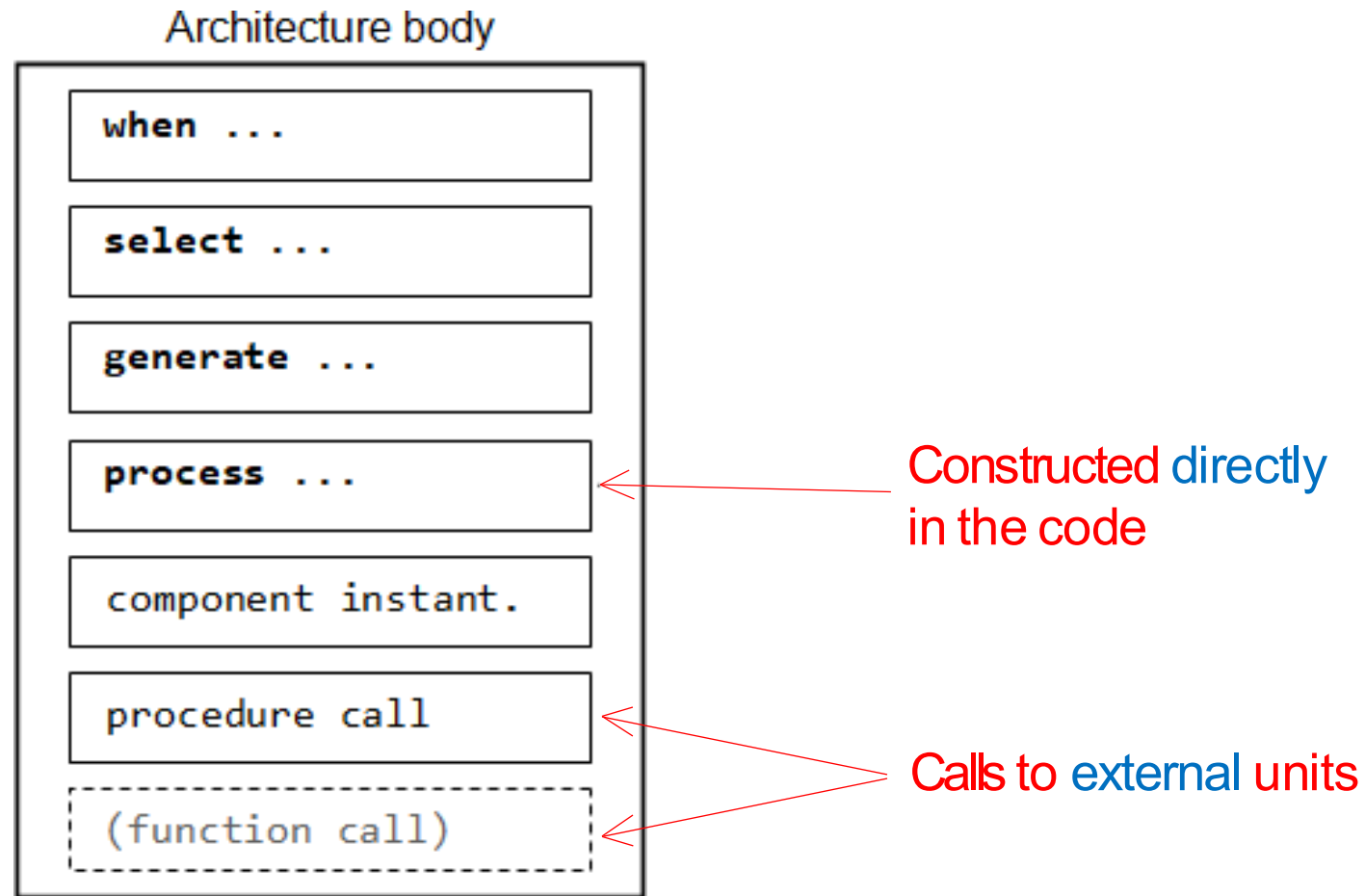
3. Subprograms

Subprogram concepts:

- `function` and `procedure` are called `subprograms`
- `process` and `subprograms` are the only regions of `sequential code`
- But while `process` is for the code proper, `subprograms` are separate units, to solve common problems, hence intended to be reused and often located in libraries (inside packages)
- **Example:** Nearly all predefined operators (chapter 9) are `functions`

Packages and Subprograms

3. Subprograms



Packages and Subprograms

3. Subprograms

Subprogram construction:

- Allowed places: `package`, `entity`, `architecture`, `process`, or another subprogram (`block` not used)
- Most *common* places: `package` and `architecture` (declarative region, of course)
- Statements allowed in `procedure`: All sequential statements (`if`, `case`, `loop`, `wait`, plus sequential versions of `when` and `select`)
- Statements allowed in `function`: All above, except `wait`
- Subprogram `calls` can be made in `both` concurrent and sequential codes

Packages and Subprograms

4. Function

```
[pure|impure] function function_name [(input_list)] return return_type is  
    [declarative_part]  
begin  
    sequential_statement_part  
end [function] [function_name];
```

- It is “**pure**” when it does not rely on external objects, so it always returns the same result when called with the same parameter values
- Don’t use “**impure**” functions for synthesis (and even for simulation, in general)
- A function can receive **any number** of parameters, but must return **exactly one**
- Such parameters can be **constant** (default), **signal**, or **file** (the word **variable** is **forbidden** in the **input_list**)

Packages and Subprograms

4. Function

```
[pure|impure] function function_name [(input_list)] return return_type is
    [declarative_part]
begin
    sequential_statement_part
end [function] [function_name];
```

- The **declarative_part** accepts declarations of **variables** but not of **signals**
- The **statement_part** cannot contain **wait** statements and **component** instantiations
- A function call is always **part of an expression**; for example:

`y <= mean(a,b);`

--*mean* is a function

`y <= a + b;`

--"+" is a function

`if rising_edge(clk) then ...`

--*rising_edge* is a function

Packages and Subprograms

4. Function

Example: Function *ceil_log2*, constructed in a *package*

a) Function construction:

```
-----  
package subprograms_pkg is  
    function ceil_log2 (input: positive) return natural;  
end package;  
  
package body subprograms_pkg is  
    function ceil_log2 (input: positive) return natural is variable result:  
        natural := 0;  
    begin  
        while 2**result < input loop  
            result := result + 1;  
        end loop;  
        return result;  
    end function ceil_log2;  
end package body;  
-----
```

Packages and Subprograms

4. Function

Example: Function *ceil_log2*, constructed in a *package*

b) Function call:

```
use work.subprograms_pkg.all;
```

```
entity test_circuit is
  generic (
    BITS: natural := 8);
  port (
    inp: in  positive range 1 to 2**BITS-1; outp: out
          natural range 0 to BITS);
end entity;
```

```
architecture test_circuit of test_circuit is begin
  outp <= ceil_log2(inp);
end architecture;
```

Packages and Subprograms

5. Procedure

```
procedure procedure_name [(input_output_list)] is
    [declarative_part]
begin
    sequential_statement_part
end [procedure] [procedure_name]
```

- A procedure can receive and return any number of parameters (multi-output problems)
- The **input_output_list** can contain objects of any class (constant, signal, variable, or file)
- Default for mode **in** is constant; default for modes **out** and **inout** is variable
- Like functions, it cannot contain signal declarations and component instantiations
- But contrary to functions, the **wait** statement is allowed

Packages and Subprograms

5. Procedure

```
procedure procedure_name [(input_output_list)] is
    [declarative_part]
begin
    sequential_statement_part
end [procedure] [procedure_name]
```

- Procedures can deal with time (with *wait* statement)
- And can easily deal (*read* and *write*) with files (important for simulation)
- A procedure call is a **standalone statement**; for example:
sort(a1,a2,a3,b1,b2,b3);
divide(dividend,divisor,quotient,remainder);
-sort is a procedure
-divide is a procedure

Packages and Subprograms

5. Procedure

Example: Procedure *divide*, constructed in a *package*

a) Procedure construction:

```
-----  
library ieee;  
use ieee.numeric_std.all;  
  
package subprograms_pkg is  
    procedure divide(  
        dividend, divisor: in unsigned;  
        signal quotient, remainder: out unsigned);  
end package;  
  
package body subprograms_pkg is  
    procedure divide(  
        dividend, divisor: in unsigned;  
        signal quotient, remainder: out unsigned) is begin  
        quotient <= dividend / divisor; remainder <=  
        dividend rem divisor;  
    end procedure;  
end package body;
```


Packages and Subprograms

5. Procedure

Example: Procedure *divide*, constructed in a *package*

b) Procedure call:

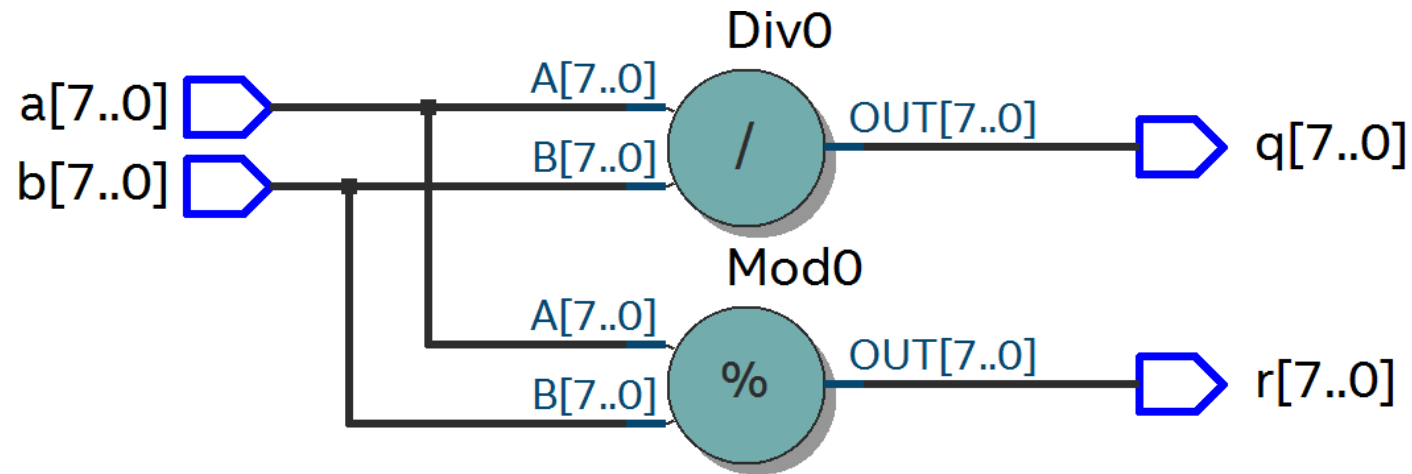
```
-----  
library ieee;  
use ieee.numeric_std.all;  
use work.subprograms_pkg.all;  
  
entity test is  
    generic (  
        BITS: positive := 8);  
    port (  
        a, b: in unsigned(BITS-1 downto 0);  
        q, r: out unsigned(BITS-1 downto 0));  
end entity;  
  
architecture rtl of test is  
begin  
    divide(a, b, q, r);  
end architecture;  
-----
```

Packages and Subprograms

5. Procedure

Example: Procedure *divide*, constructed in a *package*

RTL view:

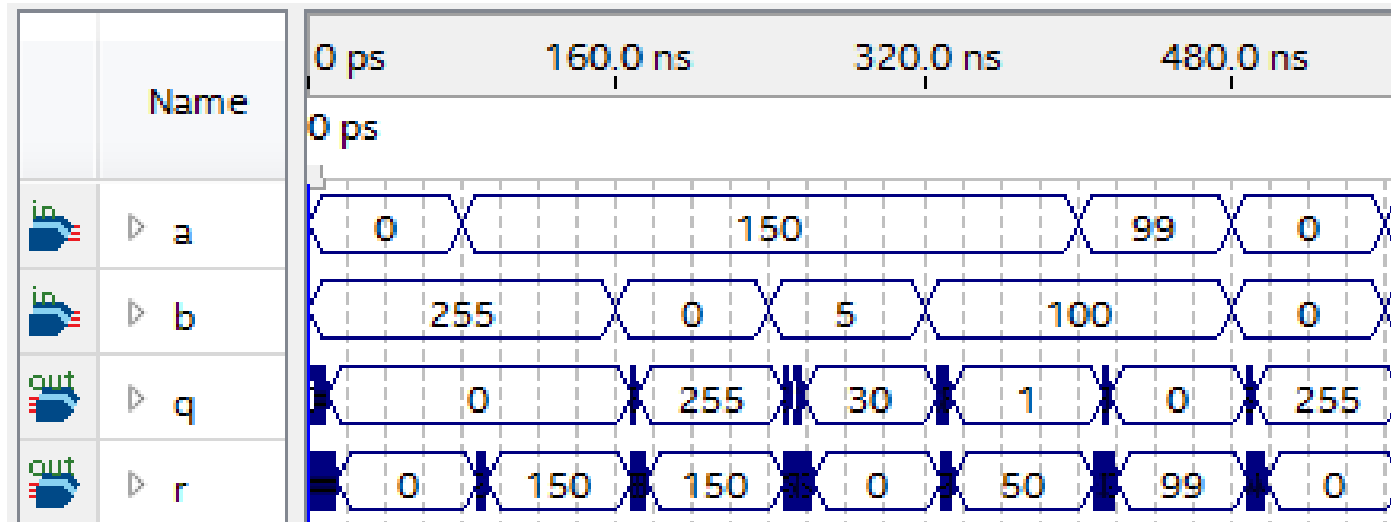


Packages and Subprograms

5. Procedure

Example: Procedure *divide*, constructed in a *package*

Simulation results:



Notes:

- Division-by-zero not checked in the code
- Simulator's default: Division-by-zero saturates the output and makes remainder = divider

Packages and Subprograms

5. Procedure

Example: Procedure *divide*, constructed in a *package*

What we didn't check in the code:

- Division-by-zero
- What table 9.4 says about the “/” (division) operator
- What table 9.4 says about the “*rem*” (remainder) operator

Table 9.4

Arithmetic operators (including output sizes or ranges)

Binary operations							Unary operations		
Left, Right	Outp	+, -	*	/	**	rem, mod	Inp	Outp	-, abs
UNS, UNS		(8)	(11)	(14)	—	(15)	—	—	—

(14) L'length-1 downto 0 (output length = length of input on the left)

(15) R'length-1 downto 0 (output length = length of input on the right)

Packages and Subprograms

6. Subprogram with generics and generic subprograms

- VHDL-2008 allows the inclusion of a **generic list** in a **subprogram**, resulting in the (complete) syntaxes below:

```
[pure|impure] function function_name  
    [generic (generic_list)]  
    [[parameter] (input_list)] return return_type is  
    [function_declarative_part]  
begin  
    sequential_statement_part  
end [function] [function_name];
```

```
procedure procedure_name  
    [generic (generic_list)]  
    [[parameter] (input_output_list)] is  
    [procedure_declarative_part]  
begin  
    sequential_statement_part  
end [procedure] [procedure_name];
```

Packages and Subprograms

6. Subprogram with generics and generic subprograms

- It allows also the inclusion of **subprograms** in **generic lists**, as in the example below:

```
-----  
entity generic_adder is  
    generic (  
        type data_type;  
        function add (signal in1, in2: data_type) return data_type);  
    port (  
        ...);  
end entity generic_adder;  
  
architecture behavioral of generic_adder is  
    ...  
end architecture behavioral;  
-----
```

- The usage rules, with examples, are detailed in **section 14.6**

Packages and Subprograms

7. The *assert* statement

Syntax:

```
[label:] assert condition [report message] [severity level];
```

- It does **not** infer hardware
- Instead, it is used for **checking** the design
- Particularly helpful for:
 - Checking subprogram parameters (input and output sizes, for example)
 - Simulation results

Packages and Subprograms

7. The *assert* statement

Syntax:

```
[label:] assert condition [report message] [severity level];
```

- It does **not** infer hardware
- Instead, it is used for **checking** the design
- Particularly helpful for:
 - Checking subprogram parameters (input and output sizes, for example)
 - Simulation results
- Reports message when the condition is **false**
- Severity levels: **note**, **warning**, **error**, **failure** (last two usually halt synthesis/simulation)

Packages and Subprograms

7. The *assert* statement

- Syntax:

```
[label:] assert condition [report message] [severity level];
```

- The reported message can be *static* or *dynamic*
- For *dynamic* messages (involving *time*), the *to_string* function is used (details in *sec. 14.8*)
- Example of static message:
 - **assert** address < 2**ADDRESS_WIDTH **report**
"Unexpected address value." **severity** failure;
- Example of dynamic message (see details in *sec. 14.8*):
 - **assert** actual_val = expected_val
 - **report** "Actual value does not match expected value at t=" & to_string(now) & "."
 - **severity** error;

Packages and Subprograms

8. The *report* statement

Syntax:

```
report message;
```

- Can report a conditional message (used with the `assert` statement)
- Or an unconditional messages (used alone)

Example:

```
report "Test was successful.";
```